

Stefan Heiss

Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik

Datensicherheit

02 – Symmetrische Verfahren

Version vom 16. Mai 2025

Vorwort

Liebe Studierende,

im ersten Studienheft 01 zum Modul Datensicherheit haben sie die Gefährdungen und Schutzziele sowie grundlegende Schutzmechanismen kennengelernt, die bei dem Betrieb und der Nutzung eines IT-Systems zu beachten sind. Die dort bei der Darstellung der Schutzmechanismen genannten kryptographischen Algorithmenklassen

- *symmetrische Verschlüsselungsverfahren,*
- *Hashfunktionen* und
- *MAC-Verfahren*

werden in diesem Studienheft 02 detailliert behandelt.

Sie werden die Konstruktionsweise aktuell im Einsatz befindlicher Algorithmen kennenlernen. Neben den bereits im Studienheft 01 angesprochenen Einsatzgebieten, werden einige Anwendungen der Algorithmen im Detail behandelt. Schließlich lernen Sie, wie Implementierungen der Algorithmen in einer aktuellen Java-Laufzeitumgebung zur Verfügung gestellt werden und innerhalb eines Java-Programms genutzt werden können.

Inhaltsverzeichnis

1	Stromverschlüsselungsverfahren (Stream Cipher)	1
1.1	Grundlagen	2
1.2	Schlüsselstrom vs. One-Time-Pad	8
1.3	Nonces zur Initialisierung eines Schlüsselstromgenerators	9
1.4	ChaCha20	11
1.5	Cipher-Instanzen: Verschlüsselungsalgorithmen in Java-Laufzeitumgebungen	19
2	Blockverschlüsselungsverfahren (Block Cipher)	21
2.1	Grundlagen	22
2.2	Padding-Verfahren	23
2.3	Betriebsmodi	23
2.4	Konstruktionsprinzipien von Blockverschlüsselungsverfahren	35
2.5	DES und Triple-DES	36
2.6	Meet-in-the-Middle-Angriffe	38
2.7	AES	41
3	Mathematischer Exkurs: Endliche Körper $\mathbb{F}_{p^n}$	45
4	Hashfunktionen	53
4.1	Grundlagen	54
4.2	MessageDigest-Instanzen: Hashfunktionen in Java-Laufzeitumgebungen	56
4.3	Anwendungsbeispiele	58
4.4	Brute-Force-Angriffe auf Hashfunktionen	63
4.4.1	Brute-Force-Urbildsuche	63
4.4.2	Brute-Force-Kollisionssuche	65
4.5	Konstruktionsverfahren von Hashfunktionen	69

5	MAC-Verfahren	71
5.1	HMAC	72
5.2	CMAC, CBC-MAC	74

Stromverschlüsselungsverfahren (Stream Cipher)

1

1.1	Grundlagen	2
1.2	Schlüsselstrom vs. One-Time-Pad	8
1.3	Nonces zur Initialisierung eines Schlüsselstromgenerators	9
1.4	ChaCha20	11
1.5	Cipher-Instanzen: Verschlüsselungsalgorithmen in Java- Laufzeitumgebungen	19

Wie bereits im Studienheft 01 mehrfach erwähnt wird grundsätzlich zwischen *symmetrischen* und *asymmetrischen* Verschlüsselungsverfahren unterschieden.

In diesem Studienheft wenden wir uns den symmetrischen Verschlüsselungsverfahren zu. Die asymmetrischen Verfahren werden in Studienheft 03 behandelt.

Bei den symmetrischen Verschlüsselungsverfahren wird zur Verschlüsselung der vollständige Schlüssel k benötigt. Dieser wird bei symmetrischen Verschlüsselungsverfahren in der englischsprachigen Literatur als *Secret Key* bezeichnet.

Die symmetrischen Verschlüsselungsverfahren lassen sich in zwei grundlegend verschiedene Klassen einteilen, nämlich in die Strom- und die Blockverschlüsselungsverfahren, die in diesem beziehungsweise im nächsten Kapitel behandelt werden. Die Bezeichnungen aus Studienheft 01, Abschnitt 4.2 (insbesondere (4.1)–(4.4)) behalten ihre Gültigkeit und werden für verschiedene Verfahren weiter verfeinert.

1.1 Grundlagen

Stromverschlüsselungsverfahren (Stream Cipher)

Bei einem *Stromverschlüsselungsverfahren* wird eine Nachricht zeichenweise verschlüsselt. Die Menge der Nachrichten $\mathcal{M}$ ist dabei durch die Menge der endlichen Folgen von Elementen aus einer *Zeichenmenge* $\mathcal{Z}$ definiert:

$$\mathcal{M} = \mathcal{Z}^{>0} := \bigcup_{l \in \mathbb{N}} \mathcal{Z}^l = \{(m_1, m_2, \dots, m_l) \mid m_i \in \mathcal{Z}, l \in \mathbb{N}\}$$

Eine Verschlüsselungsfunktion E_k eines Stromverschlüsselungsverfahrens überführt Folgen der Länge l in Folgen gleicher Länge l (für alle $l \in \mathbb{N}$) und lässt sich darüber hinaus zeichenweise beschreiben, d. h., es gibt Funktionen

$$E^{(i)} : \mathcal{K} \times \mathcal{Z}^i \rightarrow \mathcal{Z} \quad \text{für alle } i \in \mathbb{N}$$

mit:

$$\begin{aligned} E_k((m_1, m_2, \dots, m_l)) \\ = (E^{(1)}(k, (m_1)), E^{(2)}(k, (m_1, m_2)), \dots, E^{(l)}(k, (m_1, m_2, \dots, m_l))) \end{aligned}$$

In die Berechnung des i -ten Zeichens des Geheimtextes

$$c_i = E^{(i)}(k, (m_1, m_2, \dots, m_i))$$

können neben dem Schlüssel und dem i -ten Zeichen des Klartextes auch die vorherigen Zeichen einfließen. Die nachfolgenden Zeichen und die Gesamtlänge der Nachricht sind aber ohne Belang.

Bezüglich der Variablen m_i muss $E^{(i)}$ eine bijektive Funktion sein, deren Inverse zur Entschlüsselung benötigt wird.

Eine Verschlüsselung einer l Zeichen langen Nachricht ist in Abbildung 1.1 dargestellt. Wird $E^{(i)}$ als Funktion interpretiert, die m_i zu c_i verschlüsselt, so sind neben k auch $m_1, m_2, \dots, m_{i-1}$ als Parameter zur Definition von $E^{(i)}$ aufzufassen. Diese Interpretationsmöglichkeit ist durch die gestrichelt gezeichneten Pfeile in Abbildung 1.1 angedeutet.

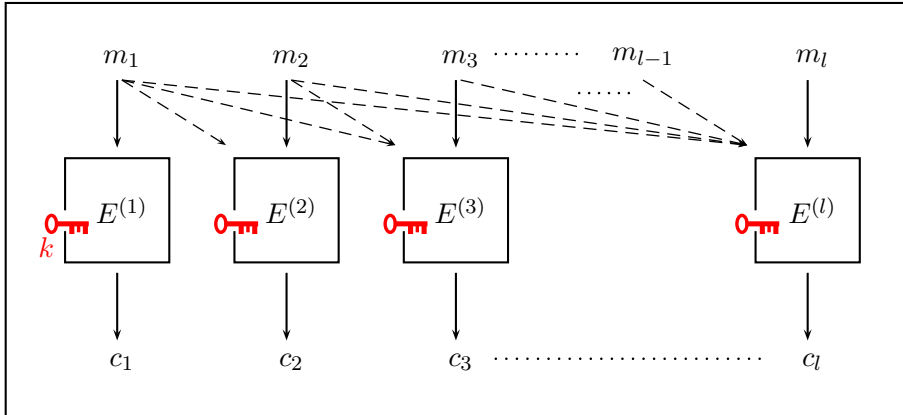


Abb. 1.1 Stromverschlüsselungsverfahren

Beispiel 1.1: Monoalphabetisches Verschlüsselungsverfahren

Das monoalphabetische Verschlüsselungsverfahren (vgl. mit Studienheft 01, Beispiel 1.3) ist ein Stromverschlüsselungsverfahren mit:

$$\mathcal{Z} = \mathbb{Z}_{26}$$

$$\mathcal{K} = \Sigma_{\mathbb{Z}_{26}} = \{k : \mathbb{Z}_{26} \rightarrow \mathbb{Z}_{26} \mid k \text{ ist bijektiv}\}$$

$$c_i = E^{(i)}(k, (m_1, m_2, \dots, m_i)) = k(m_i)$$

Zur Berechnung des Geheimtextzeichens c_i wird neben dem Schlüssel k nur das entsprechende Klartextzeichen m_i benötigt. Das monoalphabetische Verschlüsselungsverfahren ist daher ein Beispiel für ein synchrones Stromverschlüsselungsverfahren entsprechend der folgenden Definition. ▶

Synchrone Stromverschlüsselungsverfahren (Synchronous Stream Cipher)

Ist die Verschlüsselung des i -ten Zeichens für jedes $i \in \mathbb{N}$ durch eine Funktion

$$E^{(i)} : \mathcal{K} \times \mathcal{Z} \rightarrow \mathcal{Z}$$

bestimmt, so wird das Stromverschlüsselungsverfahren *synchron* genannt.

Es gilt dann:

$$E_k((m_1, m_2, \dots, m_l)) = (E^{(1)}(k, m_1), E^{(2)}(k, m_2), \dots, E^{(l)}(k, m_l))$$

Bei der Darstellung eines synchronen Stromverschlüsselungsverfahrens entfallen die in Abbildung 1.1 gestrichelt gezeichneten Pfeile und es ergibt sich die einfache Darstellung in Abbildung 1.2.

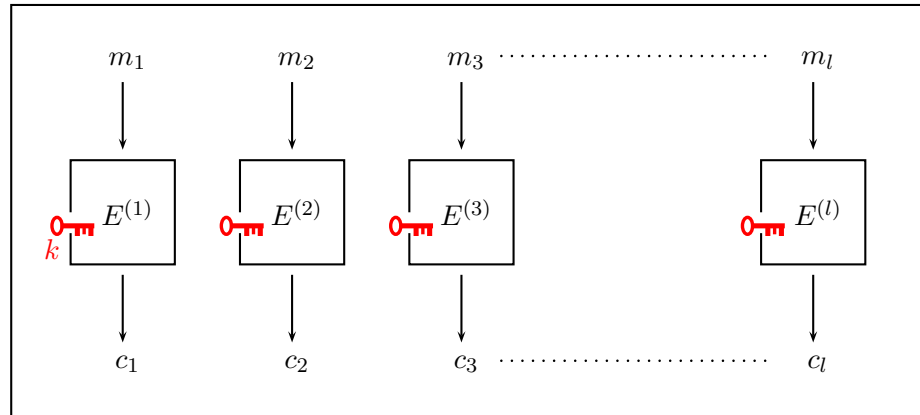


Abb. 1.2 Synchrones Stromverschlüsselungsverfahren

Kommentar Beachten Sie bitte, dass bei einem synchronen Stromverschlüsselungsverfahren die Zeichen m_i einer Klartextnachricht gleichzeitig, also *synchron*, berechnet werden können. ◀

Beispiel 1.2: Monoalphabetisches Verschlüsselungsverfahren

Wie bereits in Beispiel 1.1 erwähnt ist das monoalphabetische Verschlüsselungsverfahren ein synchrones Stromverschlüsselungsverfahren. Die in der Beschreibung von synchronen Stromverschlüsselungsverfahren auftretenden Funktionen $E^{(i)}$ sind in diesem Fall für alle $i \in \mathbb{N}$ gleich:

$$E^{(i)} : \mathcal{K} \times \mathcal{Z} \rightarrow \mathcal{Z}, \quad E^{(i)}(k, m_i) = k(m_i) \quad \left(\mathcal{Z} = \mathbb{Z}_{26}, \mathcal{K} = \Sigma_{\mathbb{Z}_{26}} \right)$$

Frage 1

Das in Studienheft 01, Beispiel 1.4 beschriebene polyalphabetische Verschlüsselungsverfahren ist ein synchrones Stromverschlüsselungsverfahren. Finden Sie nach einer Identifikation der Buchstabenmenge $\{A, B, \dots, Z\}$ mit $\mathbb{Z}_{26}$ die Abbildungsvorschrift für $E^{(i)}$ für $i \in \mathbb{N}$.

Die Beschreibung eines Stromverschlüsselungsverfahrens mit Funktionen $E^{(i)} : \mathcal{K} \times \mathcal{Z}^i \rightarrow \mathcal{Z}$ ist sehr allgemein gehalten. In der Praxis treten üblicherweise zustandsbehaftete Verfahren auf, deren Zustand sich von Zeichen zu Zeichen und eventuell auch in Abhängigkeit vom jeweiligen Klartextzeichen m_i ändert. Die Funktionen $E^{(i)}$ hängen dann von dem jeweiligen Zustand ab. Für eine formale Beschreibung eines solchen Verfahrens, kann die eingangs gegebene allgemeine Definition für ein Stromverschlüsselungsverfahren folgendermaßen abgeändert werden:

Zustandsabhängige Stromverschlüsselungsverfahren

Gegeben seien eine endliche Menge $\mathcal{S}$ von Zuständen sowie Abbildungen:

$$s_0 : \mathcal{K} \rightarrow \mathcal{S}$$

$$s : \mathcal{S} \times \mathcal{Z} \rightarrow \mathcal{S}$$

$$E : \mathcal{S} \times \mathcal{Z} \rightarrow \mathcal{Z}$$

Vor der Verschlüsselung des ersten Zeichens m_1 mit der Verschlüsselungsfunktion E wird in Abhängigkeit vom Schlüsselwert k mit s_0 ein Startzustand festgelegt. Vor der Verschlüsselung jedes weiteren Zeichens mit der Verschlüsselungsfunktion E wird mittels s jeweils eine Zustandsänderung bewirkt. Die Verschlüsselung von $(m_1, m_2, \dots, m_l)$ zu $(c_1, c_2, \dots, c_l)$ wird somit durch folgende Rekursion beschrieben:

$$\begin{aligned} \sigma_1 &:= s_0(k) & c_1 &:= E(\sigma_1, m_1) \\ \sigma_2 &:= s(\sigma_1, m_1) & c_2 &:= E(\sigma_2, m_2) \\ \sigma_3 &:= s(\sigma_2, m_2) & c_3 &:= E(\sigma_3, m_3) \\ &\vdots & &\vdots \\ \sigma_l &:= s(\sigma_{l-1}, m_{l-1}) & c_l &:= E(\sigma_l, m_l) \end{aligned}$$

Für jeden festen Wert σ in der ersten Variablen muss E bezüglich der zweiten Variablen $m \in \mathcal{Z}$ eine bijektive Funktion sein, deren Inverse zur Entschlüsselung benötigt wird.

Mit den oben eingeführten Bezeichnungen ist die Verschlüsselung von $(m_1, m_2, \dots, m_l)$ in Abbildung 1.3 dargestellt.

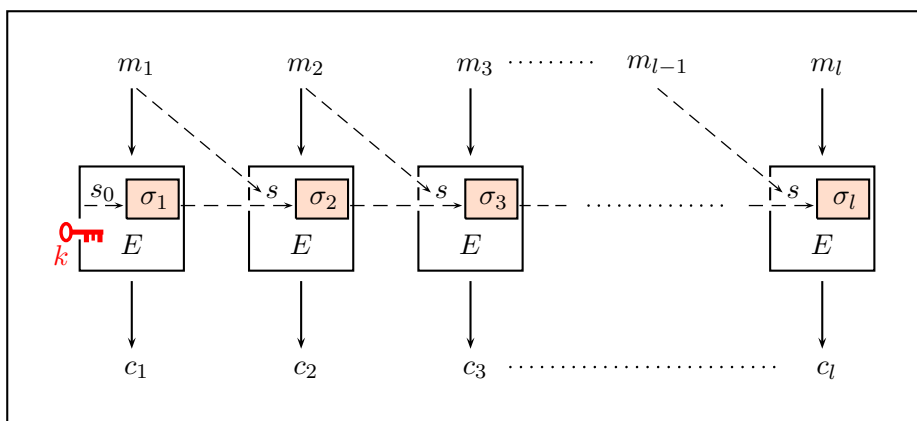


Abb. 1.3 Zustandsabhängiges Stromverschlüsselungsverfahren

Kommentar Typischerweise besteht die Nachrichtenmenge $\mathcal{M}$ bei einem Stromverschlüsselungsverfahren aus der Menge der Bitfolgen ($\mathcal{Z} = \mathbb{Z}_2$) oder der Menge der Bytefolgen

($\mathcal{Z} = \mathbb{Z}_2^8$). In historischen Verfahren (siehe Studienheft 01, Kapitel 1) und in Beispielaufgaben tritt auch noch $\mathcal{Z} = \mathbb{Z}_{26}$ als Stellvertretermenge für die Buchstabenmenge $\{A, B, \dots, Z\}$ auf.

In allen hier genannten Fällen ist eine Addition auf $\mathcal{Z}$ definiert. Für $\mathcal{Z} = \mathbb{Z}_2$ und $\mathcal{Z} = \mathbb{Z}_{26}$ ist dies die allgemein auf $\mathbb{Z}_n$ definierte Addition.

Für $\mathcal{Z} = \mathbb{Z}_2^8$ ist dies die Addition von Vektoren des 8-dimensionalen Vektorraums über dem Körper $\mathbb{Z}_2$, die der bitweisen XOR-Operation von Bytewerten entspricht. (Die Identifikation von Elementen aus $\mathbb{Z}_2^8$ mit den Werten eines Bytes wurde bereits im Studienheft 01 am Ende des Abschnitts 4.2 in (4.5) formal beschrieben.) ◀

Wichtig Im Folgenden wird immer vorausgesetzt, dass auf der Menge der Ziffern $\mathcal{Z}$ eine Addition definiert ist, auch ohne dies explizit zu erwähnen. ◀

Additive zustandsabhängige Stromverschlüsselungsverfahren

Zustandsabhängige Stromverschlüsselungsverfahren für die es eine Funktion

$$S : \mathcal{S} \rightarrow \mathcal{Z}$$

gibt, sodass für die Verschlüsselungsfunktion

$$E(\sigma, m) = S(\sigma) + m$$

für alle $\sigma \in \mathcal{S}$ und $m \in \mathcal{Z}$ gilt, wird *additiv* genannt.

Die Funktionen s_0 , s und S bilden zusammen einen *Schlüsselstromgenerator* und die Folge der Zeichen

$$z_i := S(\sigma_i) = S(s(\sigma_{i-1}, m_{i-1}))$$

bilden den durch k und die Nachrichtenzeichen $m_1, m_2, \dots$ definierten *Schlüsselstrom*.

Mit den oben eingeführten Bezeichnungen ist die Verschlüsselung von $(m_1, m_2, \dots, m_l)$ in Abbildung 1.4 dargestellt.

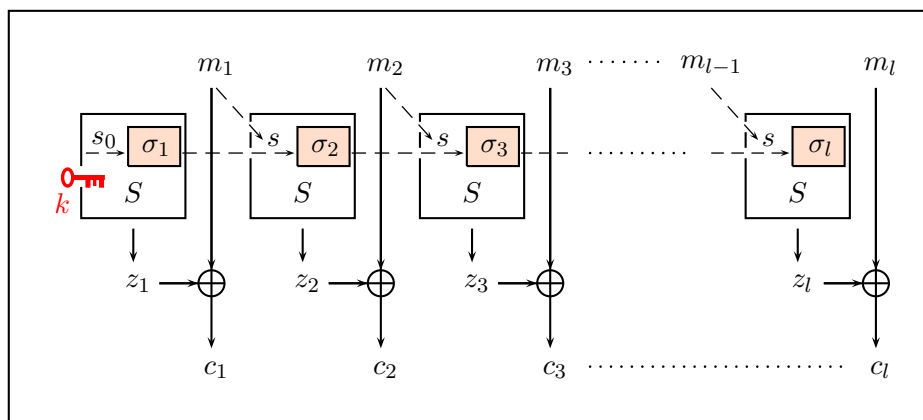


Abb. 1.4 Additives zustandsabhängiges Stromverschlüsselungsverfahren

Einen wichtigen Spezialfall der additiven zustandsabhängigen Stromverschlüsselungsverfahren bilden die Verfahren, bei denen die Berechnung des Zustands und damit des Schlüsselstroms von der Klartextnachricht nicht abhängt:

Synchrone additive Stromverschlüsselungsverfahren

Ein additives zustandsabhängiges Stromverschlüsselungsverfahren wird als *synchrone* additives Stromverschlüsselungsverfahren bezeichnet, falls die Übergangsfunktion nicht von den Zeichen der Klartextnachricht abhängt, d. h., falls gilt:

$$\sigma_{i+1} = s(\sigma_i) \quad \text{für alle } i \in \mathbb{N}$$

mit einer Funktion

$$s : \mathcal{S} \rightarrow \mathcal{S}$$

Der durch die Funktionen s_0 , s und S definierte Schlüsselstromgenerator erzeugt den Schlüsselstrom:

$$z_i = S(s^{i-1}(s_0(k))) \quad (i \in \mathbb{N})$$

Mit der oben eingeführten Bedingung vereinfacht sich die in Abbildung 1.4 dargestellte Verschlüsselung von $(m_1, m_2, \dots, m_l)$ zu der in Abbildung 1.5 dargestellten.

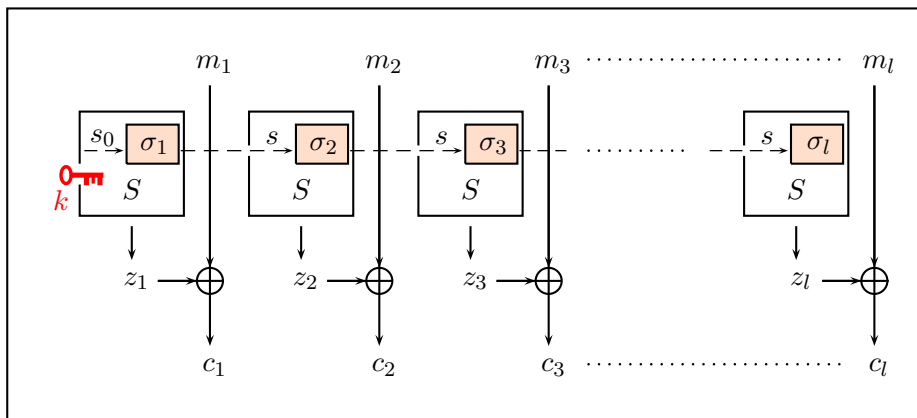


Abb. 1.5 Synchrones additives Stromverschlüsselungsverfahren

Wichtig Bei einem synchronen additiven Stromverschlüsselungsverfahren hängt der vom Schlüsselstromgenerator erzeugte Schlüsselstrom nicht von der Klartextnachricht ab und kann deshalb schon im Voraus berechnet werden (s. Abb. 1.6). Liegt die Klartextnachricht zu einem späteren Zeitpunkt vor, so müssen deren Zeichen nur noch mit den Zeichen des Schlüsselstroms addiert werden. Speziell im Fall von $\mathcal{Z} = \mathbb{Z}_2$ oder $\mathcal{Z} = \mathbb{Z}_2^8$ sind hierzu nur noch bitweise XOR-Operationen auszuführen (s. Abb. 1.7). ◀

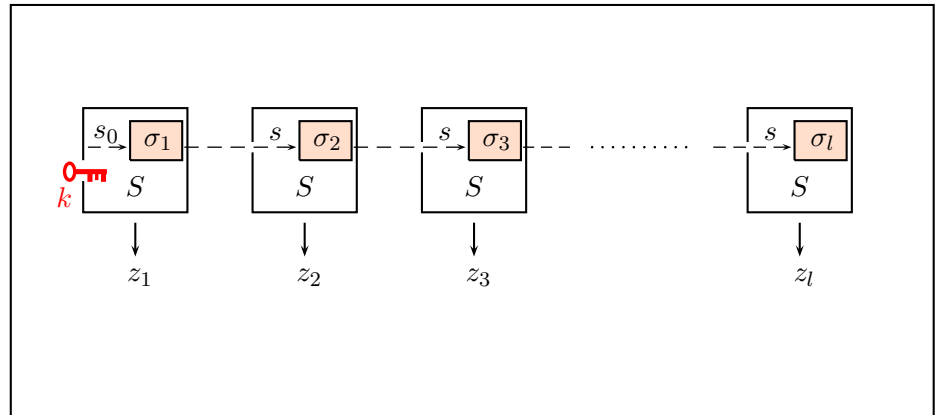
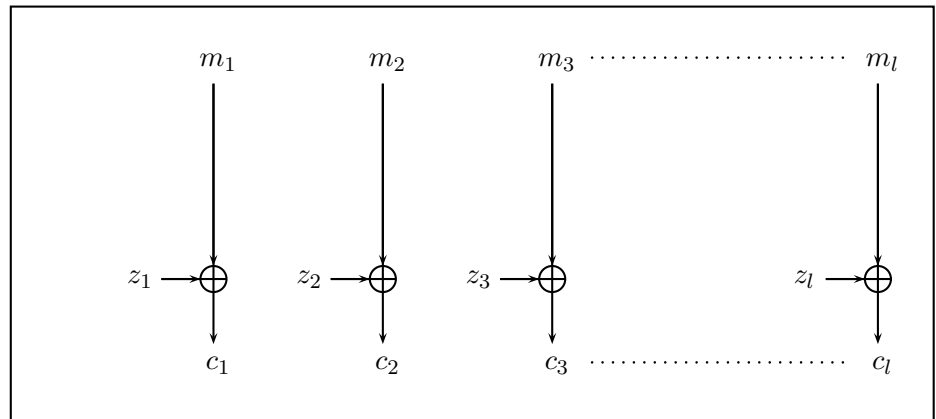


Abb. 1.6 Schlüsselstromgenerator eines synchronen additiven Stromverschlüsselungsverfahrens

Abb. 1.7 Verschlüsselung der Klartextnachricht $(m_1, m_2, \dots, m_l)$ durch Addition mit dem Schlüsselstrom $(z_1, z_2, \dots, z_l)$

1.2 Schlüsselstrom vs. One-Time-Pad

Das bereits in Studienheft 01, Abschnitt 1.3 besprochene One-Time-Pad-Verschlüsselungsverfahren kann als spezielles synchrones additives Stromverschlüsselungsverfahren angesehen werden, bei dem die Schlüsselstromzeichen den Zeichen des Schlüssels k direkt entnommen werden.

Frage 2

Gegeben sei eine Ziffernmenge $\mathcal{Z}$, für deren Elemente eine Addition definiert ist. Geben Sie eine formale Beschreibung des One-Time-Pad-Verfahrens über $\mathcal{Z}$ für Nachrichten einer maximalen Länge $L \in \mathbb{N}$ an. Geben Sie hierzu eine geeignete Menge $\mathcal{S}$ und geeignete Funktionen s_0 , s und S zur Beschreibung des Schlüsselstromgenerators an.

(Beachten Sie, dass der Schlüsselraum durch

$$\mathcal{K} = \mathcal{Z}^L$$

definiert ist.)

Der von einem Schlüsselstromgenerator erzeugte Schlüsselstrom kann als Ersatz für ein One-Time-Pad betrachtet werden, welches durch einen physikalischen Zufallsprozess erzeugt wurde. Hierzu muss der Schlüsselstromgenerator allerdings zwei fundamentale Anforderungen erfüllen.

Zunächst ist ein Schlüsselstromgenerator nur dann für ein Stromverschlüsselungsverfahren geeignet, wenn die Kenntnis einer beliebigen Anzahl von Schlüsselstromzeichen

$$z_1, z_2, \dots, z_i$$

keinerlei (praktisch relevanten) Aufschluss über den Wert des folgenden Schlüsselstromzeichens z_{i+1} erlaubt. Insbesondere müssen die Zeichen in den Schlüsselströmen statistisch gleichverteilt und unkorreliert sein. D. h., ein Schlüsselstromgenerator kann nach der Initialisierung mit einem beliebigen Schlüssel $k \in \mathcal{K}$ gut als Pseudozufallszahlengenerator genutzt werden.

In der Regel bestimmen allerdings relativ wenige Schlüsselstromzeichen bereits den zur Erzeugung des Schlüsselstroms eingesetzten Schlüssel k und damit bestimmen diese wenigen Schlüsselstromzeichen prinzipiell auch die weiteren Zeichen des Schlüsselstroms.

Die zweite fundamentale Anforderung an einen Schlüsselstromgenerator ist daher die, dass eine Bestimmung des Schlüssels k mit Kenntnis des Schlüsselstroms nicht effizienter als durch eine Brute-Force-Suche im Schlüsselraum $\mathcal{K}$ möglich sein sollte.

1.3 Nonces zur Initialisierung eines Schlüsselstromgenerators

Der praktische Vorteil der Nutzung eines synchronen additiven Stromverschlüsselungsverfahrens mit einem relativ kleinen Schlüsselraum $\mathcal{K}$ gegenüber der Nutzung des One-Time-Pad-Verfahrens liegt natürlich darin begründet, dass nach der Auswahl (und sicheren Ablage oder Übermittlung) eines kleinen Schlüsselwerts $k \in \mathcal{K}$, große Datenmengen mithilfe des mit k initialisierten Schlüsselstromgenerators verschlüsselt und entsprechend entschlüsselt werden können.

Entsprechend den Ausführungen in Studienheft 01, Abschnitt 1.3 zur strikten Vermeidung einer wiederholten Nutzung eines One-Time-Pads, sollte auch die wiederholte Verwendung des gleichen Schlüssels $k \in \mathcal{K}$ zur Initialisierung eines Schlüsselstromgenerators strikt vermieden werden.

Haben Alice und Bob andererseits mit einem gewissen Aufwand einen Schlüssel k_{AB} ausgetauscht, so wollen Sie diesen gerne für eine längere Zeitdauer zur Verschlüsselung mehrerer Nachrichten verwenden. Um dennoch keine Schlüsselströme wiederholt zu verwenden, bieten sich folgende Ansätze an:

- Alice und Bob initialisieren den Schlüsselstromgenerator *einmal* mit k_{AB} und verwenden den Schlüsselstromgenerator dann fortlaufend, um immer weitere Teile des Schlüsselstroms zu berechnen und mit diesen die jeweils neuste Nachricht zu ver- bzw. zu entschlüsseln.

Bei dieser Vorgehensweise ergibt sich natürlich die Notwendigkeit, dass Alice und Bob die Schlüsselstromerzeugung synchron halten.

- Der gemeinsame Schlüssel k_{AB} wird vor der Verschlüsselung einer Nachricht m jeweils um einen Nonce-Wert η_m (vgl. mit Studienheft 01, Abschnitt 4.6) zu einem Schlüssel

$$k = k_{AB} || \eta_m$$

des Schlüsselstromgenerators erweitert. Mit diesem nur einmal zu nutzenden Wert aus $\mathcal{K}$ wird der Schlüsselstromgenerator initialisiert und der dann berechnete Schlüsselstrom wird zur Verschlüsselung von m benutzt.

Damit eine Entschlüsselung möglich ist, wird der Nonce-Wert η_m zusammen mit der Geheimtextnachricht übertragen oder beispielsweise als Wert eines Zählers synchron zwischen Alice und Bob gehalten.

Es sollte beachtet werden, dass bei der oben beschriebenen Vorgehensweise die effektive Schlüsselraumgröße kleiner als die von $|\mathcal{K}|$ ist, da der ursprüngliche Schlüsselraum $\mathcal{K}$ in einen Raum $\mathcal{K}_{\text{eff}}$, in dem die tatsächlichen Schlüssel k_{AB} liegen, und in einen für die Nonce-Werte vorgesehenen Bereich $\mathcal{N}$ zerlegt wird:

$$\mathcal{K} = \mathcal{K}_{\text{eff}} \times \mathcal{N}$$

- Der Definitionsbereich der Funktion s_0 des Schlüsselstromgenerator wird so erweitert, dass nicht nur Schlüsselwerte aus $\mathcal{K}$, sondern auch Nonce-Werte aus einer Menge $\mathcal{N}$ bei der Initialisierung des Statuswertes einfließen:

$$s_0 : \mathcal{K} \times \mathcal{N} \rightarrow \mathcal{S}$$

Während der gesamten Verwendungszeit eines Schlüssels $k \in \mathcal{K}$ kann der Schlüsselstromgenerator dann mehrfach genutzt werden, solange bei jeder Nutzung ein neuer, zuvor noch niemals im Zusammenhang mit k genutzter Nonce-Wert aus $\eta \in \mathcal{N}$ Verwendung findet. Ein Schlüsselstromgenerator mit einer Initialisierungsfunktion $s_0 : \mathcal{K} \times \mathcal{N} \rightarrow \mathcal{S}$ ist schematisch in Abbildung 1.8 dargestellt.

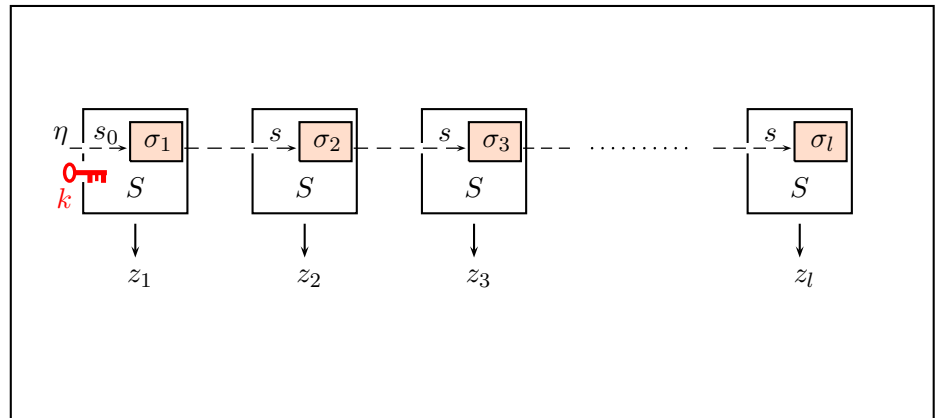


Abb. 1.8 Schlüsselstromgenerator, initialisiert mit Schlüsselwert k und Nonce-Wert η

1.4 ChaCha20

In der aktuellen Version des TLS-Protokolls (RFC 8446 (The Transport Layer Security (TLS) Protocol Version 1.3)), welches zur Absicherung des TCP-Protokolls spezifiziert ist und insbesondere dem HTTPS-Protokoll zugrunde liegt, sind zwei symmetrische Verschlüsselungsverfahren alternativ zum Einsatz vorgesehen. Dies ist neben dem im Abschnitt 2.7 behandelten Blockverschlüsselungsverfahren AES auch ein Stromverschlüsselungsverfahren, welches auf dem nachfolgend dargestellten Schlüsselstromgenerator beruht.

ChaCha20

Eine Spezifikation des Schlüsselstromgenerators ChaCha20 kann in RFC 8439 (ChaCha20 and Poly1305 for IETF Protocols) gefunden werden. Im Folgenden wird eine an unsere Notation eines synchronen additiven Stromverschlüsselungsverfahrens (s. S. 7) angepasste Beschreibung von ChaCha20 gegeben. Ergänzt wird die Beschreibung um den Java-Quellcode in Abbildung 1.9 einer lauffähigen Implementierung, der die zum Teil nicht weiter erklärten Details des Algorithmus direkt entnommen werden können.

Bei der Initialisierung des Zustands mittels der Funktion s_0 wird bei ChaCha20 von der in Abschnitt 1.3 dargestellten Möglichkeit Gebrauch gemacht, neben einem Schlüsselwert $k \in \mathcal{K}$ auch noch einen Nonce-Wert $\eta \in \mathcal{N}$ einfließen zu lassen.

Der Beschreibung des Algorithmus liegen folgende Mengen zugrunde:

$$\mathcal{Z} = \mathbb{Z}_{2^8}$$

$$\mathcal{K} = (\mathbb{Z}_{2^{32}})^8$$

$$\mathcal{N} = (\mathbb{Z}_{2^{32}})^3$$

$$\mathcal{S} = (\mathbb{Z}_{2^{32}})^{16} \times (\mathbb{Z}_{2^{32}})^{16} \times \mathbb{Z}_{2^{38+1}}$$

Die Ziffernmenge $\mathcal{Z}$ besteht also aus den möglichen Bytewerten und entspricht damit den Werten des primitiven Java-Datentyps `byte`. Schlüsselraum $\mathcal{K}$, Zustandsraum $\mathcal{S}$ und die Menge der Nonce-Werte $\mathcal{N}$ sind auf Grundlage des Zahlbereichs $\mathbb{Z}_{2^{32}}$ definiert, deren Elemente den Werten des primitiven Java-Datentyps `int` entsprechen.

Elemente des Zustandsraums $\mathcal{S}$ bestehen aus Tripeln:

$$(\mathbf{s}, \mathbf{z}, c) \in (\mathbb{Z}_{2^{32}})^{16} \times (\mathbb{Z}_{2^{32}})^{16} \times \mathbb{Z}_{2^{38+1}}$$

In der ersten Komponente $\mathbf{s}$ werden die Werte des Schlüssels k und des Nonce-Wertes η zusammen mit vier fest definierten Konstanten gespeichert. Zusätzlich enthält $\mathbf{s}$ eine Zählvariable $i \in \mathbb{Z}_{32}$. In Abhängigkeit von i wird der Wert von $\mathbf{s}$ im folgenden mit $\mathbf{s}^i$ bezeichnet. Sind

$$(k, \eta) = ((k_1, \dots, k_8), (n_1, n_2, n_3)) \in \mathcal{K} \times \mathcal{N} \quad \text{und} \quad i \in \mathbb{Z}_{2^{32}}$$

gegeben, so ist

$$\mathbf{s}^i := \mathbf{s}^i(k, \eta) := (c_1, c_2, c_3, c_4, k_1, \dots, k_8, i, n_1, n_2, n_3)$$

mit den Konstanten:

$$c_1 = 0x61707865$$

$$c_2 = 0x3320646e$$

$$c_3 = 0x79622d32$$

$$c_4 = 0x6b206574$$

Im Rahmen der Initialisierung und nach der Ausgabe von jeweils 64 Bytes des Schlüsselstroms wird mit den in s enthaltene Werten entsprechend der nachfolgenden Beschreibung von s ein Block von 64 (weiteren) Schlüsselstrombytes berechnet, der in der zweiten Komponente z abgelegt wird.

In der dritten Komponente wird die Anzahl c der bereits ausgelesenen Schlüsselstrombytes gespeichert. Die Länge eines Schlüsselstroms ist auf 2^{38} Bytes begrenzt.

Initialisierung: $s_0 : \mathcal{K} \times \mathcal{N} \rightarrow \mathcal{S}$

Sind $(k, \eta) \in \mathcal{K} \times \mathcal{N}$ so gilt mit $s^0 = s^0(k, \eta)$:

$$s_0((k, \eta)) = (s^1, s^0 \oplus f_{ib}^{10}(s^0), 0)$$

mit der nachfolgend definierten Funktion f_{ib} .

Update-Funktion: $s : \mathcal{S} \rightarrow \mathcal{S}$

Für $(s^i, z, c) \in \mathcal{S}$ (und $c < 2^{38}$) ist der Zustandsupdate wie folgt definiert:

$$s((s^i, z, c)) := \begin{cases} (s^i, z, c+1) & \text{falls } (c+1) \bmod 64 \neq 0 \\ (s^{i+1}, s^i +_{\mathbb{Z}_{2^{32}}} f_{ib}^{10}(s^i), c+1) & \text{falls } (c+1) \bmod 64 = 0 \end{cases}$$

mit einer 10 mal wiederholt auszuführenden Funktion:

$$f_{ib} : (\mathbb{Z}_{2^{32}})^{16} \rightarrow (\mathbb{Z}_{2^{32}})^{16}$$

Die Funktion f_{ib} wiederum besteht aus einer achtfachen Anwendung einer als *quater-round* bezeichneten Funktion q auf jeweils vier der sechzehn Komponenten eines Elements aus $(\mathbb{Z}_{2^{32}})^{16}$:

$$f_{ib}(s) := q(q(q(q(q(q(q(s, 0, 4, 8, 12), 1, 5, 9, 13), 2, 6, 10, 14), 3, 7, 11, 15), 0, 5, 10, 15), 1, 6, 11, 12), 2, 7, 8, 13), 3, 4, 9, 14)$$

Die Funktion

$$q : (\mathbb{Z}_{2^{32}})^{16} \times \mathbb{Z}_{16} \times \mathbb{Z}_{16} \times \mathbb{Z}_{16} \times \mathbb{Z}_{16} \rightarrow (\mathbb{Z}_{2^{32}})^{16}$$

bildet

$$(s, i, j, k, l) = ((s_0, s_1, \dots, s_{15}), i, j, k, l)$$

auf das Element aus $(\mathbb{Z}_{2^{32}})^{16}$ ab, welches sich durch Abänderung der Komponenten

$$a = s_i, \quad b = s_j, \quad c = s_k, \quad d = s_l$$

von s durch die Hintereinanderausführung folgender zwölf Abbildungen ergibt:

$$\begin{aligned}
 (a, b, c, d) &\mapsto (a + b, b, c, d) \\
 (a, b, c, d) &\mapsto (a, b, c, d \oplus a) \\
 (a, b, c, d) &\mapsto (a, b, c, r_{16}(d)) \\
 (a, b, c, d) &\mapsto (a, b, c + d, d) \\
 (a, b, c, d) &\mapsto (a, b \oplus c, c, d) \\
 (a, b, c, d) &\mapsto (a, r_{12}(b), c, d) \\
 (a, b, c, d) &\mapsto (a + b, b, c, d) \\
 (a, b, c, d) &\mapsto (a, b, c, d \oplus a) \\
 (a, b, c, d) &\mapsto (a, b, c, r_8(d)) \\
 (a, b, c, d) &\mapsto (a, b, c + d, d) \\
 (a, b, c, d) &\mapsto (a, b \oplus c, c, d) \\
 (a, b, c, d) &\mapsto (a, r_7(b), c, d)
 \end{aligned}$$

Extraktion der Schlüsselstromzeichen: $S : \mathcal{S} \rightarrow \mathcal{Z}$

Mit S werden die Bytewerte, aus denen die Zahlen $z_0, z_1, \dots, z_{15}$ der zweiten Komponente eines Elements aus $\mathcal{S}$ zusammengesetzt sind, extrahiert.

Für

$$(s^i, \mathbf{z}, c) = (s^i, (z_0, z_1, \dots, z_{15}), c) \in \mathcal{S}$$

werden hierzu

$$j = \lfloor c/4 \rfloor \bmod 16$$

$$k = c \bmod 4$$

berechnet und

$$S\left((s^i, (z_0, z_1, \dots, z_{15}), c)\right) := \left\lfloor \frac{z_j}{2^{8k}} \right\rfloor \bmod 2^8$$

gesetzt.

```

1 public class KeyStreamGenerator_ChaCha20
  extends SynchronousKeyStreamGenerator {

    // Zustand des Schlüsselstromgenerators definiert durch (s, z, c)
5 private final int[] s = new int[16];
  private final int[] z = new int[16];
  private long c;

  /**
10  * Initialisierung des Schlüsselstromgenerators
  * Teilfunktionalität von s_0: Setzen von s := s^0,
  * z wird beim ersten Aufruf von getNextKeyStreamByte() initialisiert.
  *
  * @param key Schlüssel (ByteArray der Länge 32)
15  * @param nonce Nonce-Wert (ByteArray der Länge 12)
  */
  public void initState(byte[] key, byte[] nonce) {
    initState(key, nonce, 0);
  }

20  /**
  * Initialisierung des Schlüsselstromgenerators
  * Teilfunktionalität von s_0: Setzen von s := s^blockCtr,
  * z wird beim ersten Aufruf von getNextKeyStreamByte() initialisiert.
25  *
  * @param key Schlüssel (ByteArray der Länge 32)
  * @param nonce Nonce-Wert (ByteArray der Länge 12)
  * @param blockCtr Initialwert des Zählerwerts in s[12]
  */
30  public void initState(
    byte[] key, byte[] nonce, int blockCtr) {
    if( (key.length != 32) || (nonce.length != 12) ) {
      throw new IllegalArgumentException(
        "Invalid length of key or nonce!");
35    }

    s[0] = 0x61707865;
    s[1] = 0x3320646e;
    s[2] = 0x79622d32;
40    s[3] = 0x6b206574;

    // Transformation der Schlüsselbytes in int-Werte s[4],...,s[11]
    // Die Bytes werden in 'little-endian order' eingelesen.
    for( int i = 0; i < 8; ++i ) {
45      s[i + 4] = (0xff & key[4 * i]);
      s[i + 4] |= (0xff & key[4 * i + 1]) << 8;
      s[i + 4] |= (0xff & key[4 * i + 2]) << 16;
      s[i + 4] |= (0xff & key[4 * i + 3]) << 24;
    }

50    s[12] = blockCtr;

    // Transformation der Nonce-Bytes in int-Werte s[13],s[14],s[15]
    for( int i = 0; i < 3; ++i ) {
55      s[i + 13] = (0xff & nonce[4 * i]);
      s[i + 13] |= (0xff & nonce[4 * i + 1]) << 8;
      s[i + 13] |= (0xff & nonce[4 * i + 2]) << 16;
      s[i + 13] |= (0xff & nonce[4 * i + 3]) << 24;
    }

60    c = -1; // Bewirkt Abschluss der Initialisierung durch Berechnung der
            // Werte von z beim ersten Aufruf von getNextKeyStreamByte()
  }

```

```

65  /**
   * Berechnung des nächsten Schlüsselstrombytes.
   * Entspricht einem einmaligen Aufruf der Funktionen s (Zustands-Update)
   * und S (Extraktion eines Schlüsselstrombytes.
   * Beim ersten Aufruf nach initState(...) wird anstelle von s der Abschluss
70  * von s_0 durchgeführt.
   *
   * @return Nächstes Schlüsselstrombyte
   */
@Override
75  public byte getNextKeyStreamByte() {
    // Update-Funktion s
    ++c;
    if( (c & 0x3f) == 0 ) { // (c mod 64) == 0 ?
        if( c == 0x4000000000L ) { // 2^38
80            throw new RuntimeException("Keystream exhausted!");
        }
        updateStateBlocks();
    }

    // Extraktion des Schlüsselbytes (Funktion S)
85    int stateWord = z[(int) ((c >> 2) & 0x0f)];
    byte keyStreamByte
        = (byte) (stateWord >> (int) 8 * (c & 0x03));
    return keyStreamByte;
90 }

/**
 * Update der Block-Komponenten s und z des Zustands
 * für c mod 64 == 0.
95 */
private void updateStateBlocks() {
    System.arraycopy(s, 0, z, 0, 16);

    // 10-fache Anwendung der Inner-Block_Funktion f_{ib}
100    for( int i = 0; i < 10; ++i ) {
        quaterround(0, 4, 8, 12);
        quaterround(1, 5, 9, 13);
        quaterround(2, 6, 10, 14);
        quaterround(3, 7, 11, 15);
105        quaterround(0, 5, 10, 15);
        quaterround(1, 6, 11, 12);
        quaterround(2, 7, 8, 13);
        quaterround(3, 4, 9, 14);
    }

110    for( int i = 0; i < 16; ++i ) {
        z[i] += s[i];
    }

115    s[12] += 1; // Inkrementierung des Blockzählers
}

```

```

118  /**
    * Quater-Round-Funktion (Funktion q)
    */
120  private void quaterround(int i1, int i2, int i3, int i4) {
    int a = z[i1];
    int b = z[i2];
    int c = z[i3];
125    int d = z[i4];

    a += b;
    d ^= a;
    d = (d << 16) | (d >>> 16);
130    c += d;
    b ^= c;
    b = (b << 12) | (b >>> 20);
    a += b;
    d ^= a;
135    d = (d << 8) | (d >>> 24);
    c += d;
    b ^= c;
    b = (b << 7) | (b >>> 25);

140    z[i1] = a;
    z[i2] = b;
    z[i3] = c;
    z[i4] = d;
  }

```

Abb. 1.9 Implementierung des ChaCha20-Schlüsselstromgenerators in Java

Die zur Ver- oder Entschlüsselung einer Nachricht notwendige XOR-Verknüpfung mit dem Schlüsselstrom kann separat implementiert werden. Damit diese Implementierung in der Klasse `AdditiveStreamCipher` (s. Abb. 1.12) mit einem beliebigen Schlüsselstromgenerator genutzt werden kann nutzt sie eine entsprechende abstrakte Klasse `KeyStreamGenerator` (s. Abb. 1.10), die auch von `KeyStreamGenerator_ChaCha20` indirekt über die Klasse `SynchronousKeyStreamGenerator` (s. Abb. 1.11) erweitert wird (s. Abb. 1.9, Zeile 2).

```

1  public abstract class KeyStreamGenerator {

    /**
    * Berechnung des nächsten Schlüsselstrombytes.
    * (Entspricht einem einmaligen Aufruf der
    * Update-Funktion s und der Extraktions-Funktion S.)
    *
    * @return Nächstes Schlüsselstrombyte
    */
5
10  abstract byte getNextKeyStreamByte(byte mByte);

  }

```

Abb. 1.10 Abstrakte Basisklasse für Implementierungen von Schlüsselstromgeneratoren

```

1 public abstract class SynchronousKeyStreamGenerator
  extends KeyStreamGenerator {

    @Override
5   final byte getNextKeyStreamByte(byte mByte){
      return getNextKeyStreamByte();
    }

    /**
10   * Berechnung des nächsten Schlüsselstrombytes.
    * (Entspricht eines einmaligen Aufrufs der
    * Update-Funktion s und der Extraktions-Funktion S.)
    *
    * @return Nächstes Schlüsselstrombyte
15   */
    abstract byte getNextKeyStreamByte();

    /**
20   * Generierung einer vorgegebenen Anzahl weiterer
    * Schlüsselstrombytes. (Entspricht einer entsprechenden
    * Anzahl von Aufrufen der Funktionen s und S.)
    *
    * @param noOfBytes Anzahl der zu generierenden
    * Schlüsselstrombytes
25   * @return Die nächsten noOfBytes Bytes des
    * Schlüsselstroms
    */
    public byte[] getNextKeyStreamBytes(int noOfBytes) {
        byte[] keyStream = new byte[noOfBytes];
30   for (int i = 0; i < noOfBytes; ++i) {
            keyStream[i] = getNextKeyStreamByte();
        }
        return keyStream;
    }
35 }

```

Abb. 1.11 Abstrakte Basisklasse für Implementierungen von synchronen Schlüsselstromgeneratoren

```

1 public class AdditiveStreamCipher {

    static public void encrypt(
        KeyStreamGenerator ksg, byte[] m) {
5   for (int i = 0; i < m.length; ++i) {
        m[i] ^= ksg.getNextKeyStreamByte(m[i]);
    }
    }
}

```

Abb. 1.12 Verschlüsselung (XOR-Verknüpfung) einer Nachricht *m* mit den nächsten vom Schlüsselstromgenerator *ksg* erzeugten Bytes

Aufgabe 1

Führen Sie eine Verschlüsselung des in RFC 8439, Abschnitt 2.4.2 gegebenen Klartextes mit den dort ebenfalls gegebenen Werten für Schlüssel, Nonce-Wert und Block-Zählerwert durch. Benutzen Sie hierzu die in den Abbildungen 1.9–1.12 wiedergegebenen Klassen und vergleichen Sie den von Ihnen berechneten Geheimtext mit dem in RFC 8439, Abschnitt 2.4.2 genannten.

```

1  public static void test_RFC8439_2_4_2() {
    byte[] key = new byte[32];
    for (byte i = 0; i < 32; ++i) {
        key[i] = i;
5   }

    byte[] nonce
      = new byte[]{0, 0, 0, 0, 0, 0, 0, 0, 0x4a, 0, 0, 0, 0};

10   KeyStreamGenerator_ChaCha20 ksg
      = new KeyStreamGenerator_ChaCha20();
      ksg.initState(key, nonce, 1);

    byte[] m = ("Ladies and Gentlemen of the class of '99:"
15   + " If I could offer you only one tip for the future,"
      + " sunscreen would be it.").getBytes();
      AdditiveStreamCipher.encrypt(ksg, m);
      System.out.println("Ciphertext:");
      System.out.println(Dump.dump(m));
20   System.out.println();
  }

```

Abb. 1.13 ChaCha20-Verschlüsselung mit den aus RFC 8439, Abschnitt 2.4.2 gegebenen Daten

1.5 Cipher-Instanzen: Verschlüsselungsalgorithmen in Java-Laufzeitumgebungen

In den aktuellen Java-Laufzeitumgebungen stehen eine Vielzahl von Verschlüsselungsalgorithmen zur Verfügung. Die Nutzung einer solchen Implementierung eines Verschlüsselungsalgorithmus folgt einem allgemeinen Framework. Dieses wird nachfolgend beschrieben und am Beispiel der Verschlüsselung einer Nachricht mit dem ChaCha20-Algorithmus in Abbildung 1.14 illustriert.

- Mit dem Aufruf einer statischen Methode `getInstance(...)` der Klasse `Cipher` wird eine `Cipher`-Instanz zur Verfügung gestellt, die die Implementierung eines Verschlüsselungsalgorithmus beinhaltet. Welcher Algorithmus dies ist, wird durch den Wert eines `String`-Parameters bestimmt, der der `getInstance(...)`-Methode übergeben werden muss. Steht eine Implementierung des gewünschten Algorithmus nicht zur Verfügung oder kann der übergebene Parameter dem Algorithmus nicht zugeordnet werden, so wirft `getInstance(...)` eine `NoSuchAlgorithmException`.
- Die `Cipher`-Instanz muss mit dem Aufruf einer `init(...)`-Methode initialisiert werden. Hierbei wird über die Konstanten `Cipher.ENCRYPT_MODE` bzw. `Cipher.DECRYPT_MODE` eingestellt, ob der Verschlüsselungsalgorithmus zur Ver- oder zur Entschlüsselung genutzt werden soll und über eine geeignete Instanz des `Key`-Interfaces wird der zu verwendende Schlüssel übergeben. Werden zur Durchführung des Algorithmus weitere spezifische Parameter benötigt, so können diese in einer spezifischen Instanz des Interfaces `AlgorithmParameterSpec` übergeben werden.
 - Schlüssel die in Form eines `ByteArrays` für ein symmetrisches Verschlüsselungsverfahren vorliegen, können durch Nutzung eines Konstruktors der Klasse `SecretKeySpec` in einer Instanz vom Typ `Key` gekapselt werden.
 - Speziell für den ChaCha20-Algorithmus wird `AlgorithmParameterSpec` von der Klasse `ChaCha20ParameterSpec` implementiert. Mithilfe einer Instanz von `ChaCha20ParameterSpec` werden der Nonce-Wert und der Block-Zählerwert übergeben.
- Nach der Initialisierung der `Cipher`-Instanz, kann diese zur Durchführung einer Ver- bzw. Entschlüsselung einer Nachricht genutzt werden. Hierfür stehen verschiedene `update(...)`- und `doFinal(...)`-Methoden zur Verfügung. (Die `update(...)`-Methoden werden benutzt, falls eine Nachricht nicht in einem Stück, sondern nach und nach stückweise verarbeitet werden soll.)

```

1  public static void test_JRE_RFC8439_2_4_2()
   throws NoSuchAlgorithmException, NoSuchPaddingException,
   InvalidKeyException, InvalidAlgorithmParameterException,
   IllegalBlockSizeException, BadPaddingException {
5
   byte[] key = new byte[32];
   for (byte i = 0; i < 32; ++i) {
       key[i] = i;
   }
10
   byte[] nonce
   = new byte[]{0, 0, 0, 0, 0, 0, 0, 0, 0x4a, 0, 0, 0, 0};
   ChaCha20ParameterSpec params
   = new ChaCha20ParameterSpec(nonce, 1);
15
   Cipher cipher = Cipher.getInstance("ChaCha20");

   SecretKeySpec keyChaCha20
   = new SecretKeySpec(key, "ChaCha20");
20
   cipher.init(Cipher.ENCRYPT_MODE, keyChaCha20, params);

   byte[] m = ("Ladies and Gentlemen of the class of '99:"
   + " If I could offer you only one tip for the future,"
25  + " sunscreen would be it.").getBytes();
   byte[] c = cipher.doFinal(m);
   System.out.println(Dump.dump(c));
   System.out.println();
   }

```

Abb. 1.14 ChaCha20-Verschlüsselung mit einer Cipher-Instanz der Java-Laufzeitumgebung

Aufgabe 2

Führen Sie die Methode aus Abbildung 1.14 in einem Java-Programm aus und vergleichen Sie die Ausgabe mit dem Ergebnis von Aufgabe 1.

Frage 3

Wie ändert sich der Inhalt der Variablen `c` in Abbildung 1.14, Zeile 26, falls in Zeile 21 die Konstante `Cipher.ENCRYPT_MODE` durch die Konstante `Cipher.DECRYPT_MODE` ersetzt wird?

Blockverschlüsselungsverfahren (Block Cipher)

2

2.1	Grundlagen	22
2.2	Padding-Verfahren	23
2.3	Betriebsmodi	23
2.4	Konstruktionsprinzipien von Blockverschlüsselungsver- fahren	35
2.5	DES und Triple-DES	36
2.6	Meet-in-the-Middle-Angriffe	38
2.7	AES	41

2.1 Grundlagen

Blockverschlüsselungsverfahren wurden bereits in Studienheft 01, Abschnitt 4.2 definiert. Typischerweise treten in technischen Anwendungen nur Blöcke auf, deren Komponenten Byte-werte sind. Mit dieser Einschränkung halten wir (nochmals) die Bezeichnungen für Blockverschlüsselungsverfahren fest:

Blockverschlüsselungsverfahren (Block Cipher)

Ein Verschlüsselungsverfahren mit Verschlüsselungsfunktion E

$$E : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M}$$

und Entschlüsselungsfunktion D

$$D : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M}$$

wird als *Blockverschlüsselungsverfahren* bezeichnet, falls die Menge der Nachrichten $\mathcal{M}$ durch die Menge der *Blöcke* einer festen Länge $n \in \mathbb{N}$ (der *Blocklänge* (in Bytes)) gegeben ist:

$$\mathcal{M} := (\mathbb{Z}_2^8)^n = \{(z_1, z_2, \dots, z_n) \mid z_i \in \mathbb{Z}_2^8\}$$

Wichtig

- Häufig werden Blöcke, die aus n Bytewerten bestehen als Blöcke aufgefasst, die aus $8 \cdot n$ Bitwerten bestehen:

$$(\mathbb{Z}_2^8)^n \longleftrightarrow \mathbb{Z}_2^{8 \cdot n}$$

Die Blocklänge (in Bits) wird dann mit $8 \cdot n$ angegeben.

- Mit der obigen Identifikation kann die Menge der Blöcke $\mathcal{M}$ als Vektorraum über $\mathbb{Z}_2$ aufgefasst werden und für zwei Blöcke $m_1, m_2 \in \mathcal{M}$ ist die Summe

$$m_1 + m_2$$

durch eine bitweise Addition definiert.

Diese Addition entspricht der XOR-Verknüpfung der Byte-Werte, die die Komponenten der Blöcke m_1 und m_2 bilden. In graphischen Darstellungen wird für diese Verknüpfung im Folgenden das Symbol $\oplus$ verwendet, in Formeln jedoch das einfache $+$.

- Da die Vektoren aus $\mathbb{Z}_2^{8 \cdot n}$ den Binärdarstellungen der Zahlen aus $\mathbb{Z}_{2^{8 \cdot n}}$ eindeutig entsprechen, könnte eine Addition von zwei Blöcken auch mittels der Addition in $\mathbb{Z}_{2^{8 \cdot n}}$ definiert werden. Diese Addition liefert natürlich andere Ergebnisse als die bereits oben definierte bitweise Addition (XOR-Operation).

Benötigt wird die Addition in $\mathbb{Z}_{2^{8 \cdot n}}$ allerdings bei der Beschreibung des CTR-Modus. Für $m \in \mathcal{M}$ werde die m entsprechende Zahl in $\mathbb{Z}_{2^{8 \cdot n}}$ durch

$$\overline{m} \in \mathbb{Z}_{2^{8 \cdot n}}$$

bezeichnet. Dann wird für $m \in \mathcal{M}$ und $z \in \mathbb{Z}$ der Block

$$m \boxplus z$$

definiert durch:

$$\overline{m \boxplus z} = (\overline{m} + z) \bmod 2^{8 \cdot n} \quad (2.1)$$

In Worten: Zur Berechnung von $m \boxplus z$ werden m und z als ganze Zahlen aufgefasst, deren Summe berechnet und diese wird (wenn nötig) modulo $2^{8 \cdot n}$ reduziert. Das Ergebnis ist schließlich als Block zu interpretieren.



Für jeden Schlüssel $k \in \mathcal{K}$ ist E_k eine Permutation der Menge der Blöcke $\mathcal{M}$ mit Umkehrfunktion D_k . Bei einem guten Blockverschlüsselungsverfahren sollte hierbei gelten:

Die Abbildung

$$\mathcal{K} \rightarrow \Sigma_{\mathcal{M}}, \quad k \mapsto E_k$$

lässt sich mit statistischen Verfahren nicht von einer Zuordnung der Schlüsselwerte zu zufällig ausgewählten Permutationen aus $\Sigma_{\mathcal{M}}$ unterscheiden.

Wichtig Diese Eigenschaft allein garantiert keinesfalls die Sicherheit des Verfahrens, ist aber als eine Anforderung an ein sicheres Verfahren zu stellen. ◀

Beispielsweise ergibt sich aus der oben genannten Eigenschaft, dass sich bei der Abänderung eines einzelnen Bits in einem Eingabeblock m oder in einem Schlüsselwert k der Ausgabeblock $E_k(m)$ sich durchschnittlich an $4 \cdot n$ vielen Bitstellen vom ursprünglichen Ausgabeblock unterscheidet.

2.2 Padding-Verfahren

In der Regel sind Daten (Byte-Arrays) zu verschlüsseln, die nicht genau die Länge eines Blocks aufweisen. Um Daten beliebiger Länge mit einem Blockverschlüsselungsverfahren zu verschlüsseln, werden die Daten zunächst in Blöcke zerlegt, deren Längen der Blocklänge des Verschlüsselungsverfahrens entsprechen.

Damit eine solche Zerlegung möglich ist, muss die Datenlänge ein ganzzahliges Vielfaches der Blocklänge sein. Ist dies nicht der Fall, müssen die Daten um weitere sogenannte *Padding*-Bytes ergänzt werden. Diese Padding-Bytes müssen nach der Entschlüsselung wieder entfernt werden. Hierzu müssen diese Bytes (bzw. ihre Anzahl) bekannt sein. Diese Information kann beispielsweise in folgender Weise vorliegen:

- Die Anzahl der Padding-Bytes wird zusätzlich zum Geheimtext gespeichert bzw. übertragen.
- Als Padding-Bytes wird ein Bytewert genutzt, der niemals in einer Klartextnachricht vorkommt. (Das funktioniert natürlich nur dann, falls es ein solches Zeichen gibt, zum Beispiel bei ASCII-kodierten Textnachrichten, bei denen der Bytewert 00 kein reguläres Textzeichen repräsentiert.)
- Die Daten werden immer um mindestens ein und höchstens so viele Bytes ergänzt, wie in einen Block passen. In den Padding-Bytes (oder auch nur im letzten Padding-Byte) wird dabei die Länge des Paddings kodiert.

2.3 Betriebsmodi

Zur Ver- und Entschlüsselung von Nachrichten beliebiger Bytelänge $l(m)$ gibt es eine Vielzahl verschiedener Betriebsmodi. In diesem Abschnitt werden die in der NIST-Publikation SP 800-38A (Recommendation for Block Cipher Modes of Operation: Methods and Techniques) und dem zugehörigen Addendum (Recommendation for Block Cipher Modes of Operation: Three Variants of Ciphertext Stealing for CBC Mode) beschriebenen Betriebsmodi behandelt. Dies sind die folgenden Betriebsmodi:

- ECB (Electronic Code Book)
- CBC (Cipher Block Chaining)
- CTS (Ciphertext Stealing)
- CTR (Counter)
- OFB (Output Feedback)
- CFB (Cipher Feedback)

In den Betriebsmodi ECB und CBC können nur Nachrichten verschlüsselt werden, deren Bytelängen ganzzahlige Vielfache der Blocklänge n sind. Gegebenenfalls sind also auch Padding-Verfahren zu nutzen (siehe Abschnitt 2.2).

Der CTS-Modus stellt eine trickreiche Modifikation des CBC-Modus dar, bei der für beliebige Klartextnachrichten m mit $l(m) \geq n$ die Verschlüsselung eine Geheimtextnachricht c gleicher Länge $l(c) = l(m)$ liefert.

Im CTR-, OFB- und CFB-Modus werden Blockverschlüsselungsverfahren zur Konstruktion von Stromverschlüsselungsverfahren benutzt.

Betriebsmodus ECB (Electronic Code Book)

Im *ECB-Modus* wird mit einem Blockverschlüsselungsverfahren E jedes Tupel

$$(m_1, \dots, m_r)$$

von Blöcken $m_i \in \mathcal{M}$ für einen gegebenen Schlüssel $k \in \mathcal{K}$ blockweise verschlüsselt:

$$E_k((m_1, \dots, m_r)) := (E_k(m_1), \dots, E_k(m_r))$$

Diese einfache komponentenweise Verschlüsselung ist in Abbildung 2.1 dargestellt.

Zur Entschlüsselung ist die Entschlüsselungsfunktion D ebenfalls einfach komponentenweise anzuwenden:

$$D_k((c_1, \dots, c_r)) := (D_k(c_1), \dots, D_k(c_r))$$

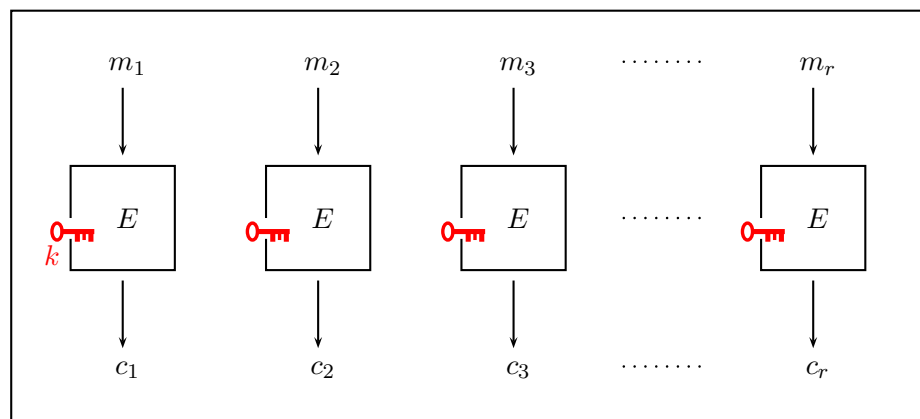


Abb. 2.1 Blockverschlüsselungsverfahren im ECB-Modus

Frage 4

Vergleichen Sie Abbildung 2.1 mit Abbildung 1.2 und bringen Sie die jeweils dargestellten Verfahren in einen Zusammenhang.

Aufgabe 3

Führen Sie die Methode aus Abbildung 2.2 zur Verschlüsselung einer 48 Byte langen Nachricht aus. Beachten Sie, dass nach der Instanziierung des Cipher-Objekts mit "AES/ECB/NOPADDING" dieses Objekt eine Implementierung des AES-Algorithmus im ECB-Modus zur Verfügung stellt, bei der die Daten nicht mit einem Padding ergänzt werden, sodass die Datenlänge ein Vielfaches der Blocklänge sein muss! (In dem Beispiel ist dies erfüllt, da beim AES-Verfahren die Blöcke 16 Byte lang sind.)

Beachten Sie, dass der erste und dritte Klartextblock und entsprechend auch der erste und dritte Geheimtextblock gleich sind.

Erweitern Sie das Programm, sodass der in c gespeicherte Geheimtext wieder entschlüsselt wird.

```

1  public static void main(String[] args) throws Exception {
    byte[] key = Dump.hexString2byteArray(
        "0102030405060708090A0B0C0D0E0F10");
    SecretKey secretKey = new SecretKeySpec(key, "AES");

5      Cipher cipher
        = Cipher.getInstance("AES/ECB/NOPADDING");

    cipher.init(Cipher.ENCRYPT_MODE, secretKey);

10     byte[] m = new byte[48];
    m[0] = m[32] = 'T';
    m[1] = m[33] = 'e';
    m[2] = m[34] = 's';
15     m[3] = m[35] = 't';
    System.out.println("Klartext:");
    System.out.println(Dump.dump(m));

    byte[] c = cipher.doFinal(m);

20     System.out.println("Geheimtext:");
    System.out.println(Dump.dump(c));
  }

```

Abb. 2.2 Quellcode zur AES-Verschlüsselung einer 48 Byte langen Nachricht im ECB-Modus

Aufgabe 4

Verschlüsseln Sie die nur aus den ersten 4 Byte von m aus Abbildung 2.2 bestehende Nachricht mit dem AES-Verfahren, indem Sie eines der unterstützten Padding-Verfahren (PKCS5PADDING, ISO10126PADDING) benutzen.

Ersetzen Sie hierzu beispielsweise die Zeilen 6–15 aus Abbildung 2.2 durch den Quellcode aus Abbildung 2.3.

```

6      Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5PADDING");
      cipher.init(Cipher.ENCRYPT_MODE, secretKey);
10     byte[] m = "Test".getBytes();

```

Abb. 2.3 Änderungen des Quellcodes aus Abbildung 2.2 zur Verschlüsselung einer 4 Byte langen Nachricht unter Verwendung eines Padding-Verfahrens

Frage 5

Untersuchen Sie die unterstützten Paddingverfahren PKCS5PADDING und ISO10126PADDING indem Sie mehrfach eine Verschlüsselung mit diesen Verfahren durchführen und die Geheimtexte anschließend mit der Option NOPADDING entschlüsseln, sodass das Padding nach der Entschlüsselung zusammen mit dem Klartext sichtbar wird.

Wie unterscheiden sich die Padding-Verfahren PKCS5PADDING und ISO10126PADDING?

Der Name des Modus ECB legt es bereits nahe: Das Verfahren ist durch Wörterbuchangriffe angreifbar, da gleiche Klartextblöcke immer wieder in die gleichen Geheimtextblöcke verschlüsselt werden. Entsprechend der im Studienheft 01, Abschnitt 4.6 beschriebenen Vorgehensweise könnten Nonce-Werte zur Abwehr dieser Angriffsmöglichkeit genutzt werden. In Abbildung 2.4 ist diese Möglichkeit dargestellt, wobei je Klartextblock m_i ein Nonce-Wert v_i genutzt wird (und ggf. zusammen mit den Geheimtextblöcken zu übertragen wäre).

Geschickter ist es jedoch, anstelle der Nonce-Werte $v_2, \dots, v_r$ den jeweils zuletzt berechneten Geheimtextblock zu benutzen. So ergibt sich der in Abbildung 2.5 dargestellte Betriebsmodus *CBC* (*Cipher Block Chaining*). Hier wird nur noch ein Nonce-Wert benötigt, der in diesem Zusammenhang häufig als *Initialisierungsvektor IV* (*Initial Value, Initial Vector*) bezeichnet wird.

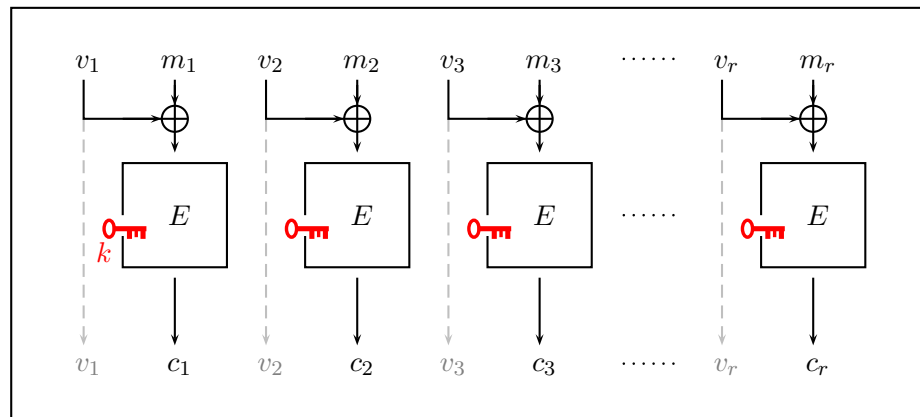


Abb. 2.4 Blockverschlüsselungsverfahren mit Nonce-Werten v_i

Betriebsmodus CBC (Cipher Block Chaining)

Im *CBC-Modus* wird mit einem Blockverschlüsselungsverfahren E jedes Tupel

$$(m_1, \dots, m_r)$$

von Blöcken $m_i \in \mathcal{M}$ für einen gegebenen Schlüssel $k \in \mathcal{K}$ und einen Nonce-Wert

$$IV \in \mathcal{M}$$

folgendermaßen rekursiv blockweise verschlüsselt:

$$c_0 := IV$$

$$c_i := E_k(m_i + c_{i-1}) \quad \text{für } i = 1, \dots, r$$

$$E_{k,IV}(m_1, \dots, m_r) := (c_1, \dots, c_r)$$

Die Verschlüsselung im CBC-Modus ist in Abbildung 2.5 dargestellt.

Die Entschlüsselung eines CBC-verschlüsselten Geheimtextes $c = (c_1, \dots, c_r)$ ist durch

$$c_0 := IV$$

$$m_i := D_k(c_i) + c_{i-1} \quad \text{für } i = 1, \dots, r$$

$$D_{k,IV}(c_1, \dots, c_r) := (m_1, \dots, m_r)$$

gegeben. Die Entschlüsselung im CBC-Modus ist in Abbildung 2.6 dargestellt.

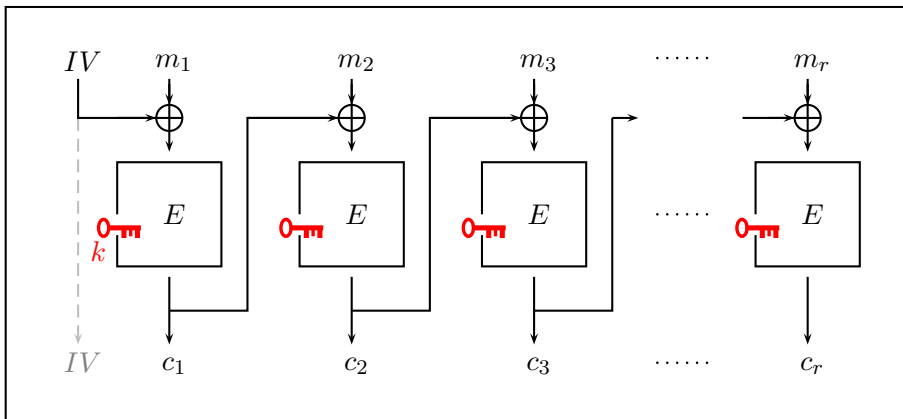


Abb. 2.5 Verschlüsselung mit Blockverschlüsselungsverfahren im CBC-Modus

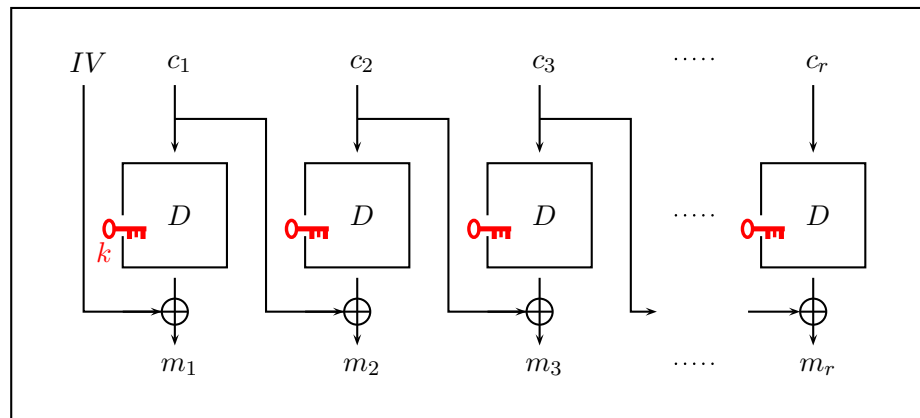


Abb. 2.6 Entschlüsselung mit einem Blockverschlüsselungsverfahren im CBC-Modus

Aufgabe 5

Ändern Sie den Quellcode aus Abbildung 2.2 so ab, dass anstelle des ECB-Modus ohne Padding der CBC-Modus mit dem PKCS5PADDING-Verfahren zum Einsatz kommt.

Ersetzen Sie hierzu beispielsweise die Zeilen 6–9 aus Abbildung 2.2 durch den Quellcode aus Abbildung 2.7. (IV -Werte müssen durch Instanzen der Klasse `IvParameterSpec` gekapselt werden, um sie einer geeigneten `init(...)`-Methode als Parameter übergeben zu können.)

Beachten Sie, dass der erste und dritte Geheimtextblock nun verschieden sind, obwohl erster und dritter Klartextblock identisch sind.

Erweitern Sie das Programm, sodass der in `c` gespeicherte Geheimtext wieder entschlüsselt wird.

```

6      Cipher cipher
      = Cipher.getInstance("AES/CBC/PKCS5PADDING");

      byte[] ivBytes = new byte[16];
10     new SecureRandom().nextBytes(ivBytes);
      IvParameterSpec iv = new IvParameterSpec(ivBytes);
      cipher.init(Cipher.ENCRYPT_MODE, secretKey, iv);

```

Abb. 2.7 Änderungen des Quellcodes aus Abbildung 2.2 zur Verschlüsselung im CBC-Modus

Frage 6

Alice verschlüsselt die Nachricht $(m_1, m_2, \dots, m_9)$ mit dem CBC-Verfahren unter Nutzung des Initialisierungswerts IV . Bei der Übertragung von IV und $(c_1, c_2, \dots, c_9)$ an Bob wird ein Bit des Blocks c_2 fehlerhaft empfangen.

Welche Teile der ursprünglichen Klartextnachricht kann Bob fehlerfrei lesen?

Frage 7

Lässt sich die Verschlüsselung und/oder Entschlüsselung im CBC-Modus synchron blockweise durchführen?

Ein Betriebsmodus, der den CBC-Modus verallgemeinert und die Verschlüsselung von Nachrichten beliebiger Länge $\geq n$ (ohne Padding) erlaubt, ist der *CBC-CS-Modus* (*Ciphertext Stealing for CBC Mode*), der auch kurz als *CTS-Modus* (*Ciphertext Stealing Mode*) bezeichnet wird.

Betriebsmodus CBC-CS1 (Ciphertext Stealing for CBC, Variante 1)

Gegeben sei ein Blockverschlüsselungsverfahren E für Blöcke der Byte-Länge n , ein Schlüssel $k \in \mathcal{K}$ und ein Nonce-Wert $IV \in \mathcal{M}$.

Nachrichten, die mindestens n Bytes umfassen, können in der Form

$$m = (m_1, \dots, m_{r-1}, m_r^*)$$

mit Byte-Arrays

$$m_1, \dots, m_{r-1} \in \mathcal{M} \quad \text{und} \quad l = l(m_r^*) \in \{1, \dots, n\}$$

geschrieben werden, wobei $l(m_r^*)$ die Byte-Länge von m_r^* bezeichnet.

Es sei

$$m_r \in \mathcal{M}$$

der Block, der aus m_r^* durch das Anfügen von $n-l$ vielen 00-Bytes hervorgeht (Padding von m_r^* mit 00 ... 00 zu m_r). Gilt $l = n$, so ist $m_r = m_r^*$.

Weiterhin sei

$$(c_1, \dots, c_r) = E_{k,IV}(m_1, \dots, m_r)$$

die Verschlüsselung von $(m_1, \dots, m_r)$ im CBC-Modus.

Die Verschlüsselung im CBC-CS1-Modus von m ist schließlich definiert durch

$$E\text{-CBC-CS1}_{k,IV}(m_1, \dots, m_{r-1}, m_r^*) := (c_1, \dots, c_{r-2}, c_{r-1}^*, c_r)$$

wobei c_{r-1}^* nur aus den ersten l Bytes von c_{r-1} besteht.

Beachten Sie, dass die Geheimtextnachricht $(c_1, \dots, c_{r-2}, c_{r-1}^*, c_r)$ die gleiche Länge wie die Klartextnachricht m besitzt.

Kommentar

- (i) Die Vorgehensweise beim CBC-CS1-Modus lässt sich kurz so zusammenfassen, dass die Klartextnachricht mit einer minimalen Anzahl p von 00-Bytes auf ein Vielfaches der Blocklänge gebracht wird (Padding), dann im CBC-Modus verschlüsselt wird und vom vorletzten Geheimtextblock die letzten p Bytes entfernt („gestohlen“) werden.
- (ii) Ist die Länge der Klartextnachricht ein Vielfaches der Blocklänge, so entspricht eine Verschlüsselung im CBC-CS1-Modus einer Verschlüsselung im CBC-Modus.



Frage 8

Beschreiben Sie, wie eine Geheimtextnachricht c im CBC-CS1-Modus zu entschlüsseln ist.

Betriebsmodus CBC-CS2 (Ciphertext Stealing for CBC, Variante 2)

Ebenso wie beim CBC-CS1-Modus werden Klartextnachrichten, deren Länge ein Vielfaches der Blocklänge n ist, einfach ohne weitere Veränderung im CBC-Modus verschlüsselt.

Die Verschlüsselung einer Klartextnachricht, deren Länge $> n$ und kein Vielfaches von n ist, wird im CBC-CS2-Modus grundsätzlich genauso wie im CBC-CS1-Modus berechnet, nur dass zum Abschluss die beiden letzten Tupel der Geheimtextnachricht vertauscht werden. Mit den Bezeichnungen aus der Beschreibung des CBC-CS1-Modus gilt also in diesem Fall:

$$E\text{-CBC-CS2}_{k,IV}(m_1, \dots, m_{r-1}, m_r^*) := (c_1, \dots, c_{r-2}, c_r, c_{r-1}^*)$$

Betriebsmodus CBC-CS3 (Ciphertext Stealing for CBC, Variante 3)

Die Verschlüsselung einer Klartextnachricht wird im CBC-CS3-Modus wiederum zunächst wie im CBC-CS1-Modus berechnet, nur dass danach noch die beiden letzten Tupel der Geheimtextnachricht immer vertauscht werden (also auch dann, falls die Länge der Klartextnachricht ein Vielfaches der Blocklänge ist).

Frage 9

Eine Implementierung einer der drei oben beschriebenen CBC-CS-Varianten steht in Java für AES nach einer Initialisierung der Cipher-Klasse mit "AES/CTS/NOPADDING" zur Verfügung. Bestimmen Sie mit einem kleinen Testprogramm, welche Variante dies ist.

In einer Reihe von Betriebsmodi wird ein Blockverschlüsselungsverfahren eingesetzt, um einen Schlüsselstrom zu erzeugen, der in einem additiven synchronen Schlüsselstromverfahren genutzt werden kann. Wie bei jedem additiven synchronen Schlüsselstromverfahren ist streng darauf zu achten, dass kein Schlüsselstrom mehrfach verwendet wird. Hierzu ist die Verwendung von Nonce-Werten geeignet vorzusehen (s. Abschnitt 1.3).

Als erstes Beispiel wird der Counter-Modus beschrieben. Hierbei wird von der in (2.1) definierten Operation $\boxplus$ Gebrauch gemacht (Addition modulo $2^{8 \cdot n}$ der m entsprechenden Zahl mit einer weiteren ganzen Zahl):

Betriebsmodus CTR (Counter)

Im CTR-Modus wird mit einem Blockverschlüsselungsverfahren E jedes Tupel

$$(m_1, \dots, m_r)$$

von Blöcken $m_i \in \mathcal{M}$ für einen gegebenen Schlüssel $k \in \mathcal{K}$ und einen Nonce-Wert

$$Ctr \in \mathcal{M}$$

folgendermaßen blockweise verschlüsselt:

$$c_i := m_i + E_k(Ctr \boxplus (i-1)) \quad \text{für } i = 1, \dots, r$$

$$E\text{-CTR}_{k,Ctr}(m_1, \dots, m_r) := (c_1, \dots, c_r)$$

Die Verschlüsselung im CTR-Modus ist in Abbildung 2.8 dargestellt.

Der CTR-Modus definiert ein synchrones additives Stromverschlüsselungsverfahren, dessen Schlüsselstrom aus den Bytewerten der Blöcke

$$z_i = E_k(ctr \boxplus (i-1)) \quad \text{für } i = 1, 2, \dots$$

besteht.

Die Entschlüsselungsfunktion ist daher identisch zur Verschlüsselungsfunktion:

$$D-CTR_{k,ctr}(c_1, \dots, c_r) = E-CTR_{k,ctr}(c_1, \dots, c_r)$$

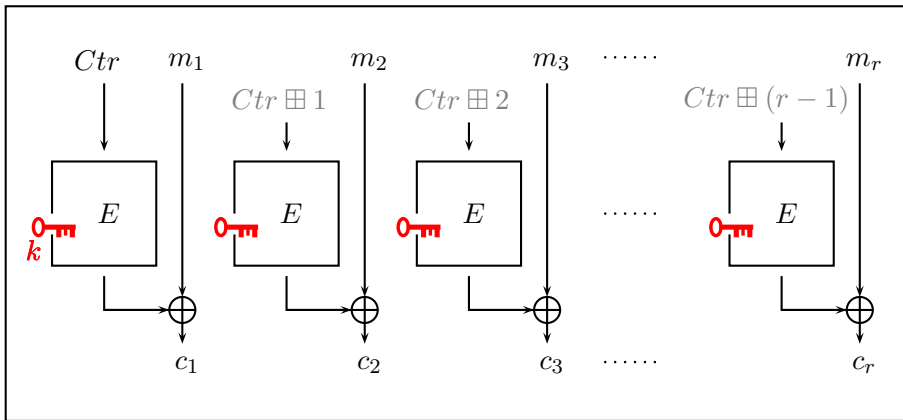


Abb. 2.8 Blockverschlüsselungsverfahren im Counter-Modus

Warnung Zum Schutz vor Wörterbuchangriffen ist bei einer Nutzung des CTR-Modus darauf zu achten, dass nicht nur die ctr -Werte sondern auch die im Rahmen einer Verschlüsselung vorkommenden Werte $ctr \boxplus 1, \dots, ctr \boxplus (r-1)$ Nonce-Werte sind, d. h. nicht bei anderen Verschlüsselungen (mit gleichem Schlüssel k) erneut auftreten. ◀

Wichtig Da die Blöcke $z_i = E_k(ctr \boxplus (i-1))$ zur Erzeugung eines Schlüsselstroms benutzt werden, muss die Länge einer zu verschlüsselnden Nachricht kein Vielfaches der Blocklänge sein, d. h. auf ein Padding kann verzichtet werden.

Zur Verschlüsselung einer Klartextnachricht, die aus l Bytes besteht, werden

$$r = \lceil l/n \rceil$$

Schlüsselstromblöcke z_i benötigt, wobei vom letzten Block z_r nur die ersten

$$((l-1) \bmod n) + 1$$

Bytes für die XOR-Operation verwendet werden. ◀

Frage 10

Sind $l, n \in \mathbb{N}$, so lässt sich l auf eine eindeutig bestimmte Weise als Summe eines ganzzahligen Vielfachen von n und einer positiven Zahl $\leq n$ schreiben. D. h., es gibt eindeutig bestimmte v, w mit:

$$l = v \cdot n + w, \quad v \in \mathbb{Z}, \quad w \in \{1, 2, \dots, n\}$$

Zeigen Sie:

$$v = \lfloor (l-1)/n \rfloor$$

$$w = ((l-1) \bmod n) + 1$$

Frage 11

Schreiben Sie den Quellcode für eine Klasse `KeyStreamGenerator_AES_CTR`, die die Klasse `SynchronousKeyStreamGenerator` aus Abbildung 1.11 erweitert und den durch den CTR-Modus definierten Schlüsselstromgenerator auf Basis des AES-Blockverfahrens (Cipher-Instanz vom Typ `"AES/ECB/NOPADDING"`) implementiert.

Aufgabe 6

Testen Sie Ihre Lösung zu Frage 11 indem Sie verschiedene Klartextnachrichten (unterschiedlicher Längen) für verschiedene Ctr-Werte mit der von Ihnen implementierten Methode und zum Vergleich mit einer Cipher-Instanz vom Typ `"AES/CTR/NOPADDING"` verschlüsseln.

Eine Methode für einen solchen Vergleich findet sich beispielhaft in Abbildung 2.9.

```

1  public static void main(String[] args) throws Exception {
    byte[] key = Dump.hexString2byteArray(
        "0102030405060708090A0B0C0D0E0F10");
    SecretKey secretKey = new SecretKeySpec(key, "AES");

5
    Cipher cipher
        = Cipher.getInstance("AES/CTR/NOPADDING");

    byte[] ctr = Dump.hexString2byteArray(
10     "FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF");
    IvParameterSpec iv = new IvParameterSpec(ctr);
    cipher.init(Cipher.ENCRYPT_MODE, secretKey, iv);

    byte[] m = new byte[35];

15
    byte[] c = cipher.doFinal(m);
    System.out.println("AES-CTR-Verschlüsselung mit "
        + "Cipher-Instanz vom Typ AES/CTR/NOPADDING:");
    System.out.println(Dump.dump(c));
20    System.out.println();

    KeyStreamGenerator_AES_CTR ksg
        = new KeyStreamGenerator_AES_CTR();
    ksg.initState(key, ctr);
25    AdditiveStreamCipher.encrypt(ksg, m);
    System.out.println("AES-CTR-Verschlüsselung mit "
        + "KeyStreamGenerator_AES_CTR-Implementierung:");
    System.out.println(Dump.dump(m));
    System.out.println();
30 }

```

Abb. 2.9 AES-CTR-Verschlüsselung mit einer Cipher-Instanz vom Typ `"AES/CTR/NOPADDING"` und mit der Implementierung `KeyStreamGenerator_AES_CTR` aus Frage 11

Eine weitere Konstruktion eines synchronen Schlüsselstromgenerators liefert die Nutzung eines Blockverschlüsselungsalgorithmus E im *Output-Feedback-Modus* (OFB-Modus), welche in Abbildung 2.10 dargestellt ist. Vom OFB-Modus gibt es die Varianten OFB- $8n$, wobei $n \in \mathbb{N}$ eine Zahl ist, die kleiner oder gleich der Blocklänge von E ist. Bei einer solchen Variante werden immer nur n Bytes eines Geheimtextblocks für die XOR-Verknüpfung mit einem entsprechend kleineren Klartextblock m_i der Länge n verwendet. Als Feedback werden auch nur diese n Bytes eines Geheimtextblocks genutzt. Der Klartextblock für die folgende Blockverschlüsselung setzt sich daher aus diesen n Bytes und weiteren $l - n$ Bytes aus dem vorherigen Klartextblock zusammen, wobei l die Blocklänge von E bezeichnet.

Die Details der OFB- $8n$ -Modi können der beispielhaften Implementierung aus Abbildung 2.11 entnommen werden.

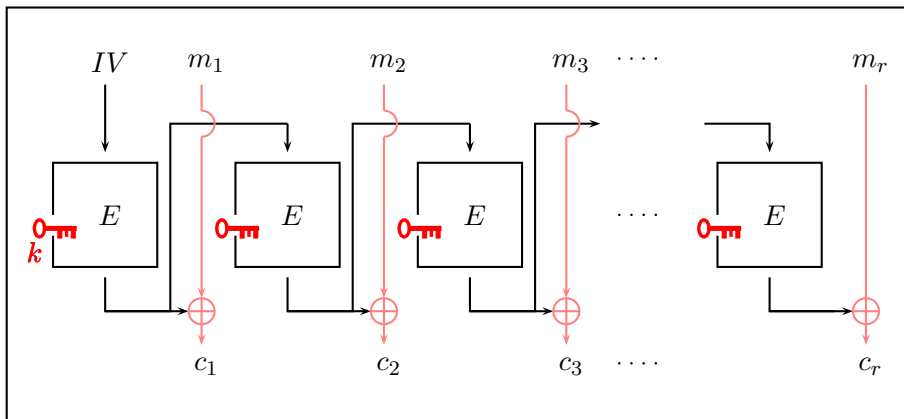


Abb. 2.10 Blockverschlüsselungsverfahren im OFB-Modus

```

1 public class KeyStreamGenerator_AES_OFB
  extends SynchronousKeyStreamGenerator {

    // Zustand des Schlüsselstromgenerators
5 private byte[] mBlock = new byte[16];
  private byte[] cBlock;
  private int c = 0;
  private Cipher aes;
  private int n; // = nBits/8

10 /**
   * Initialisierung des Schlüsselstromgenerators
   * (Funktion s_0)
   *
15 * @param nBits Anzahl der Bits, die pro
   * Blockverschlüsselung als Schlüsselstrom und Feedback
   * genutzt werden. nBits = 8, 16, 24, ..., 120 oder 128
   * @param key Schlüssel zu Initialisierung des
   * Schlüsselstromgenerators (ByteArray der Länge 16)
20 * @param iv IV-Wert zu Initialisierung des
   * Schlüsselstromgenerators (ByteArray der Länge 16)
   */
  public void initState(int nBits, byte[] key, byte[] iv) {

```

```

24     if ((nBits & 0x07) != 0 || (nBits < 0)
25         || (nBits > 128)) {
            throw new IllegalArgumentException(
                "Invalid value for nBits!");
        }
        n = nBits >> 3;

30     if ((key.length != 16) || (iv.length != 16)) {
            throw new IllegalArgumentException(
                "Invalid length of key or iv!");
        }

35     System.arraycopy(iv, 0, mBlock, 0, 16);

    try {
        aes = Cipher.getInstance("AES/ECB/NOPADDING");
        aes.init(
40            Cipher.ENCRYPT_MODE, new SecretKeySpec(key, "AES"));
        cBlock = aes.doFinal(iv);
    } catch (Exception ex) {
        throw new RuntimeException(ex.toString());
45    }
}

/**
 * Berechnung des nächsten Schlüsselstrombytes.
 * (Entspricht eines einmaligen Aufrufs der Funktionen s
 * und S.)
 *
 * @return Nächstes Schlüsselstrombyte
 */
55 @Override
public byte getNextKeyStreamByte() {
    // Update des Zustands (Funktion s)
    if (c == n) {
        System.arraycopy(mBlock, n, mBlock, 0, 16 - n);
        System.arraycopy(cBlock, 0, mBlock, 16 - n, n);
60        try {
            cBlock = aes.doFinal(mBlock);
        } catch (Exception ex) {
            throw new RuntimeException(ex.toString());
        }
65        c = 0;
    }
    c++;

70    // Extraktion des nächsten Schlüsselstrombytes
    // (Funktion S)
    return cBlock[c - 1];
}
}

```

Abb. 2.11 Implementierung von AES-OFB-8n-Schlüsselstromgeneratoren

Eine große Ähnlichkeit zum OFB-Modus weist der *Cipher-Feedback-Modus* (CFB-Modus) auf. Die Konstruktion ist in Abbildung 2.12 dargestellt. Wie beim OFB-Modus gibt es entsprechende Varianten CFB-8n für natürliche Zahlen $n \leq l$, wobei l die Blocklänge von E bezeichnet.

Die Implementierung der OFB-8n-Modi aus Abbildung 2.11 kann durch wenige Änderungen in eine Implementierung der CFB-8n-Modi überführt werden. Die CFB-8n-Modi definieren keinen synchronen Schlüsselstromgenerator. Eine Implementierung der CFB-8n-Modi muss daher anstelle der Klasse `SynchronousKeyStreamGenerator` die Klasse

KeyStreamGenerator erweitern (s. Z. 2 in Abb. 2.11). Darüber hinaus sind nur wenige Änderungen bei der Implementierung der Methode `getNextKeyStreamByte(...)` notwendig. Die für die CFB-8n-Modi angepasste `getNextKeyStreamByte(...)`-Methode ist in Abbildung 2.13 dargestellt.

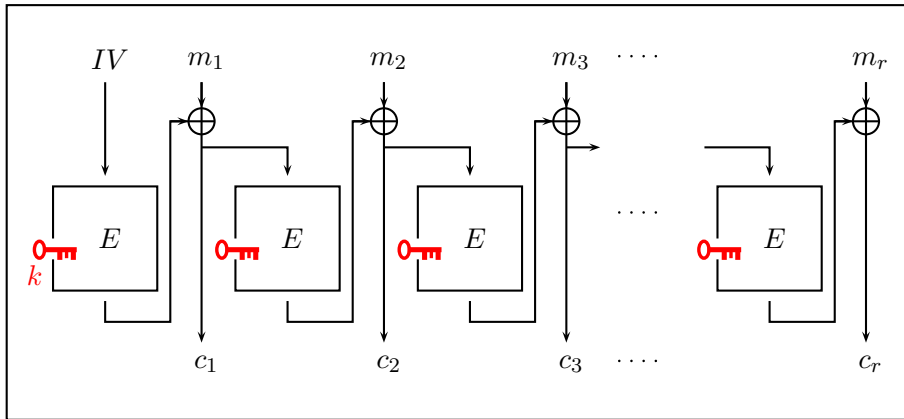


Abb. 2.12 Blockverschlüsselungsverfahren im CFB-Modus

```

1  public byte getNextKeyStreamByte(byte mByte) {
    // Update des Zustands (Funktion s)
    if (c == n) {
        System.arraycopy(mBlock, n, mBlock, 0, 16 - n);
        System.arraycopy(cBlock, 0, mBlock, 16 - n, n);
        try {
            cBlock = aes.doFinal(mBlock);
        } catch (Exception ex) {
            throw new RuntimeException(ex.toString());
        }
        c = 0;
    }
    byte cByte = cBlock[c];
    cBlock[c] ^= mByte;
    c++;

    // Extraktion des nächsten Schlüsselstrombytes
    // (Funktion S)
    return cByte;
20 }

```

Abb. 2.13 `getNextKeyStreamByte(...)`-Methode für AES-CFB-8n-Schlüsselstromgeneratoren

2.4 Konstruktionsprinzipien von Blockverschlüsselungsverfahren

Zur Konstruktion von Verschlüsselungsverfahren unter Berücksichtigung aller bekannten Angriffsmöglichkeiten bedarf es eines hochspezialisierten Fachwissens, welches hier nicht vermittelt werden kann. Nichtsdestotrotz können einige grundsätzliche Konstruktionsprinzipien beschrieben und im folgenden Abschnitt am Beispiel des *Advanced Encryption Standards* (AES) nachvollzogen werden.

Die grundsätzliche Anforderung an eine gute Implementierbarkeit (in Software und Hardware) und die Notwendigkeit im Rahmen einer nachvollziehbaren Kryptoanalyse, die Qualität des Verfahrens begründen zu können, führt bei den üblichen Blockverschlüsselungsverfahren zu einer Definition von elementaren Operationen, die auf den Blöcken eine Permutation bewirken und in einer Schleife wiederholt angewendet werden. Die Schleifendurchläufe werden auch als *Runden* bezeichnet, wobei sich die erste oder letzte Runde auch etwas von den sonstigen Runden unterscheiden kann.

Bei den elementaren Operationen handelt es sich insbesondere um

- *Transpositionen* von Bitwerten innerhalb eines Block, definiert durch eine Permutation der $8 \cdot n$ Bitstellen eines Blocks,
- *Substitutionen* von Werten innerhalb kleinerer Teilblöcke, definiert durch eine Abbildungsvorschrift für eine Permutation der möglichen Werte eines Teilblocks. Die Abbildungsvorschrift kann dabei durch eine Rechenvorschrift oder eine Wertetabelle spezifiziert sein. In einem solchen Kontext werden die Wertetabellen häufig als *S-Boxen* bezeichnet.

Natürlich muss auch der Schlüsselwert k bei den Operationen auf den Blöcken eine Berücksichtigung finden. Wird das Gesamtverfahren in r Runden aufgeteilt, so werden hierzu aus dem Schlüssel k zunächst *Rundenschlüssel* k_i ($i = 1, \dots, r$) abgeleitet. Dieser Schritt wird als *Schlüsselexpansion* bezeichnet. In jeder Runde fließt der entsprechende Rundenschlüssel dann bei den Operationen auf den Blöcken ein, entweder durch eine simple XOR-Operation oder durch eine Steuerung (Parametrisierung) der oben genannten elementaren Operationen.

Bitte beachten Sie: Werden mehrere Blöcke mit dem gleichen Schlüssel verschlüsselt, so können die einmal berechneten Rundenschlüssel zwischengespeichert und direkt wiederverwendet werden.

Bei einer Entschlüsselung muss das skizzierte Verschlüsselungsverfahren rückwärts durchlaufen werden. Der prinzipielle Aufbau bleibt dabei offensichtlich gleich. Es werden jeweils die Umkehrabbildungen der in den elementaren Operationen verwendeten Transpositionen und Substitutionen und die gleichen Rundenschlüssel benötigt.

2.5 DES und Triple-DES

Zu Beginn des Jahres 1977 wurde vom *National Institute of Standards and Technology* (NIST) der *Data Encryption Standard* (DES) als *Federal Information Processing Standard* FIPS PUB 46 veröffentlicht. Zuletzt wurde der Standard im Jahr 1999 in der Version FIPS PUB 46-3 publiziert, zurückgezogen wurde der Standard im Jahr 2005.

Der Standard beschreibt ein Blockverschlüsselungsverfahren auf Blöcken, die 8 Bytes umfassen. Der Schlüsselraum hat eine Größe von $|\mathcal{K}| = 2^{56}$. Grundlage des Verfahrens bildete ein unter dem Namen *Lucifer* bei der Firma IBM von H. Feistel¹ und weiteren Experten entwickelter Algorithmus. Ohne auf die Details der DES-Algorithmus einzugehen halten wir fest:

DES

Sowohl die Blöcke als auch die Schlüssel des DES-Verfahrens setzen sich jeweils aus 8 Bytes zusammen:

$$\mathcal{M} = (\mathbb{Z}_2^8)^8 = \{(m_1, m_2, \dots, m_8) \mid m_i \in \mathbb{Z}_2^8\}$$

$$\mathcal{K} = \{(k_1, k_2, \dots, k_8) \mid k_i \in \mathbb{Z}_2^8, \text{parity}(k_i) = 1\} \subseteq (\mathbb{Z}_2^8)^8$$

¹ Horst Feistel (*1915 Berlin, †1990)

Hierbei ist für $k_i \in \mathbb{Z}_2^8$ die *Parität* (*parity*) definiert durch:

$$\text{parity} : \mathbb{Z}_2^8 \rightarrow \mathbb{Z}_2, \quad \text{parity} \left(\sum_{i=0}^7 b_i \cdot 2^i \right) := \left(\sum_{i=0}^7 b_i \right) \bmod 2$$

Für jedes Schlüsselbyte muss die Anzahl der 1-Bits in der Binärdarstellung des Bytes also ungerade sein.

Kommentar Für jedes DES-Schlüsselbyte können die Werte von 7 Bits beliebig gewählt werden, das achte Bit ist dann aufgrund der Paritätsbedingung eindeutig festgelegt. Daher ist die Bitlänge des Schlüsselraums tatsächlich $8 \cdot 7 = 56$. ◀

Aufgrund des nach heutigen Maßstäben sehr kleinen Schlüsselraums konnten erfolgreiche Brute-Force-Angriffe seit dem Jahr 1997 öffentlich (d. h. mit eher bescheidenem Budget) durchgeführt und publiziert werden.

Eine Möglichkeit der Vergrößerung des Schlüsselraums bietet die mehrfache Verwendung des DES-Verschlüsselungsverfahrens, wobei bei jeder Verwendung ein anderer (Teil-)Schlüssel zum Einsatz gebracht werden kann. Allerdings genügt es nicht, das Verfahren zweimal hintereinander auszuführen, um eine Sicherheit zu erreichen, die einer Verdopplung der Bitlänge des Schlüsselraums entspricht. Bei der zweifachen Anwendung eines Verschlüsselungsverfahrens sind nämlich sogenannte Meet-in-the-Middle-Angriffe möglich, die in Abschnitt 2.6 behandelt werden.

Um die effektive Größe des Schlüsselraums signifikant zu erhöhen, ist daher eine dreifache Hintereinanderausführung des DES-Verfahrens notwendig, welche als *Triple-DES* bezeichnet wird. Natürlich verschlechtert sich durch eine dreifache Hintereinanderausführung des Algorithmus auch die Ausführungsgeschwindigkeit um den Faktor drei. Üblicherweise wird *Triple-DES* im EDE-Modus und mit einem 16 Byte langen Schlüssel (entspricht einer effektiven Schlüssellänge von 112 Bits) wie folgt ausgeführt:

Triple-DES (3DES)

Die Blöcke des 3DES-Verfahrens setzen sich wie beim DES-Verfahren aus 8 Bytes zusammen:

$$\mathcal{M} = (\mathbb{Z}_2^8)^8 = \{(m_1, m_2, \dots, m_8) \mid m_i \in \mathbb{Z}_2^8\}$$

Der Schlüsselraum des 3DES-Verfahrens besteht aus der Menge der Paare von DES-Schlüsseln:

$$\mathcal{K} = \{(k_1, k_2, \dots, k_8) \mid k_i \in \mathbb{Z}_2^8, \text{parity}(k_i) = 1\}^2$$

Bezeichnen E_{DES} und D_{DES} die DES-Ver- beziehungsweise DES-Entschlüsselungsfunktion, so ist das 3DES-Verschlüsselungsverfahren definiert durch:

$$E \left(\left(k^{(1)}, k^{(2)} \right), m \right) := E_{DES} \left(k^{(1)}, D_{DES} \left(k^{(2)}, E_{DES} \left(k^{(1)}, m \right) \right) \right) \quad (2.2)$$

Die Hintereinanderausführung der DES-Verfahren ist in Abbildung 2.14 dargestellt.

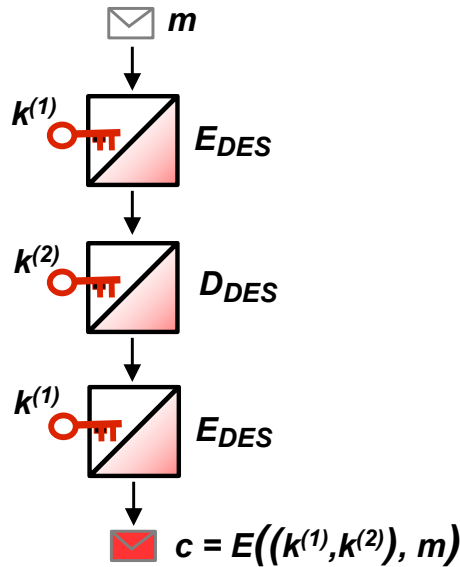


Abb. 2.14 Triple-DES-Verschlüsselung des Blocks m mit dem Schlüssel $k = (k^{(1)}, k^{(2)})$.

Frage 12

Geben Sie entsprechend der Definition der 3DES-Verschlüsselungsfunktion in (2.2) eine Abbildungsvorschrift für die 3DES-Entschlüsselungsfunktion an.

Frage 13

Untersuchen Sie, was passiert, wenn eine 3DES-Verschlüsselung mit einem Schlüssel der Form $k = (\tilde{k}, \tilde{k})$ ausgeführt wird, d. h., falls gilt:

$$k^{(1)} = k^{(2)} = \tilde{k}$$

2.6 Meet-in-the-Middle-Angriffe

Wie bereits bei der Einführung des 3DES-Verfahrens erwähnt, lässt sich durch eine zweifache Anwendung eines kryptographischen Verschlüsselungsverfahrens eine Verdopplung der Bitlänge des für einen Brute-Force-Angriffs zu durchsuchenden Schlüsselraums nicht erreichen. Der Aufwand für einen Brute-Force-Angriff wird sogar nur um einen unbedeutend kleinen Faktor größer, wenn ein hinreichend großer Speicher zur Verfügung steht. Ein solcher Angriff, der *Meet-in-the-Middle-Angriff*, wird im Folgenden beschrieben.

Es sei also ausgehend von einem Verschlüsselungsverfahren E mit Schlüsselraum $\mathcal{K}$ das $2E$ -Verschlüsselungsverfahren $2E$ mit Schlüsselraum $\mathcal{K}^2$ definiert durch:

$$2E((k_1, k_2), m) := E(k_2, E(k_1, m))$$

Die Konstruktion von $2E$ ist in Abbildung 2.15 dargestellt.

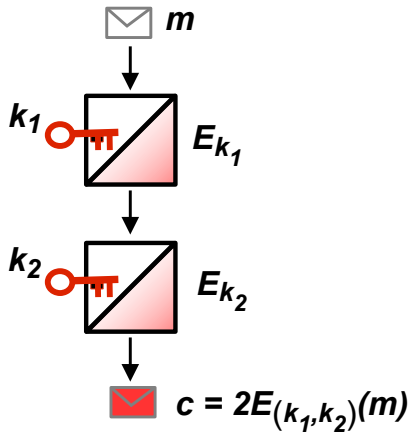


Abb. 2.15 Konstruktion des Verschlüsselungsverfahrens $2E$

Nehmen wir nun an, dass für einen unbekannten Schlüssel k mit den Teilschlüsseln k_1 und k_2 , d. h. für

$$k = (k_1, k_2) \in \mathcal{K}^2$$

einige zusammengehörige Klartext- und Geheimtextblöcke (m_i, c_i) bezüglich der Verschlüsselung mit $2E_{(k_1, k_2)}$ bekannt sind:

$$c_i = E_{k_2}(E_{k_1}(m_i)) \quad (2.3)$$

Nehmen wir weiterhin an, dass ein hinreichend großer Speicher zur Verfügung steht, um für alle $\tilde{k}_1 \in \mathcal{K}$ den Block

$$c_{\tilde{k}_1} := E(\tilde{k}_1, m_1)$$

zusammen mit $\tilde{k}_1$ in einer Tabelle zu speichern. Beispielhaft ist der Anfang und das Ende einer solchen Tabelle für das DES-Verschlüsselungsverfahren in Abbildung 2.16 dargestellt.

$\tilde{k}_1$	$c_{\tilde{k}_1} = E(\tilde{k}_1, m_1)$
0101010101010101	8CA64DE9C1B123A7
0101010101010102	2F3F20C13DEF464C
0101010101010104	6CB9F25ABAAA9DDD
0101010101010107	A28C9CC39D2A5B8D
$\vdots$	$\vdots$
FEFEFEFEFEFEFEF8	92A838AD92ADA020
FEFEFEFEFEFEFEFB	8D8CA49AF5475AB8
FEFEFEFEFEFEFEFD	614650FAE8FC8C0
FEFEFEFEFEFEFEFE	CAAAAF4DEAF1DBAE

Abb. 2.16 DES-Verschlüsselung des Blocks $m_1 = 0000000000000000$ für alle möglichen Schlüssel $\tilde{k}_1$ des DES-Verschlüsselungsverfahrens.

Da (2.3) äquivalent zu folgender Übereinstimmung der Werte in der "Mitte" des Algorithmus

$$D_{k_2}(c_i) = E_{k_1}(m_i) \quad (2.4)$$

ist, kann nach einer einmaligen Sortierung der Tabelle aus Abbildung 2.16 nach den Werten in der zweiten Spalte, für jeden Wert $\tilde{k}_2 \in \mathcal{K}$ in der Tabelle nachgeschlagen werden, ob der Block

$$d_{\tilde{k}_2} := D_{\tilde{k}_2}(c_1)$$

in der zweiten Spalte auftritt, und für welche $\tilde{k}_1 \in \mathcal{K}$ gilt:

$$D_{\tilde{k}_2}(c_1) = d_{\tilde{k}_2} = c_{\tilde{k}_1} = E_{\tilde{k}_1}(m_1) \quad (2.5)$$

Durchschnittlich (Erwartungswert) ist zu jedem $\tilde{k}_2 \neq k_2$ mit

$$\frac{|\mathcal{K}|}{|\mathcal{M}|}$$

solchen Übereinstimmungen zu rechnen. Um für eine solche Übereinstimmung zu überprüfen, ob $(\tilde{k}_1, \tilde{k}_2)$ gleich (k_1, k_2) ist, können die weiteren zusammengehörigen Klartext- und Geheimtextblöcke $(m_2, c_2), (m_3, c_3), \dots$ überprüft werden. Gilt eine der Gleichungen

$$D_{\tilde{k}_2}(c_2) = E_{\tilde{k}_1}(m_2), \quad D_{\tilde{k}_2}(c_3) = E_{\tilde{k}_1}(m_3), \quad \dots$$

nicht, so ist $(\tilde{k}_1, \tilde{k}_2) \neq (k_1, k_2)$. Andernfalls ist (k_1, k_2) (mit großer Wahrscheinlichkeit, s. Frage 14) gefunden.

Bitte beachten Sie, dass bei dem Angriff nicht der Schlüsselraum $\mathcal{K}^2$ von $2E$ durchlaufen wird, sondern nur zweimal der Schlüsselraum $\mathcal{K}$ von E .

Kommentar Um abzuschätzen, wie viele zusammengehörigen Klartext- und Geheimtextblöcke $(m_1, c_1), (m_2, c_2), \dots$ zur eindeutigen Bestimmung eines Schlüssels (k_1, k_2) für das Verschlüsselungsverfahren $2E$ benötigt werden, können für alle Schlüssel $(\tilde{k}_1, \tilde{k}_2) \neq (k_1, k_2)$ die Abbildungen $2E_{(\tilde{k}_1, \tilde{k}_2)}$ als zufällig gewählte Permutationen aus $\Sigma_{\mathcal{M}}$ betrachtet werden.

Für $(m_1, c_1), (m_2, c_2), \dots, (m_r, c_r)$ und eine zufällig gewählte Permutationen aus $\sigma \in \Sigma_{\mathcal{M}}$ gilt

$$\sigma(m_1) = c_1, \quad \sigma(m_2) = c_2, \quad \dots, \quad \sigma(m_r) = c_r \quad (2.6)$$

mit einer Wahrscheinlichkeit von:

$$p = \frac{1}{|\mathcal{M}|^r}$$

Der Erwartungswert für die Anzahl der Permutationen $\sigma = 2E_{(\tilde{k}_1, \tilde{k}_2)}$ mit $(\tilde{k}_1, \tilde{k}_2) \neq (k_1, k_2)$, die (2.6) erfüllen, ist:

$$\frac{|\mathcal{K}|^2 - 1}{|\mathcal{M}|^r}$$

Die Wahrscheinlichkeit, dass keine der Permutationen $\sigma = 2E_{(\tilde{k}_1, \tilde{k}_2)}$ mit $(\tilde{k}_1, \tilde{k}_2) \neq (k_1, k_2)$, die Bedingung (2.6) erfüllt, beträgt:

$$\begin{aligned} \left(1 - \frac{1}{|\mathcal{M}|^r}\right)^{|\mathcal{K}|^2 - 1} &\approx \left(1 - \frac{\frac{|\mathcal{K}|^2}{|\mathcal{M}|^r}}{|\mathcal{K}|^2}\right)^{|\mathcal{K}|^2} \\ &\approx e^{-\frac{|\mathcal{K}|^2}{|\mathcal{M}|^r}} \\ &\approx 1 - \frac{|\mathcal{K}|^2}{|\mathcal{M}|^r} \quad \text{für } |\mathcal{M}|^r \gg |\mathcal{K}|^2 \end{aligned} \quad (2.7)$$



Frage 14

Es seien m_1, m_2, m_3, c_1, c_2 und c_3 zufällig gewählte Blöcke für das DES-Verschlüsselungsverfahren E .

- (i) Schätzen Sie die Anzahl der Schlüssel $(k_1, k_2) \in \mathcal{K}^2$ ab, für die gilt:

$$2E_{(k_1, k_2)}(m_1) = c_1$$

- (ii) Schätzen Sie die Wahrscheinlichkeit dafür ab, dass es mindestens einen Schlüssel $(k_1, k_2) \in \mathcal{K}^2$ gibt, mit:

$$2E_{(k_1, k_2)}(m_i) = c_i \quad \text{für } i = 1 \text{ und } i = 2$$

- (iii) Schätzen Sie die Wahrscheinlichkeit dafür ab, dass es mindestens einen Schlüssel $(k_1, k_2) \in \mathcal{K}^2$ gibt, mit:

$$2E_{(k_1, k_2)}(m_i) = c_i \quad \text{für } i = 1, 2 \text{ und } 3$$

2.7 AES

Bei einem vom NIST organisierten mehrstufigen Wettbewerb zur Auswahl eines *Advanced Encryption Standards (AES)* gewann der von J. Daemen² und V. Rijmen³ entwickelte *Rijndael*-Algorithmus. Die offizielle Spezifikation erfolgte 2001 (FIPS PUB 197).

AES-128

Spezifiziert ist AES für Schlüssel der Bitlängen 128, 192 und 256. Der grundsätzliche Aufbau der entsprechenden Algorithmen AES-128, AES-192 und AES-256 ist jedoch gleich. Die folgende Beschreibung beschränkt sich daher auf eine Betrachtung von AES-128 und die Bezeichnung AES (ohne weitere Nennung der Schlüssellänge) wird synonym für AES-128 benutzt.

Sowohl die Blöcke als auch die Schlüssel setzen sich jeweils aus 16 Bytes zusammen:

$$\mathcal{M} = (\mathbb{Z}_2^8)^{16} = \{(m_1, m_2, \dots, m_{16}) \mid m_i \in \mathbb{Z}_2^8\}$$

$$\mathcal{K} = (\mathbb{Z}_2^8)^{16} = \{(k_1, k_2, \dots, k_{16}) \mid k_i \in \mathbb{Z}_2^8\}$$

Blöcke werden zur Beschreibung des AES-Algorithmus mit den $(4, 4)$ -Matrizen mit Einträgen aus $\mathcal{Z} = \mathbb{Z}_2^8$ identifiziert und in dieser Form auch als *State-Array* bezeichnet. Ein Eingabeblock $(m_1, m_2, \dots, m_{16})$ wird hierbei zunächst wie folgt geschrieben:

$$\mathbf{S} = \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix} := \begin{pmatrix} m_1 & m_5 & m_9 & m_{13} \\ m_2 & m_6 & m_{10} & m_{14} \\ m_3 & m_7 & m_{11} & m_{15} \\ m_4 & m_8 & m_{12} & m_{16} \end{pmatrix}$$

Im AES sind vier elementare Operationen auf den als State-Arrays dargestellten Blöcken definiert:

² Joan Daemen (*1965)

³ Vincent Rijmen (*1970)

(1) SubBytes()

SubBytes() führt die in Abbildung 2.17 definierte Substitution S auf allen Einträgen des State-Arrays aus:

$$\begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix} \mapsto \begin{pmatrix} S(s_{0,0}) & S(s_{0,1}) & S(s_{0,2}) & S(s_{0,3}) \\ S(s_{1,0}) & S(s_{1,1}) & S(s_{1,2}) & S(s_{1,3}) \\ S(s_{2,0}) & S(s_{2,1}) & S(s_{2,2}) & S(s_{2,3}) \\ S(s_{3,0}) & S(s_{3,1}) & S(s_{3,2}) & S(s_{3,3}) \end{pmatrix}$$

	*0	*1	*2	*3	*4	*5	*6	*7	*8	*9	*A	*B	*C	*D	*E	*F
0*	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1*	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2*	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3*	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4*	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5*	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6*	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7*	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8*	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9*	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A*	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B*	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C*	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D*	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E*	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F*	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Abb. 2.17 Die der Operation SubBytes() zugrundeliegende Permutation $S \in \Sigma_{\mathcal{X}}$, wobei das Bild des Bytes XY unter S im Schnittpunkt der mit X* bezeichneten Zeile und der mit *Y bezeichneten Spalte steht.

(2) ShiftRows()

ShiftRows() führt eine Transposition aus, die folgendermaßen auf Ebene der State-Array-Einträge definiert ist:

$$\begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix} \mapsto \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,1} & s_{1,2} & s_{1,3} & s_{1,0} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,3} & s_{3,0} & s_{3,1} & s_{3,2} \end{pmatrix}$$

(3) MixColumns()

MixColumns() führt eine Substitution auf den Spalten des State-Arrays $\mathbf{S}$ aus, die durch eine Multiplikation mit einer fest definierten Matrix $\mathbf{M}$ erklärt ist:

$$\mathbf{S} \mapsto \mathbf{M} \cdot \mathbf{S}, \quad \text{mit } \mathbf{M} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}$$

Zur Berechnung der Einträge in $\mathbf{M} \cdot \mathbf{S}$ werden Summen und Produkte von Bytewerten **nicht** in $\mathbb{Z}_{256}$ berechnet. (Andernfalls definierte die Matrizenmultiplikation auch keine injektive Abbildung und eine Verschlüsselung wäre dann nicht mehr eindeutig umkehrbar!) Zur Berechnung der Einträge in $\mathbf{M} \cdot \mathbf{S}$ sind die Bytewerte vielmehr als Elemente des Körpers $\mathbb{F}_{256}$ aufzufassen, der bezüglich des irreduziblen Polynoms

$$m(x) := x^8 + x^4 + x^3 + x + 1 \in \mathbb{F}_2[x] \quad (2.8)$$

definiert ist.

Details zur Definition und zur Durchführung von Berechnungen in endlichen Körpern von Primzahlpotenzordnung $\mathbb{F}_{p^n}$ mit einer Primzahl p und einem $n \in \mathbb{N}, n > 1$ sind im folgenden Kapitel 3 zusammengefasst.

(4) AddRoundKey()

Nach einer initialen Schlüsselexpansion, die hier nicht weiter besprochen werden soll, steht für jede Runde ein aus 16 Bytes zusammengesetzter Rundenschlüssel zur Verfügung, der jeweils als Matrix

$$\mathbf{K} = \begin{pmatrix} k_{0,0} & k_{0,1} & k_{0,2} & k_{0,3} \\ k_{1,0} & k_{1,1} & k_{1,2} & k_{1,3} \\ k_{2,0} & k_{2,1} & k_{2,2} & k_{2,3} \\ k_{3,0} & k_{3,1} & k_{3,2} & k_{3,3} \end{pmatrix}$$

dargestellt werden kann.

AddRoundKey() führt einfach eine XOR-Operation aller Einträge des State-Arrays $\mathbf{S}$ mit den entsprechenden Einträgen aus $\mathbf{K}$ durch. In $\mathbb{F}_{256}$ entspricht dies einer Addition der Einträge und damit der Matrizenaddition:

$$\mathbf{S} \mapsto \mathbf{K} + \mathbf{S}$$

AES-128 führt in 10 Runden die oben beschriebenen Operationen aus, wobei in der letzten Runde MixColumns() nicht ausgeführt wird. Zusätzlich wird AddRoundKey() einmal zu Beginn des Verfahrens ausgeführt, sodass insgesamt 11 Rundenschlüssel $\mathbf{K}_0, \mathbf{K}_1, \dots, \mathbf{K}_{10}$ zum Einsatz kommen. Abbildung 2.18 enthält den Pseudocode zur Verschlüsselung eines durch das State-Array $\mathbf{S}$ repräsentierten Blocks mit AES-128.

```

AES_Encrypt(S, K0, K1, ..., K10) {
    AddRoundKey(S, K0);
    for( int i=1; i < 10; ++i ){
        SubBytes(S);
        ShiftRows(S);
        MixColumns(S);
        AddRoundKey(S, Ki);
    }
    SubBytes(S);
    ShiftRows(S);
    AddRoundKey(S, K10);
}

```

Abb. 2.18 Pseudocode zur AES-128-Verschlüsselung eines durch $\mathbf{S}$ repräsentierten Blocks mit den Rundenschlüsseln $\mathbf{K}_i$.

Frage 15

Begründen Sie, dass `SubBytes()` eine umkehrbare Abbildung auf der Menge der State-Arrays definiert, indem Sie die Umkehrabbildung $\text{InvSubBytes}() := \text{SubBytes}^{-1}()$ angeben.

Berechnen Sie $\text{InvSubBytes}(\mathbf{S})$ für: $\mathbf{S} = \begin{pmatrix} 6B & 17 & D1 & F2 \\ E1 & 2C & 42 & 47 \\ F8 & BC & E6 & E5 \\ 63 & A4 & 40 & F2 \end{pmatrix}$

Frage 16

Begründen Sie, dass `ShiftRows()` eine umkehrbare Abbildung auf der Menge der State-Arrays definiert, indem Sie die Umkehrabbildung $\text{InvShiftRows}() := \text{ShiftRows}^{-1}()$ angeben.

Frage 17

Begründen Sie, dass `AddRoundKey()` für jeden Rundenschlüssel $\mathbf{K}$ eine umkehrbare Abbildung auf der Menge der State-Arrays definiert, indem Sie die Umkehrabbildung $\text{InvAddRoundKey}() := \text{AddRoundKey}^{-1}()$ angeben.

Mathematischer Exkurs: Endliche Körper $\mathbb{F}_{p^n}$

3

Ist p eine Primzahl, so ist nach Studienheft 01, Satz 2.11

$$\mathbb{F}_p := \mathbb{Z}_p$$

ein Körper.

Körper können auch für beliebige Primzahlpotenzen p^n mit $n \in \mathbb{N}, n > 1$ mithilfe der Polynome mit Koeffizienten aus $\mathbb{F}_p$ vom Grad $< n$ konstruiert werden:

$$\begin{aligned}\mathbb{F}_{p^n} &:= \{a(x) \mid a(x) \in \mathbb{F}_p[x], \text{grad}(a(x)) < n\} \\ &= \{a_{n-1}x^{n-1} + \cdots + a_1x + a_0 \mid a_i \in \mathbb{F}_p\}\end{aligned}$$

Die übliche Polynomaddition definiert bereits eine Addition auf $\mathbb{F}_{p^n}$. Bezüglich der üblichen Polynommultiplikation ist $\mathbb{F}_{p^n}$ allerdings nicht abgeschlossen, da nicht alle Produkte von zwei Polynomen aus $\mathbb{F}_{p^n}$ einen Grad $< n$ haben. Analog zur Konstruktion von $\mathbb{Z}_p$ lässt sich jedoch nach der Festlegung eines irreduziblen Polynoms $p(x)$ vom Grad n (entspricht einer Primzahl in $\mathbb{Z}$) das Ergebnis einer Multiplikation mithilfe einer Division durch $p(x)$ mit Rest reduzieren.

In Analogie zur ganzzahligen Division mit Rest in $\mathbb{Z}$ können wir für die Polynomdivision mit Rest festhalten:

Polynomdivision mit Rest

Ist $\mathbb{F}$ ein Körper und sind $a(x), b(x) \in \mathbb{F}[x], b(x) \neq 0$, so gibt es eindeutig bestimmte Polynome $q(x), r(x) \in \mathbb{F}[x]$ mit:

$$a(x) = b(x) \cdot q(x) + r(x) \quad \text{und} \quad 0 \leq \text{grad}(r(x)) < \text{grad}(b(x)) \quad (3.1)$$

Für den Rest $r(x)$ der Polynomdivision wird auch

$$a(x) \bmod b(x) := r(x) \quad (3.2)$$

geschrieben.

Mit der Bezeichnung aus (3.2) halten wir folgende Definition für endliche Körper mit Primzahlpotenzordnung p^n ($n > 1$) fest:

$\mathbb{F}_{p^n}$ (p Primzahl, $n \in \mathbb{N}, n > 1$)

Gegeben sei ein irreduzibles Polynom:

$$p(x) \in \mathbb{F}_p[x] \quad \text{mit} \quad \text{grad}(p(x)) = n$$

Dann bilden die Polynome der Menge

$$\mathbb{F}_{p^n} := \{a(x) \mid a(x) \in \mathbb{F}_p[x], \text{grad}(a(x)) < n\}$$

zusammen mit der üblichen Polynomaddition und der Multiplikation

$$a(x) \cdot b(x) := (a(x) \cdot b(x)) \bmod p(x) \quad (3.3)$$

einen Körper. (Das Produkt $a(x) \cdot b(x)$ auf der rechten Seite von (3.3) ist hierbei das übliche Produkt von Polynomen.)

Kommentar

■ Alternative Bezeichnung

Für endliche Körper wird häufig auch die Bezeichnung $GF(p)$ bzw. $GF(p^n)$ gewählt. Diese Bezeichnung spiegelt die englischsprachige Bezeichnung für einen endlichen Körper als *Galois¹ field* wider. (Algebraische Körper werden englischsprachig allgemein als *field* bezeichnet.)

■ Körperaxiome

Auf einen formalen Beweis, dass mit der obigen Definition tatsächlich ein Körper definiert wird, d. h., dass alle Körperaxiome erfüllt sind, soll hier verzichtet werden. Anzumerken ist allerdings, dass der Beweis der Körperaxiome mit den gleichen Begründungen wie beim Beweis der Axiome für $\mathbb{F}_p = \mathbb{Z}_p$ gelingt. Insbesondere gilt:

■ Berechnung von Inversen

Der größte gemeinsame Teiler von zwei Polynomen $a(x)$ und $b(x)$ (definiert als normiertes Polynom maximalen Grades, welches $a(x)$ und $b(x)$ ohne Rest teilt), kann mit dem *Euklid'schen Algorithmus* berechnet werden. Ebenfalls kann mit dem *erweiterten Euklid'schen Algorithmus* eine Linearkombination der Ausgangspolynome zur Darstellung des größten gemeinsamen Teilers gefunden werden. Ist $\mathbb{F}_{p^n}$ bezüglich des irreduziblen Polynoms $p(x)$ definiert, so kann daher in Analogie zur in Studienheft 01, Abschnitt 2.5 beschriebenen Vorgehensweise für jedes $a(x) \in \mathbb{F}_{p^n}$ mithilfe einer Anwendung des erweiterten Euklid'schen Algorithmus auf die Polynome $a(x)$ und $b(x)$ das inverse Element zu $a(x)$ in $\mathbb{F}_{p^n}$ bestimmt werden.

■ Eindeutigkeit

Um beliebige Produkte in $\mathbb{F}_{p^n}$ berechnen zu können, muss das zur Definition von $\mathbb{F}_{p^n}$ benutzte irreduzible Polynom $p(x)$ bekannt sein. Insofern wäre eine Bezeichnung wie $\mathbb{F}_{p(x)}$ anstelle von $\mathbb{F}_{p^n}$ sinnvoller.

Es kann aber gezeigt werden, dass für je zwei irreduzible Polynome $p_1(x), p_2(x) \in \mathbb{F}_p[x]$ vom Grad n die durch sie definierten Körper $\mathbb{F}_{p_1(x)}$ und $\mathbb{F}_{p_2(x)}$ gleichartig sind, d. h. es gibt eine bijektive Abbildung von $\mathbb{F}_{p_1(x)}$ nach $\mathbb{F}_{p_2(x)}$, die mit Addition und Multiplikation verträglich ist. (Eine solche Abbildung wird als *Isomorphismus* bezeichnet.) Diese Tatsache rechtfertigt die Schreibweise $\mathbb{F}_{p^n}$.

Allerdings ist bei einer konkreten Verwendung eines solchen Körpers für $n > 1$ immer das Polynom $p(x)$ anzugeben, bezüglich dem die Ergebnisse einer Multiplikation von Polynomen gegebenenfalls zu reduzieren sind.

■ Existenz

Es kann gezeigt werden, dass es zu jeder Primzahl und jedem $n \in \mathbb{N}$ mindestens ein irreduzibles Polynom in $\mathbb{F}_p[x]$ gibt.



Im Weiteren werden wir nur noch Körper betrachten die über $\mathbb{F}_2$ definiert sind, d. h. mithilfe von Polynomen aus $\mathbb{F}_2[x]$ konstruiert werden. Einige elementare Eigenschaften von Polynomen aus $\mathbb{F}_2[x]$ sind in den folgenden Fragen enthalten.

Frage 18

Geben Sie für ein Polynom

$$a(x) = a_n x^n + \cdots + a_1 x + a_0 \in \mathbb{F}_2[x]$$

Kriterien dafür an, dass

- (i) 0 eine Nullstelle von $a(x)$ ist,
- (ii) 1 eine Nullstelle von $a(x)$ ist.

¹ Évariste Galois (*1811 Bourg-la-Reine, †1832 Paris)

Frage 19

Zeigen Sie, dass Quadrate in $\mathbb{F}_2[x]$ folgendermaßen leicht berechnet werden können:

$$(a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0)^2 = a_n x^{2n} + a_{n-1} x^{2n-2} + \cdots + a_1 x^2 + a_0$$

Beispiel 3.1: Irreduzible Polynome über $\mathbb{F}_2$ vom Grad ≤ 4

Polynome vom Grad 1 (lineare Polynome) sind immer irreduzibel. In $\mathbb{F}_2[x]$ sind dies die Polynome:

$$p_{1,1}(x) = x$$

$$p_{1,2}(x) = x + 1$$

In $\mathbb{F}_2[x]$ gibt es insgesamt vier paarweise verschiedene Polynome vom Grad 2. Drei dieser Polynome besitzen eine Nullstelle in $\mathbb{F}_2$ und damit eine Zerlegung in Linearfaktoren. Das einzige irreduzible Polynom in $\mathbb{F}_2[x]$ vom Grad 2 ist:

$$p_2(x) = x^2 + x + 1$$

Ist ein Polynom $p(x)$ dritten Grades nicht irreduzibel, so besitzt $p(x)$ eine Faktorisierung:

$$p(x) = a(x) \cdot b(x) \quad \text{mit } 1 \leq \text{grad}(a(x)) \leq \text{grad}(b(x))$$

Mit

$$\text{grad}(p(x)) = \text{grad}(a(x)) + \text{grad}(b(x))$$

folgt $\text{grad}(a(x)) = 1$, d. h., $a(x)$ ist ein Linearfaktor von $p(x)$ und $p(x)$ besitzt eine Nullstelle. Ein Polynom $p(x)$ dritten Grades ist also genau dann irreduzibel, falls $p(x)$ keine Nullstelle besitzt. In $\mathbb{F}_2[x]$ sind dies die Polynome:

$$p_{3,1}(x) = x^3 + x + 1$$

$$p_{3,2}(x) = x^3 + x^2 + 1$$

Ein Polynom $p(x) \in \mathbb{F}_2[x]$ vierten Grades ist genau dann reduzibel, falls es eine Nullstelle besitzt oder das Quadrat des einzigen irreduziblen Polynoms $p_2(x)$ vom Grad 2 ist:

$$(p_2(x))^2 = (x^2 + x + 1)^2 = x^4 + x^2 + 1$$

Die irreduziblen Polynome über $\mathbb{F}_2$ vom Grad 4 sind:

$$p_{4,1}(x) = x^4 + x + 1$$

$$p_{4,2}(x) = x^4 + x^3 + 1$$

$$p_{4,3}(x) = x^4 + x^3 + x^2 + x + 1$$



Beispiel 3.2: Irreduzibilität von $m(x) = x^8 + x^4 + x^3 + x + 1 \in \mathbb{F}_2[x]$

Die Irreduzibilität des in (2.8) bei der Konstruktion des AES-Verfahrens definierten Polynoms soll überprüft werden.

Wäre $m(x)$ reduzibel, so besäße $m(x)$ einen irreduziblen Faktor $d(x)$ mit:

$$\text{grad}(d(x)) \in \{1, 2, 3, 4\}$$

Wegen

$$m(0) = m(1) = 1 \neq 0$$

besitzt $m(x)$ keinen linearen Faktor. Es bleibt zu überprüfen, dass keines der in Beispiel 3.1 gefundenen irreduziblen Polynome mit einem Grad ≥ 2 das Polynom ohne Rest teilt.

Eine Division von $m(x)$ durch $p_2(x) = x^2 + x + 1$ liefert:

$$\begin{array}{r} (x^8 + x^4 + x^3 + x + 1) / (x^2 + x + 1) = x^6 + x^5 + x^3 \\ -(x^8 + x^7 + x^6) \\ \hline x^7 + x^6 + x^4 + x^3 + x + 1 \\ -(x^7 + x^6 + x^5) \\ \hline x^5 + x^4 + x^3 + x + 1 \\ -(x^5 + x^4 + x^3) \\ \hline x + 1 \end{array}$$

Es gilt also:

$$m(x) \bmod (x^2 + x + 1) = x + 1 \neq 0$$

Division von $m(x)$ durch die irreduziblen Polynome vom Grad 3 und 4 liefert entsprechend:

$$\begin{aligned} m(x) \bmod p_{3,1}(x) &= m(x) \bmod (x^3 + x + 1) &&= x^2 \\ m(x) \bmod p_{3,2}(x) &= m(x) \bmod (x^3 + x^2 + 1) &&= x + 1 \\ m(x) \bmod p_{4,1}(x) &= m(x) \bmod (x^4 + x + 1) &&= x^3 + x^2 + 1 \\ m(x) \bmod p_{4,2}(x) &= m(x) \bmod (x^4 + x^3 + 1) &&= x^3 + x^2 \\ m(x) \bmod p_{4,3}(x) &= m(x) \bmod (x^4 + x^3 + x^2 + x + 1) &&= x^3 + x^2 \end{aligned}$$

Ein Polynom vom Grad n

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

ist durch die Folge seiner $n + 1$ Koeffizienten

$$a_n, a_{n-1}, \dots, a_1, a_0$$

eindeutig bestimmt. Für Polynome aus $\mathbb{F}_2[x]$ sind dies Bitfolgen. Beispielsweise entspricht dem Polynom

$$m(x) = x^8 + x^4 + x^3 + x + 1 \in \mathbb{F}_2[x]$$

die Bitfolge:

1 0 0 0 1 1 0 1 1

Die Berechnung des Rests, der sich bei der in Beispiel 3.2 ausgeführten Polynomdivision von $m(x)$ durch $p_2(x) = x^2 + x + 1$ (entspricht 1 1 1) ergibt, lässt sich kompakt mithilfe der Bitfolgen von $m(x)$ und der Bitfolgen der zur Reduktion benötigten Polynome der Form $(x^2 + x + 1) \cdot x^i$ wie folgt darstellen:

$$\begin{array}{rcl}
 x^8 + x^4 + x^3 + x + 1 & : & \mathbf{1\ 0\ 0\ 0\ 1\ 1\ 0\ 1\ 1} \\
 (x^2 + x + 1) \cdot x^6 & : & \mathbf{1\ 1\ 1\ 0\ 0\ 0\ 0\ 0\ 0} \\
 (x^2 + x + 1) \cdot x^5 & : & \mathbf{1\ 1\ 1\ 0\ 0\ 0\ 0\ 0\ 0} \\
 (x^2 + x + 1) \cdot x^3 & : & \mathbf{1\ 1\ 1\ 0\ 0\ 0} \\
 \hline
 x + 1 & : & \mathbf{1\ 1}
 \end{array}$$

Frage 20

Berechnen Sie die Reste, die sich bei einer Polynomdivision von $m(x)$ durch die irreduziblen Polynome vom Grad 3 und 4 ergeben und vergleichen Sie die Ergebnisse mit den in Beispiel 3.2 genannten Resten.

Verwenden Sie zur Berechnung der Reste die oben beschriebene Darstellung der Polynome mithilfe von Bitfolgen.

Beispiel 3.3: $\mathbb{F}_{256}$, definiert bezüglich $m(x) = x^8 + x^4 + x^3 + x + 1$

Die Elemente in $\mathbb{F}_{256}$ sind definitionsgemäß Polynome aus $\mathbb{F}_2[x]$ vom Grad ≤ 7 . Diese entsprechen den Bitfolgen der Länge 8, welche wiederum den zweistelligen Hexadezimalzahlen (d. h. Bytewerten) entsprechen. Beispielsweise gelten die folgenden Entsprechungen:

$$x^7 + x^5 + x^4 + x^2 + 1 \leftrightarrow 10110101 \leftrightarrow 0xB5$$

$$x^5 + x^3 + x \leftrightarrow 00101010 \leftrightarrow 0x2A$$

Mit dieser Identifikation entspricht die Addition von zwei Bytewerten ihrer XOR-Verknüpfung:

$$0xB5 + 0x2A \leftrightarrow x^7 + x^4 + x^3 + x^2 + x + 1 \leftrightarrow 10011111 \leftrightarrow 0x9F$$

Die Multiplikation von zwei Bytewerten ist bei dieser Identifikation mittels der Multiplikation der entsprechenden Polynome und der Reduktion des Produktes bezüglich $m(x)$ zu berechnen. Beispielsweise gilt:

$$0xB5 \cdot 0x2A \leftrightarrow (x^7 + x^5 + x^4 + x^2 + 1)(x^5 + x^3 + x) \bmod m(x)$$

Berechnung des Produkts:

$$\begin{array}{r}
 \mathbf{1\ 0\ 1\ 1\ 0\ 1\ 0\ 1\ 0\ 0\ 0\ 0\ 0} \\
 \mathbf{1\ 0\ 1\ 1\ 0\ 1\ 0\ 1\ 0\ 0\ 0} \\
 \mathbf{1\ 0\ 1\ 1\ 0\ 1\ 0\ 1\ 0} \\
 \hline
 \mathbf{1\ 0\ 0\ 1\ 0\ 0\ 1\ 1\ 0\ 0\ 0\ 1\ 0}
 \end{array}$$

Reduktion bezüglich $m(x) = x^8 + x^4 + x^3 + x + 1$:

$$\begin{array}{r}
 1001001100010 \\
 1000110110000 \\
 1000110110 \\
 100011011 \\
 \hline
 11111111
 \end{array}$$

Somit gilt:

$$0xB5 \cdot 0x2A = 0xFF$$

Frage 21

Berechnen Sie $\text{MixColumns}(\mathbf{S})$ für: $\mathbf{S} = \begin{pmatrix} D4 & E0 & B8 & 1E \\ BF & B4 & 41 & 27 \\ 5D & 52 & 11 & 98 \\ 30 & AE & F1 & E5 \end{pmatrix}$

Frage 22

Begründen Sie, dass die AES-Operation $\text{MixColumns}()$ eine umkehrbare Abbildung auf der Menge der State-Arrays definiert, indem Sie die Umkehrabbildung $\text{InvMixColumns}() := \text{MixColumns}^{-1}()$ angeben.

Zeigen Sie hierzu, dass die Matrix $\mathbf{M}$ durch die $\text{MixColumns}()$ definiert ist, die folgende Inverse besitzt:

$$\mathbf{M}^{-1} = \begin{pmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{pmatrix}$$

Hashfunktionen

4

4.1	Grundlagen	54
4.2	MessageDigest-Instanzen: Hashfunktionen in Java- Laufzeitumgebungen	56
4.3	Anwendungsbeispiele	58
4.4	Brute-Force-Angriffe auf Hashfunktionen	63
4.5	Konstruktionsverfahren von Hashfunktionen	69

4.1 Grundlagen

Das Ziel einer *kryptographischen Hashfunktion* h ist es, einer jeden Nachricht m einen (im Rahmen praktischer Anwendungen) eindeutigen Wert fester Länge $H(m)$ zuzuweisen, mit dem m identifiziert werden kann. Dies ist vergleichbar mit der Möglichkeit einer Identifikation von Menschen anhand ihrer Fingerabdrücke. Englischsprachig wird ein *Hashwert* daher gelegentlich auch als *digital fingerprint* bezeichnet.

Für kryptographische Hashfunktionen sind die englischsprachigen Bezeichnungen *Cryptographic Hash Function* und *Message Digest* gebräuchlich. Der Einfachheit halber wird von nun an meist auf die Nennung des Adjektivs *kryptographisch* verzichtet, da im Folgenden nur solche Hashfunktionen betrachtet werden.

Beginnen wir mit einer mathematischen Präzisierung des oben genannten Ziels, mithilfe einer Hashfunktion beliebige Nachrichten identifizieren zu können:

(kryptographische) Hashfunktion (Message Digest)

Gegeben sei eine Nachrichtenmenge $\mathcal{M}$, beispielsweise:

$$\mathcal{M} = \mathcal{Z}^{\geq 0} := \bigcup_{n \in \mathbb{N}_0} \mathcal{Z}^n = \{(z_1, \dots, z_n) \mid z_i \in \mathcal{Z}, n \in \mathbb{N}_0\} \quad \text{mit } \mathcal{Z} = \mathbb{Z}_2$$

Eine Abbildung

$$H: \mathcal{M} \rightarrow \mathbb{Z}_2^l$$

mit einem festen $l \in \mathbb{N}$ wird *Hashfunktion* genannt, falls folgende Bedingungen gelten:

- (i) H ist eine *Einwegfunktion*, d. h. es gibt zu keinem $y \in \mathbb{Z}_2^l$ einen effizienten Algorithmus zum Auffinden eines Urbilds (d. h. einer Nachricht) m mit $H(m) = y$.
Diese Eigenschaft wird auch als *Urbildresistenz* (*preimage resistance*) bezeichnet.
- (ii) H ist (*stark*) *kollisionsresistent*, d. h. es gibt keinen effizienten Algorithmus, um eine *Kollision* zu finden, wobei eine Kollision durch zwei Nachrichten m_1, m_2 mit $m_1 \neq m_2$ und $H(m_1) = H(m_2)$ definiert ist.

Unter einem *effizienten Algorithmus* wird ein Algorithmus verstanden, der das Auffinden von Urbildern bzw. von Kollisionen auf eine effizientere Art als durch eine Brute-Force-Suche ermöglicht.

Kommentar Üblicherweise werden Hashfunktionen so konstruiert, dass gilt:

- Die Berechnung von Hashwerten ist effizient möglich und die Berechnungszeit ist (bei längeren Nachrichten) im Wesentlichen proportional zur Nachrichtenlänge.
- $\mathcal{M}$ ist eine Teilmenge von $\mathcal{Z}^{\geq 0}$ mit $\mathcal{Z} = \mathbb{Z}_2^8$, wobei die maximale Länge der in $\mathcal{M}$ vorkommenden Bytefolgen durch eine sehr große Zahl $N \in \mathbb{N}$ beschränkt ist:

$$\mathcal{M} = (\mathbb{Z}_2^8)^{\leq N} := \{(z_1, \dots, z_n) \mid z_i \in \mathbb{Z}_2^8, n \in \mathbb{N}_0, n \leq N\}$$

In der Regel ist N so groß, dass alle in einer Anwendung vorkommenden Nachrichten eine kleinere Länge besitzen.

- Die Bitlänge l der Hashwerte ist ein Vielfaches von 8, sodass Hashwerte auch als $\frac{l}{8}$ -Tupel von Bytewerten aufgefasst werden können.



Kommentar Ebenso wenig wie beispielsweise AES ein *beweisbar* sicheres Blockverschlüsselungsverfahren ist, ist auch von keiner der in der Praxis genutzten Hashfunktionen *bewiesener* Maßen bekannt, ob sie auch nur eine der beiden geforderten Bedingungen an eine Hashfunktion tatsächlich erfüllt. Für die in den Anwendungen benutzten Hashfunktionen ist die jeweilige Aussage, dass es keinen effizienteren Algorithmus als eine Brute-Force-Suche *gibt*, daher durch die Aussage zu ersetzen, dass kein effizienterer Algorithmus als eine Brute-Force-Suche *bekannt* ist. In diesem Sinne soll auch in diesem Kapitel der Begriff Hashfunktion benutzt werden, wenn der Einsatz solcher Funktionen in konkreten Anwendungsfällen zur Sprache kommt.

Die Hashfunktionen MD5 und SHA-1, die über viele Jahre die weltweit meistgenutzten Hashfunktionen waren, wurden bereits im Jahr 2004 bzw. 2005 erfolgreich angegriffen, indem Algorithmen angegeben werden konnten mit deren Hilfe Kollisionen wesentlich effizienter als mit einer Brute-Force-Suche gefunden werden können. (Siehe hierzu RFC 4270 (Attacks on Cryptographic Hashes in Internet Protocols).) Im Sinne unserer Definition dürften MD5 und SHA-1 somit eigentlich streng genommen nicht mehr als Hashfunktionen bezeichnet werden. Auf ihre Nutzung sollte jedenfalls verzichtet werden. ◀

Kommentar Die beiden Eigenschaften (i) und (ii) einer Hashfunktion sind tatsächlich unabhängige Eigenschaften, d.h. weder ist eine Einwegfunktion notwendigerweise kollisionsresistent noch folgt aus der Kollisionsresistenz einer Funktion, dass diese eine Einwegfunktion sein muss. Hierzu die folgenden Beispiele:

- Es sei $\mathcal{M} = \mathbb{Z}_2^{\geq 0}$ und $H : \mathcal{M} \rightarrow \{0, 1\}^l$ eine Hashfunktion. Dann wird durch

$$\tilde{H} : \mathcal{M} \rightarrow \{0, 1\}^l, \quad \tilde{H}((m_1, \dots, m_n)) = \begin{cases} H(()) & \text{für } n = 0 \\ H((m_2, \dots, m_n)) & \text{sonst} \end{cases}$$

eine Einwegfunktion definiert, die aber nicht mehr kollisionsresistent ist.

- Es sei wiederum $\mathcal{M} = \mathbb{Z}_2^{\geq 0}$ und $H : \mathcal{M} \rightarrow \{0, 1\}^l$ eine Hashfunktion. Durch

$$\hat{H} : \mathcal{M} \rightarrow \{0, 1\}^{l+1}$$

$$\hat{H}((m_1, \dots, m_n)) = \begin{cases} (0, m_1, \dots, m_l) & \text{für } n = l \\ (1, h_1, \dots, h_l) & \text{mit } (h_1, \dots, h_l) = H((m_1, \dots, m_n)) \text{ für } n \neq l \end{cases}$$

wird dann eine Funktion definiert, die nach wie vor kollisionsresistent ist.

$\hat{H}$ ist aber keine Einwegfunktion mehr, da für die Hälfte aller möglichen Bilder der Funktion $\hat{H}$ (nämlich alle Bitfolgen der Länge $l + 1$, die mit 0 beginnen) sofort ein Urbild angegeben werden kann. (Der entsprechende Algorithmus ist trivialerweise effizient.) ◀

Bei der Anwendung einer Hashfunktion wird oft anstelle der (starken) Kollisionsresistenz eine verwandte Eigenschaft, die als schwache Kollisionsresistenz bezeichnet wird, benötigt:

Schwache Kollisionsresistenz (Second preimage resistance)

Eine Abbildung

$$H : \mathcal{M} \rightarrow \mathbb{Z}_2^l$$

heißt *schwach-kollisionsresistent*, falls es keinen effizienten Algorithmus gibt um zu einem beliebigen $m \in \mathcal{M}$ ein $\tilde{m} \in \mathcal{M}$ mit

$$\tilde{m} \neq m \quad \text{und} \quad H(\tilde{m}) = H(m)$$

zu finden.

Kommentar Häufig wird behauptet, dass aus der Kollisionsresistenz einer Funktion ihre schwache Kollisionsresistenz folgt. (Dies wird ja auch durch die Bezeichnung der Eigenschaft nahe gelegt.) Begründung hierfür: Ist die Funktion H nicht schwach kollisionsresistent, so lässt sich zu einem $m \in \mathcal{M}$ ein $\tilde{m} \neq m$ mit $H(\tilde{m}) = H(m)$ finden. Das Paar $(m, \tilde{m})$ ist dann eine Kollision und H ist deshalb nicht kollisionsresistent.

Die obige Begründung ist allerdings bei genauerer Betrachtung ungenau. Die Annahme, dass sich bei einer schwach kollisionsresistenten Hashfunktion zu einem $m \in \mathcal{M}$ ein $\tilde{m} \neq m$ mit $H(\tilde{m}) = H(m)$ finden lässt, ist zu vage, da sie keine konkrete Annahme über den benötigten Aufwand zum Auffinden von $\tilde{m}$ macht. Kann ein solches $\tilde{m}$ nämlich effizienter als bei einer Brute-Force-Suche durch die Menge aller Elemente aus $\mathcal{M} \setminus \{m\}$ gefunden werden, so könnte der Aufwand dennoch immer noch wesentlich größer als für eine Brute-Force-Suche einer beliebigen Kollision sein! (Der Aufwand der Brute-Force-Suchen wird in Abschnitt 4.4 bestimmt.) ◀

Richtig und für die Praxis entscheidend ist:

Ein Angriff auf die schwache Kollisionsresistenz ist mindestens so aufwendig, wie ein Angriff auf die Kollisionsresistenz.

Ist die Bestimmung einer Kollision praktisch nicht möglich, so kann auch die schwache Kollisionsresistenz in der Praxis nicht erfolgreich angegriffen werden.

4.2 MessageDigest-Instanzen: Hashfunktionen in Java-Laufzeitumgebungen

In den aktuellen Java-Laufzeitumgebungen können eine Vielzahl von Hashfunktionen als Instanzen der Klasse `MessageDigest` genutzt werden. Die Instanziierung und Nutzung einer solchen Implementierung folgt dem gleichen Framework, wie die in Abschnitt 4.2 beschriebene Verwendung von Instanzen der Klasse `Cipher` als Verschlüsselungsalgorithmus.

- Der Aufruf einer statischen Methode `getInstance(...)` der Klasse `MessageDigest` liefert eine `MessageDigest`-Instanz, die die Implementierung einer Hashfunktion beinhaltet. Der spezifische Algorithmus wird wiederum durch den Wert eines `String`-Parameters bestimmt, der der `getInstance(...)`-Methode übergeben werden muss. Steht eine Implementierung des gewünschten Algorithmus nicht zur Verfügung oder kann der übergebene Parameter dem Algorithmus nicht zugeordnet werden, so wirft `getInstance(...)` eine `NoSuchAlgorithmException`.
- Ohne weitere Initialisierung kann eine `MessageDigest`-Instanz zur Berechnung von Hashwerten genutzt werden. Hierfür stehen verschiedene `update(...)`- und `digest(...)`-Methoden zur Verfügung. (Die `update(...)`-Methoden werden benutzt, falls eine Nachricht nicht in einem Stück, sondern nach und nach stückweise verarbeitet werden soll.)

Aufgabe 7

Laden Sie eine Kopie des RFC 4270 in der txt-Version von <https://tools.ietf.org/rfc/rfc4270.txt> auf ihren Rechner und berechnen Sie den SHA-256-Hashwert dieser Datei (s. Abb. 4.1).

```

1  InputStream is = new FileInputStream("rfc4270.txt");
   byte[] m = is.readAllBytes();

   MessageDigest md = MessageDigest.getInstance("SHA-256");
5  byte[] hashValue = md.digest(m);
   System.out.println("Hashwert:");
   System.out.println(Dump.dump(hashValue));

```

Abb. 4.1 Quellcode zur Berechnung eines Hashwerts mit der Hashfunktion SHA-256

Beispiel 4.1: Kollision für MD5

Im Jahr 2004 wurden von Xiaoyun Wang, Dengguo Feng, Xuejia Lai und Hongbo Yu in der Arbeit *Collisions for Hash Functions MD4, MD5, HAVAL-128 and RIPEMD* (<http://eprint.iacr.org/2004/199>) erstmals Kollisionen für die Hashfunktion MD5 publiziert. Eine solche Kollision ist in Abbildung 4.2 wiedergegeben. ◀

```

1  byte[] data1 = Dump.hexString2byteArray("
   + "D131DD02C5E6EEC4693D9A0698AFF95C2FCAB58712467EAB"
   + "4004583EB8FB7F8955AD340609F4B30283E488832571415A"
   + "085125E8F7CDC99FD91DBDF280373C5BD8823E3156348F5B"
5  + "AE6DACD436C919C6DD53E2B487DA03FD02396306D248CDA0"
   + "E99F33420F577EE8CE54B67080A80D1EC69821BCB6A88393"
   + "96F9652B6FF72A70");

10 byte[] data2 = Dump.hexString2byteArray("
   + "D131DD02C5E6EEC4693D9A0698AFF95C2FCAB50712467EAB"
   + "4004583EB8FB7F8955AD340609F4B30283E4888325F1415A"
   + "085125E8F7CDC99FD91DBD7280373C5BD8823E3156348F5B"
   + "AE6DACD436C919C6DD53E23487DA03FD02396306D248CDA0"
15 + "E99F33420F577EE8CE54B67080280D1EC69821BCB6A88393"
   + "96F965AB6FF72A70");

```

Abb. 4.2 Kollision für die Hashfunktion MD5

Frage 23

Berechnen Sie die MD5-Hashwerte von data1 und data2 aus Abbildung 4.2 und überzeugen Sie sich davon, dass sie gleich sind.

An wie vielen Bitstellen unterscheiden sich data1 und data2?

Beispiel 4.2: Kollision für SHA-1

Seit dem Jahr 2005 sind diverse Arbeiten publiziert worden, die zeigen, dass ein Auffinden von Kollisionen für die Hashfunktion SHA-1 deutlich effizienter als mit einem Brute-Force-Angriff möglich ist. Dennoch dauerte es ungefähr weitere 12 Jahre bis im Jahre 2017 eine SHA-1-Kollision erstmals öffentlich publiziert wurde.

Unter <https://shattered.io> können Sie Links sowohl zu dieser Publikation, die in der Einführung eine lesenswerte historische Zusammenfassung der zuvor bereits publizierten

Angriffe auf die Hashfunktionen der MD-SHA-Familie und deren praxisrelevanten Implikationen enthält, als auch zu zwei verschiedenen PDF-Dateien shattered-1.pdf und shattered-2.pdf finden. ◀

Frage 24

Betrachten Sie die in Beispiel 4.2 genannten Dateien shattered-1.pdf und shattered-2.pdf und überzeugen Sie sich so von deren Verschiedenheit.

Berechnen Sie die SHA-1-Hashwerte von shattered-1.pdf und shattered-2.pdf und überzeugen Sie sich von deren Gleichheit.

An wie vielen Bitstellen unterscheiden sich shattered-1.pdf und shattered-2.pdf? In welchem Bereich der Dateien (Bytestellen) befinden sich die Unterschiede?

Tipp: Zur Beantwortung der Fragen können Sie die Methode `noOfDiffBits()` aus Abbildung 4.3 direkt bzw. mit einer kleinen Ergänzung benutzen.

```

1  public static int noOfDiffBits(byte[] ba1, byte[] ba2) {
    if( ba1 == null || ba2 == null || ba1.length != ba2.length ) {
        throw new IllegalArgumentException(
            "Length of input byte arrays must be equal!");
    }
5   int ctr = 0;
    for( int i = 0; i < ba1.length; ++i ) {
        int bc = Integer.bitCount((ba1[i] ^ ba2[i]) & 0xff);
        ctr += bc;
10  }
    return ctr;
    }

```

Abb. 4.3 Methode zur Abzählung unterschiedlicher Bitstellen von zwei gleichlangen byte-Arrays

4.3 Anwendungsbeispiele

Anwendungsbeispiel: Passwortdateien

Zur Authentifizierung von Nutzern an einem IT-System werden häufig Passwörter verwendet. Ein Nutzer (User) u_i , der sich an dem System anmeldet, muss dann neben dem Nutzernamen sein geheim zu haltendes Passwort p_i eingeben, um sich zu authentifizieren. Von dem IT-System muss entschieden werden, ob das eingegebene Passwort den Nutzer korrekt authentifiziert. Hierzu könnte einfach eine Tabelle aller Nutzer und der zugehörigen Passwörter, wie sie beispielhaft in Abbildung 4.4 dargestellt ist, in einer Datei (oder Datenbank) gespeichert werden, sodass ein von einem Nutzer eingegebenes Passwort mit dem gespeicherten verglichen werden kann.

u_i	p_i
Alice	123456
Bob	rj5gt7hog4m9gxtinokx
Charly	admin
$\vdots$	$\vdots$

Abb. 4.4 Passwortdatei

Gelangt diese Datei allerdings in den Besitz eines Angreifers, so kann dieser sich im Namen eines jeden Nutzers am System authentifizieren und dann im Namen des entsprechenden Nutzers aktiv werden. (Je nach System, könnte der Angreifer im Namen der angegriffenen Nutzer kompromittierende Emails verschicken, Einkäufe tätigen, Bankauszüge einsehen, usw.)

Um eine solche weitreichende Angriffsmöglichkeit auszuschließen, sollten die Passwörter auf dem IT-System nicht direkt im Klartext gespeichert werden. Vielmehr bietet es sich an, nach Wahl einer Hashfunktion H anstelle eines Passwortes p_i den Hashwert $H(p_i)$ zusammen mit dem Nutzernamen u_i zu speichern. Beispielsweise ist in Abbildung 4.5 eine entsprechende Tabelle dargestellt, die die SHA-256-Hashwerte der Passwörter aus Abbildung 4.4 enthält. (Aus Platzgründen sind in Abbildung 4.5 jeweils nur die ersten und letzten drei Bytes der Hashwerte wiedergegeben.) Zur Authentifizierung eines Nutzers wird dann zunächst der Hashwert des vom Nutzer übermittelten Passwortes berechnet und anschließend mit dem gespeicherten Hashwert verglichen.

u_i	$H(p_i)$
Alice	8D969E...DC6C92
Bob	714A52...D2D382
Charly	8C6976...48A918
$\vdots$	$\vdots$

Abb. 4.5 Passwortdatei mit den SHA-256-Hashwerten der Passwörter aus Abbildung 4.4

Gelangt nun ein Angreifer in den Besitz der Datei mit den Hashwerten der Passwörter, so kann dieser sich nur dann im Namen eines Nutzers u_i anmelden, falls er für den entsprechenden Hashwert ein passendes Passwort finden kann. (Dies müsste nicht zwingend das Passwort p_i des Nutzers u_i sein. Es muss „nur“ irgendein Wort $\tilde{p}_i$ mit $H(\tilde{p}_i) = H(p_i)$ sein.)

Aufgrund der *Urbildresistenz* einer Hashfunktion ist das Auffinden eines Passwortes nicht leichter als per Brute-Force-Suche möglich. Ist l die Bitlänge der Hashfunktion und wird das Passwort zufällig aus einer Menge von Wörtern gewählt, die mehr als 2^l Elemente enthält, so wird die Brute-Force-Suche durchschnittlich 2^{l-1} Versuche benötigen, was z. B. für $l = 128$ in keiner praxisrelevanten Zeitspanne möglich ist.

Häufig verwenden Nutzer jedoch Passwörter, die Elemente einer relativ kleinen Menge von schwachen Passwörtern sind. Eine erfolgreiche Brute-Force-Suche braucht sich dann natürlich auch nur über die Menge der schwachen Passwörter zu erstrecken!

Beispielsweise umfasst die Menge der Passwörter, die aus 6 zufällig gewählten Kleinbuchstaben besteht, nur

$$26^6 \approx 3 \cdot 10^8 \approx 2^{28}$$

Elemente, für die alle Hashwerte zum Vergleich berechnet werden können.

Besteht das Ziel eines Angreifers, der in den Besitz der Datei mit den Hashwerten der Passwörter gelangt ist, darin, Passwörter von (möglichst vielen) Nutzern zu bestimmen (ohne jedoch einen spezifischen Nutzer u_i angreifen zu wollen), so bietet es sich an, für eine Menge schwacher Passwörter alle Hashwerte im Voraus zu berechnen und in einer Liste (sortiert nach den Hashwerten) zu speichern. Die Nutzung solcher Listen führt zu *Wörterbuchangriffen* (*dictionary attacks*), bei denen alle schwachen Passwörter durch ein simples „Nachschlagen“ der Hashwerte aus der Passwortdatei in der Liste (dem Wörterbuch) sofort ohne weitere Rechnung gefunden werden können. ◀

Frage 25

Berechnen Sie die SHA-256 Hashwerte der Passwörter aus Abbildung 4.4 und vergleichen Sie diese mit den in Abbildung 4.5 angegebenen.

Hinweis: Hashwerte sind vom ASCII-Code der Passwörter zu berechnen. Den ASCII-Code eines Strings liefert die Methode `getBytes()` der Klasse `String`.

Frage 26

Der Nutzer David hat ein Passwort gewählt, welches aus sechs Kleinbuchstaben (ohne Umlaute) besteht. Ein Angreifer konnte den SHA-256-Hashwert des Passworts lesen. Dieser ist:

ADDB0F5E7826C857D7376D1BD9BC33C0C544790A2EAC96144A8AF22B1298C940

Wie lautet das Passwort?

Frage 27

Bestimmen Sie die maximale Zeitdauer $T(26,6)$, für einen Brute-Force-Angriff zur Bestimmung eines Passworts, welches aus sechs Kleinbuchstaben (ohne Umlaute) besteht und von dem der SHA-256-Hashwert des Passworts bekannt ist.

Angenommen $T(26,6) = 70$ s. Mit welchen maximalen Zeitdauern ist dann für einen erfolgreichen Brute-Force-Angriff in den folgenden Fällen zu rechnen:

- $T(26,8)$: Das Passwort besteht aus acht Kleinbuchstaben (ohne Umlaute).
- $T(102,6)$: Das Passwort ist sechs Zeichen lang. Die Zeichen sind druckbare ASCII-Zeichen oder Zeichen, die den deutschen Umlauten entsprechen.
- $T(102,8)$: Das Passwort ist acht Zeichen lang. Jedes Zeichen ist wie in $T(102,6)$ ein Element einer 102 verschiedenen Zeichen umfassenden Menge.

Frage 28

Ein Angreifer gelangt in den Besitz einer Datei mit den Hashwerten von Passwörtern und stellt fest, dass zwei Hashwerte, die zwei verschiedenen Nutzern u_i und u_j zugeordnet sind, identisch sind. Welche Schlussfolgerung kann der Angreifer aus dieser Beobachtung ziehen?

Um die global wirksamen Wörterbuchangriffe auf die Hashwerte einer Passwortdatei abzuwehren, können sogenannte *Salt*-Werte genutzt werden und um die Nutzer-spezifischen Angriffe zu erschweren, kann der Aufwand (Ausführungszeit) zur Berechnung eines finalen Hashwerts durch eine vielfache iterative Anwendung der Hashfunktion beliebig erhöht werden:

Anwendungsbeispiel: Passwortdateien mit Salt und Iteration Count

Zur Abwehr möglicher Wörterbuchangriffe wird jedem Nutzer u_i ein zufällig gewählter sogenannter *Salt*-Wert zugeordnet. Dieser wird zusammen mit dem Nutzernamen in der Passwortdatei gespeichert. Das Passwort p_i des Nutzers u_i wird nun vor der Hashberechnung um den Salt-Wert s_i ergänzt, d. h. anstelle von $H(p_i)$ wird $H(p_i||s_i)$ berechnet und dem Eintrag für den Nutzer u_i in der Passwortdatei hinzugefügt.

Kommt ein Angreifer in den Besitz einer solchen Passwortdatei, kann er nur noch nutzerspezifische Brute-Force-Angriffe durchführen. Bei einer Vorausberechnung würde für jeden Salt-Wert ein spezifisches „Wörterbuch“ benötigt. Die Anzahl der hierzu notwendigen Berechnungen und der Speicherbedarf für die Ergebnisse (Wörterbücher) wird keinem Angreifer zur Verfügung stehen, falls die Salt-Werte aus einer hinreichend großen Menge zufällig gewählt werden.

Schließlich kann der Aufwand für nutzerspezifische Brute-Force-Angriffe noch dadurch deutlich erhöht werden, indem nicht nur eine sondern eine größere Anzahl c (z. B. $c = 10^6$) von Hashberechnungen zur Berechnung der Einträge in der Passwortdatei erforderlich sind. Hierzu wird für den Nutzer u_i anstelle von $H(p_i||s_i)$ beispielsweise der Wert

$$H^c(p_i||s_i) = \underbrace{H(H(\dots H(p_i||s_i)\dots))}_c \quad (*)$$

in der Passwortdatei gespeichert. Die Zahl c wird als *Iteration Count* bezeichnet.

Beispielsweise ist in Abbildung 4.6 der Inhalt einer Passwortdatei dargestellt, die sich bei der Anwendung von (*) mit $H = \text{SHA-256}$ und einem gewissen c (s. Frage 29) auf die Passwörter aus Abbildung 4.4 ergibt.

u_i	s_i	$H^c(p_i s_i)$
Alice	22445CBCBDFD10E00EA6	DADCC5...786532
Bob	6E25B70D10ABAC317C54	6FDAD4...87FD6E
Charly	83CCA240B57595D58192	2C2FA8...68B999
$\vdots$	$\vdots$	$\vdots$

Abb. 4.6 Passwortdatei mit Salt-Werten s_i und iterativ berechneten Hashwerten

Durch die Nutzung einer solchen iterativen Berechnung des Eintrags in der Passwortdatei erhöht sich natürlich auch der Aufwand bei der regulären Überprüfung eines Passwortes um den Faktor c . Für eine *einzig*e Überprüfung stellt dies bei einer geeigneten Wahl von c kein Problem dar. Einem Angreifer hingegen, der ja eine (hoffentlich) große Menge an potentiellen Passwörtern überprüfen muss, wird die Suche nach dem richtigen Passwort durch eine Laufzeitverlängerung um einen Faktor von beispielsweise $c = 10^6$ signifikant erschwert. ◀

Frage 29

Welcher Wert wurde für den Iteration Count c zur Berechnung der Passwortdatei aus Abbildung 4.6 festgelegt?

(Beachten Sie, dass die Salt-Werte in Abbildung 4.6 als Hex-Strings dargestellt sind und somit byte-Arrays der Länge 10 entsprechen.)

Kommentar Das oben beschriebene konkrete Verfahren zur Nutzung von Salt-Werten und Iteration Counts zur Berechnung der Einträge in einer Passwortdatei entspricht der in dem Standard *PKCS #5: Password-Based Cryptography Specification - Version 2.1* (RFC 8018) definierten Funktion *pbkdf1* (*password-based key derivation function 1*). Aufgrund ihrer Einfachheit lassen sich anhand dieser Funktion der Sinn von Salt-Werten und Iteration Count gut nachvollziehen.

Der Standard *PKCS #5* empfiehlt allerdings anstelle einer Verwendung von *pbkdf1* eine Nutzung der ebenfalls in dem Standard definierten Funktion *pbkdf2*. ◀

Anwendungsbeispiel: Integritätsschutz von Dateien

Soll in einem Rechnersystem sichergestellt werden, dass nur solche Dateien zur Ausführung kommen, denen aufgrund definierter Kriterien vertraut wird, so lässt sich dies durch eine Erweiterung des Betriebssystems folgendermaßen erreichen:

Zunächst werden mit einer Hashfunktion die Hashwerte der zulässigen ausführbaren Dateien berechnet und in einer Tabelle (*Whitelist*) in einem sicheren Speicherbereich, d. h. einem Speicherbereich auf den ein potentieller Angriff keinen Zugriff hat, abgelegt. Soll eine Datei zur Ausführung gebracht werden, so wird nun vom Betriebssystem der Hashwert der Datei berechnet und mit dem sicher abgespeicherten verglichen. Sind die Werte verschieden, so wurde die Datei offensichtlich verändert und ihre Ausführung wird deshalb unterbunden. (Dateien, für die kein Eintrag in der *Whitelist* enthalten ist, werden natürlich auch nicht ausgeführt.)

Versucht ein Angreifer eine in der *Whitelist* aufgeführte Datei mit Schadcode abzuändern, so sieht er sich vor die Herausforderung gestellt, dass die modifizierte Datei den gleichen Hashwert besitzen muss wie die ursprüngliche Datei. Ist die Hashfunktion schwach kollisionsresistent, so sollte ihm dies nicht gelingen!

Eine etwas andere Situation liegt vor, falls die Entwicklerin einer zulässigen ausführbaren Datei böse Absichten hegt. Sie kann nämlich im Voraus versuchen zwei Dateien mit unterschiedlicher Funktionalität (z. B. ohne und mit einer *Backdoor*) zu erstellen, die den gleichen Hashwert besitzen. Ist die Hashfunktion kollisionsresistent, so sollte ihr dies aber nicht gelingen. Die schwache Kollisionsresistenz begründet in diesem Fall jedoch noch keinen Schutz vor einem solchen Angriff. ▶

Anwendungsbeispiel: Integritätsschutz bei einem Dateidownload

Häufig wird auf Webseiten neben einem Link zum Download einer Datei auch ein Hashwert der Datei angegeben. Nach einem Download der Datei kann der Hashwert berechnet und mit dem auf der Webseite angegebenen verglichen werden.

Diese Überprüfung bietet aber so gut wie keinen Schutz gegen einen Angreifer, der eine solche Webseite erfolgreich angreift und die Datei durch eine Datei mit Schadsoftware ersetzt. Der Angreifer wird in der Regel auch ohne Probleme die Angabe des Hashwerts auf der angegriffenen Webseite geeignet anpassen können.

Ein gewisser Schutz vor dem Download von Schadsoftware ist möglich, wenn die entsprechende Datei auch auf anderen Webseiten zum Download zur Verfügung steht. Nach dem Download der Datei und der Berechnung ihres Hashwerts kann dieser dann nämlich mit der Angabe von einer anderen Webseite verglichen werden. ▶

4.4 Brute-Force-Angriffe auf Hashfunktionen

4.4.1 Brute-Force-Urbildsuche

Bei einem Angriff auf die Urbildresistenz oder auch auf die schwache Kollisionsresistenz einer Hashfunktion

$$H : \mathcal{M} \rightarrow \mathbb{Z}_2^l$$

ist zu einem vorgegebenen Hashwert $y \in \mathbb{Z}_2^l$ (wobei im Fall der schwachen Kollisionsresistenz $y = H(\tilde{m})$ für ein $\tilde{m} \in \mathcal{M}$ ist) ein $m \in \mathcal{M}$ (ggf. mit $m \neq \tilde{m}$) zu finden mit:

$$H(m) = y$$

Bei einem Brute-Force-Angriff werden für die Elemente aus $\mathcal{M}$ (bzw. $\mathcal{M} \setminus \{\tilde{m}\}$) der Reihe nach die Hashwerte berechnet bis sich schließlich der vorgegebene Wert y ergibt. Unter der Annahme, dass sich H von einer Zufallsfunktion nicht unterscheiden lässt, ist hierbei mit durchschnittlich 2^l Ausführungen der Hashfunktion zu rechnen (Erwartungswert der geometrischen Wahrscheinlichkeitsverteilung mit $p = 2^{-l}$).

Es ist auch möglich, die Anzahl von Ausführungen der Hashfunktion abzuschätzen, um mit einer vorgegebenen Wahrscheinlichkeit p (z. B. $p = 50\%$, $p = 90\%$ oder $p = 99\%$) ein gesuchtes Urbild zu finden. Hierzu werden die Hashwerte, die sich bei der Berechnung beliebiger Elemente $m \in \mathcal{M}$ ergeben, als unabhängige Ereignisse aufgefasst, welche bei einem Zufallsexperiment mit insgesamt $n = 2^l$ gleichwahrscheinlichen Ereignissen auftreten können. Die Abschätzungen ergeben sich dann mit dem folgenden Satz:

Satz 4.1

Es sei p die Wahrscheinlichkeit, bei der Beobachtung von r unabhängigen Ereignissen aus einer Menge von insgesamt n möglichen gleichwahrscheinlichen Ereignissen, ein vorgegebenes Ereignis mindestens einmal zu beobachten.

(Urnenmodell: Wahrscheinlichkeit, dass beim Ziehen mit Zurücklegen aus einer Urne mit insgesamt n Kugeln spätestens nach r Versuchen eine bestimmte Kugel gezogen wird.)

Dann gilt:

(i)

$$p = 1 - \left(1 - \frac{1}{n}\right)^r$$

(ii) Ist $n \geq 2$ und $p_0 \in (0, 1)$, so ist $p \geq p_0$ genau dann falls gilt:

$$r \geq \frac{\ln(1 - p_0)}{\ln(1 - \frac{1}{n})}$$

(iii) Ist $n \geq 2$ und $p_0 \in (0, 1)$, so ist $p \geq p_0$ falls gilt:

$$r \geq \lceil \ln(1 - p_0) \rceil \cdot n$$

(iv) Ist $n \geq 2$, so ist $p \geq \frac{1}{2}$ falls gilt:

$$r \geq \ln(2) \cdot n \approx 0,69315 \cdot n$$

Beweis

- (i): Das Gegenereignis, nämlich r maliges Ziehen ohne Erfolg, tritt mit der Wahrscheinlichkeit

$$1 - p = \left(1 - \frac{1}{n}\right)^r$$

auf. Hieraus folgt (i).

- (ii): Ist $p \geq p_0$ so folgt mit (i):

$$p_0 \leq p = 1 - \left(1 - \frac{1}{n}\right)^r$$

$$\Leftrightarrow \left(1 - \frac{1}{n}\right)^r \leq 1 - p_0$$

$$\Leftrightarrow r \ln\left(1 - \frac{1}{n}\right) \leq \ln(1 - p_0)$$

$$\Leftrightarrow r \geq \frac{\ln(1 - p_0)}{\ln\left(1 - \frac{1}{n}\right)}$$

- (iii): Allgemein gilt für alle $x \in \mathbb{R}^+$:

$$\ln(x) \leq x - 1$$

Insbesondere gilt für $n \geq 2$

$$\ln\left(1 - \frac{1}{n}\right) \leq -\frac{1}{n} \Rightarrow \left|\ln\left(1 - \frac{1}{n}\right)\right| \geq \frac{1}{n} \Rightarrow \frac{1}{\left|\ln\left(1 - \frac{1}{n}\right)\right|} \leq n$$

und deshalb:

$$\frac{\ln(1 - p_0)}{\ln\left(1 - \frac{1}{n}\right)} = \frac{|\ln(1 - p_0)|}{\left|\ln\left(1 - \frac{1}{n}\right)\right|} \leq |\ln(1 - p_0)| \cdot n$$

Für $r \geq |\ln(1 - p_0)| \cdot n$ gilt daher auch

$$r \geq \frac{\ln(1 - p_0)}{\ln\left(1 - \frac{1}{n}\right)}$$

und mit (ii) folgt dann $p \geq p_0$.

- (iv): Spezialfall von (iii) für $p_0 = \frac{1}{2}$.

■

Beispiel 4.7

Wieviele Menschen müssen nach ihrem Geburtstag gefragt werden, damit mit einer Wahrscheinlichkeit $p \geq \frac{1}{2}$ mindestens einer der Befragten an einem 14. Mai geboren wurde?

Gehen wir der Einfachheit halber von 365 Tagen pro Jahr aus, und nehmen wir zudem an, dass an jedem Tag des Jahres gleich viele Menschen ihren Geburtstag haben, so ergibt sich mit Satz 4.1 (ii), dass genau dann $p \geq \frac{1}{2}$ gilt, falls

$$r \geq \frac{\ln(1/2)}{\ln(364/365)} \approx 252,7$$

viele Menschen nach ihrem Geburtstag gefragt werden.

Werden 253 Menschen befragt so ist die Wahrscheinlichkeit also $> \frac{1}{2}$, dass einer der Befragten am 14. Mai Geburtstag hat. Die Abschätzung in Satz 4.1 (iv) ist übrigens für diesen Wert von n bereits perfekt, da sie den gleichen ganzzahligen Wert für r liefert:

$$r \geq \ln(2) \cdot 365 \approx 252,999$$

Frage 30

Gegeben sei eine Hashfunktion

$$H : \mathcal{M} \rightarrow \mathbb{Z}_2^l$$

und ein $y \in \mathbb{Z}_2^l$.

Wie oft muss die Hashfunktion ungefähr ausgeführt werden, um mit einer Wahrscheinlichkeit $p = 0,5$, $p = 0,9$ oder $p = 0,99$ ein Urbild $m \in \mathcal{M}$ für y zu finden?

4.4.2 Brute-Force-Kollisionssuche

Bei einem Angriff auf die Kollisionsresistenz einer Hashfunktion

$$H : \mathcal{M} \rightarrow \mathbb{Z}_2^l$$

sind zwei verschiedene Elemente $m_1, m_2 \in \mathcal{M}$ zu finden mit:

$$H(m_1) = H(m_2)$$

Hierzu können für die Elemente aus $\mathcal{M}$ der Reihe nach die Hashwerte berechnet und in einer (nach den Hashwerten sortierten) Tabelle abgespeichert werden, bis ein Hashwert, der bereits in der Tabelle steht, ein zweites Mal auftritt.

Im Gegensatz zur Brute-Force-Urbildsuche ergibt sich aus dem oben beschriebenen Angriff eine nicht zu vernachlässigende Anforderung an den Speicherbedarf zum Aufbau der Tabelle mit den bereits berechneten Hashwerten. Andererseits ist aber bei der obigen Brute-Force-Kollisionssuche mit einer wesentlich geringeren Anzahl von Ausführungen der Hashfunktion zu rechnen, nämlich nur von der Größenordnung $\sqrt{2^l} = 2^{l/2}$. Diese bemerkenswerte Tatsache wird häufig als *Geburtstagsparadoxon* bezeichnet (siehe hierzu Beispiel 4.8).

Wiederum ist es möglich, die Anzahl von Ausführungen der Hashfunktion abzuschätzen, um mit einer vorgegebenen Wahrscheinlichkeit p (z. B. $p = 50\%$, $p = 90\%$ oder $p = 99\%$) eine Kollision zu finden. Werden die Ergebnisse von Hashwertberechnungen wieder als gleichwahrscheinliche Elementarereignisse eines Zufallsexperiments interpretiert, so ergeben sich die Abschätzungen mit folgenden Sätzen:

Satz 4.2

Es sei $p = p(n, r)$ die Wahrscheinlichkeit bei der Beobachtung von r unabhängigen Ereignissen aus einer Menge von insgesamt n möglichen gleichwahrscheinlichen Ereignissen, lauter verschiedene Ereignisse zu beobachten.

(Urnenmodell: Wahrscheinlichkeit, dass beim r -maligen Ziehen mit anschließendem Zurücklegen einer Kugel aus einer Urne mit insgesamt n Kugeln lauter verschiedene Kugeln gezogen werden.)

Dann gilt:

$$p(n, r) = \frac{n \cdot (n-1) \cdots (n-r+1)}{n^r} = \left(1 - \frac{1}{n}\right) \cdots \left(1 - \frac{r-1}{n}\right) \leq e^{-\frac{r(r-1)}{2n}}$$

Beweis Bei dem ersten Versuch wird natürlich ein zuvor noch nicht beobachtetes Ereignis auftreten. Bei dem zweiten Versuch ist die Wahrscheinlichkeit für das Auftreten eines anderen Ereignisses durch

$$\frac{n-1}{n} = \left(1 - \frac{1}{n}\right)$$

gegeben. Treten fortlaufend lauter verschiedene Ereignisse auf, so ist die Wahrscheinlichkeit, dass auch noch beim r -ten Versuch ein neues Ereignis auftritt durch

$$\frac{n-r+1}{n} = \left(1 - \frac{r-1}{n}\right)$$

gegeben. Aus diesem Argument folgt die behauptete Gleichheit:

$$p(n, r) = \left(1 - \frac{1}{n}\right) \cdots \left(1 - \frac{r-1}{n}\right)$$

Aus der für alle $x \in \mathbb{R}$ gültigen Ungleichung

$$1 - x \leq e^{-x}$$

folgt die behauptete Ungleichung:

$$p(n, r) = \left(1 - \frac{1}{n}\right) \cdots \left(1 - \frac{r-1}{n}\right) \leq e^{-\frac{1}{n}} \cdots e^{-\frac{r-1}{n}} = e^{-\frac{r(r-1)}{2n}}$$

■

Das Komplementärereignis zu dem im letzten Satz beschriebene Ereignis ist das Auftreten einer Kollision, d. h. das mindestens einmalige wiederholte Auftreten eines bereits zuvor beobachteten Elementarereignisses. Die Wahrscheinlichkeit für das Auftreten einer Kollision bei insgesamt r Versuchen, die *Kollisionswahrscheinlichkeit*, ist daher:

$$c(n, r) = 1 - p(n, r)$$

Satz 4.3

Für die Kollisionswahrscheinlichkeit $p = c(n, r)$ gilt:

(i)

$$1 - e^{-\frac{r(r-1)}{2n}} \leq c(n, r) \leq \frac{r(r-1)}{2n}$$

(ii) Ist $r \leq \sqrt{2n}$, so gilt:

$$\left(1 - \frac{1}{e}\right) \cdot \frac{r(r-1)}{2n} \leq c(n, r) \leq \frac{r(r-1)}{2n}$$

- (iii) Ist $p_0 \in (0, 1)$, so ist $c(n, r) \geq p_0$ falls gilt:

$$r \geq \sqrt{-2 \cdot \ln(1 - p_0)} \cdot \sqrt{n} + 1$$

- (iv) Ist $p_0 \in (0, 1)$, so ist $c(n, r) \geq p_0$ falls gilt:

$$r \geq \sqrt{-2 \cdot \ln(1 - p_0) \cdot n + \frac{1}{4}} + \frac{1}{2}$$

- (v) Es ist $c(n, r) \geq \frac{1}{2}$ falls

$$r \geq \sqrt{\ln(4)} \cdot \sqrt{n} + 1 \approx 1,18 \cdot \sqrt{n} + 1$$

oder die etwas schärfere Ungleichung

$$r \geq \sqrt{\ln(4) \cdot n + \frac{1}{4}} + \frac{1}{2}$$

gilt.

Beweis

- (i): Wegen $p(n, r) \leq e^{-\frac{r(r-1)}{2n}}$ (s. Satz 4.2) gilt:

$$1 - e^{-\frac{r(r-1)}{2n}} \leq 1 - p(n, r) = c(n, r)$$

Die zweite Ungleichung ergibt sich aus der Überlegung, dass die Wahrscheinlichkeit für das Auftreten einer Kollision beim i -ten Versuch höchstens $\frac{i-1}{n}$ betragen kann. Bezeichnet E_i das Ereignis, dass beim i -ten Versuch eine Kollision auftritt, so gilt für die Wahrscheinlichkeit, dass spätestens nach r Versuchen eine Kollision auftritt:

$$c(n, r) = P\left(\bigcup_{i=1}^r E_i\right) \leq \sum_{i=1}^r P(E_i) \leq \frac{0}{n} + \frac{1}{n} + \dots + \frac{r-1}{n} = \frac{r(r-1)}{2n}$$

- (ii): Die obere Schranke ist wie in (i). Zum Beweis der unteren Schranke vergleichen wir die Funktionen $f(x) = 1 - e^{-x}$ und $g(x) = \left(1 - \frac{1}{e}\right) \cdot x$ miteinander. Es gilt $f(0) = g(0)$ und $f(1) = g(1)$. Außerdem ist $f(x)$ im Intervall $[0, 1]$ streng konkav und $g(x)$ linear. Daher gilt:

$$\left(1 - \frac{1}{e}\right) \cdot x < 1 - e^{-x} \quad \text{für alle } x \in (0, 1)$$

Für $x = \frac{r(r-1)}{2n} \in (0, 1)$ folgt aus dieser Ungleichung, dass der linke Ausdruck in der Ungleichungskette in (ii) kleiner als der linke Ausdruck in der Ungleichungskette in (i) ist, d. h. (ii) folgt aus (i).

- (iii): Die untere Schranke in (i) liefert $c(n, r) \geq p_0$, falls gilt:

$$1 - e^{-\frac{r(r-1)}{2n}} \geq p_0$$

Diese Ungleichung ist äquivalent zu:

$$r(r-1) \geq -2 \ln(1 - p_0) \cdot n$$

Diese Ungleichung ist sicherlich erfüllt, falls

$$(r-1)^2 \geq -2 \ln(1 - p_0) \cdot n$$

gilt, und diese letzte Ungleichung folgt aus der Annahme in (iii).

(iv): Wie im Beweis zu (iii) gesehen, genügt es sicherzustellen, dass die Ungleichung

$$r(r-1) \geq -2 \ln(1-p_0) \cdot n$$

erfüllt ist. Wegen $r(r-1) = \left(r - \frac{1}{2}\right)^2 - \frac{1}{4}$ ist diese Ungleichung zu der in (iv) angegebenen äquivalent.

(v): Folgt direkt aus (iii) und (iv) mit $p_0 = \frac{1}{2}$.



Beispiel 4.8

Wieviele Kinder müssen mindestens in einer Klasse sein, damit mit einer Wahrscheinlichkeit $p \geq \frac{1}{2}$ mindestens zwei der Kinder am gleichen Tag im Jahr ihren Geburtstag feiern?

Mit Satz 4.3 (v) gilt

$$c(365, r) \geq \frac{1}{2}$$

falls

$$r \geq \sqrt{\ln(4)} \cdot \sqrt{365} + 1 \approx 23,49$$

bzw. sogar nur

$$r \geq \sqrt{\ln(4) \cdot 365 + \frac{1}{4}} + \frac{1}{2} \approx 22,99994$$

gilt. Sind 23 oder mehr Kinder in einer Klasse, so folgt also mit der zweiten Ungleichung, dass die Wahrscheinlichkeit für einen gemeinsamen Geburtstag von zwei Kindern größer als 50% ist.

Die erste Ungleichung in Satz 4.3 (i) liefert für $r = 23$:

$$c(365, 23) \geq 1 - e^{-\frac{506}{730}} \approx 0.50000175$$

Diese Abschätzung liegt übrigens bereits nahe am genauen Wert für die Kollisionswahrscheinlichkeit:

$$c(365, 23) = 1 - \left(1 - \frac{1}{365}\right) \cdots \left(1 - \frac{22}{365}\right) \approx 0.507297$$



Frage 31

Gegeben sei eine Hashfunktion:

$$H: \mathcal{M} \rightarrow \mathbb{Z}_2^l$$

Wie oft muss die Hashfunktion ungefähr ausgeführt werden, um mit einer Wahrscheinlichkeit $p = 0,5$, $p = 0,9$ oder $p = 0,99$ eine Kollision zu finden?

Frage 32

Wie viele Personen müssen auf einer Party anwesend sein, damit mit einer Wahrscheinlichkeit von $p = 0,9$ oder $p = 0,99$ zwei der Gäste am gleichen Tag im Jahr ihren Geburtstag feiern?

4.5 Konstruktionsverfahren von Hashfunktionen

Der grundlegende Aufbau vieler Hashfunktionen basiert auf einer iterativen Anwendung einer sogenannten *Kompressionsfunktion*

$$f : \mathbb{Z}_2^{r+s} \rightarrow \mathbb{Z}_2^s \quad (4.1)$$

auf jeweils r Bits der zu hashenden Nachricht m und den s Bits eines internen Zustandsregisters sowie einer einmaligen Ausführung einer *Ausgabefunktion*:

$$g : \mathbb{Z}_2^s \rightarrow \mathbb{Z}_2^l$$

Die Ausgabefunktion überführt also zum Abschluss der Hashberechnung den Wert des Zustandsregisters in den Hashwert. In vielen Konstruktionen ist $l = s$ und der finale Wert des Zustandsregisters ist bereits der Hashwert, d. h. $g = id_{\mathbb{Z}_2^s}$ oder etwas allgemeiner ist $l \leq s$ und g ist die Projektion von $\mathbb{Z}_2^s$ auf die ersten l Komponenten. (Bei allen in Tabelle 4.7 genannten Hashfunktionen ist dies der Fall.)

Mit f , g und einer Konstante $t_0 \in \mathbb{Z}_2^s$ zur Initialisierung des Zustandsregisters wird der Hashwert $H(m)$ einer Nachricht m folgendermaßen berechnet:

- Für die Nachricht m werden mit einem definierten Paddingverfahren Padding-Bits pad bestimmt, die nur von der Länge von m abhängen und für die gilt: $l(m || pad) = l(m) + l(pad)$ ist ein Vielfaches von r .
- $m || pad$ wird in Blöcke $m_1, \dots, m_n$ zerlegt, die alle r Bit lang sind:

$$m || pad(m) = m_1 || m_2 || \dots || m_n \quad \text{mit } l(m_1) = \dots = l(m_n) = r$$

- In Abhängigkeit von den Blöcken m_i wird der mit t_0 initialisierte Wert des Zustandsregisters iterativ unter Anwendung der Kompressionsfunktion neu berechnet:

$$t_i := f(m_i || t_{i-1}) \quad \text{für } i = 1, 2, \dots, n$$

- Ausgabe des Hashwerts von m :

$$H(m) := g(t_n)$$

Für eine Reihe von Hashfunktionen sind die Bitlängen der Hashwerte, der Blöcke und der Zustandsregister sowie die minimale Anzahl von Padding-Bits in Tabelle 4.7 angegeben.

H	$l = l(H(m))$	$r = l(m_i)$	$s = l(t_i)$	$\min\{l(pad)\}$
MD5	128	512	128	65
SHA-1	160	512	160	65
SHA-224	224	512	256	65
SHA-256	256	512	256	65
SHA-512/224	224	1024	512	129
SHA-512/256	256	1024	512	129
SHA-384	384	1024	512	129
SHA-512	512	1024	512	129
SHA3-224	224	1152	1600	4
SHA3-256	256	1088	1600	4
SHA3-384	384	832	1600	4
SHA3-512	512	576	1600	4

Abb. 4.7 Bitlängen von Hashwerten, Blöcken, Zustandsregistern und dem minimalem Padding für MD5 und für die vom NIST standardisierten SHA-Funktionen

Einige weitere Details zu den Hashfunktionen aus Tabelle 4.7:

- MD5: Zu Beginn der 1990er Jahre wurden von R. Rivest eine Reihe von Hashfunktionen konstruiert. Von diesen fand die Hashfunktion MD5 eine sehr starke Verbreitung, und das, obwohl (zunächst theoretische) Schwächen schon bald nach der Veröffentlichung der Spezifikation gefunden wurden. Spätestens seitdem Methoden zur praktischen Konstruktion von Kollisionen veröffentlicht wurden (s. Beispiel 4.1) sollte auf eine Verwendung von MD5 generell verzichtet werden. Eine Spezifikation von MD5 findet sich in RFC 1321.
- SHA-1: Als Verbesserung von MD5 wurde vom NIST im Jahr 1995 die Hashfunktion (*Secure Hash Algorithm*) SHA-1 im Standard FIPS 180-1 spezifiziert. Der Inhalt des NIST-Standards FIPS 180-1 ist auch als RFC 3174 verfügbar. Da die Kollisionsresistenz von SHA-1 nicht mehr gegeben ist (s. Beispiel 4.2), sollte auch SHA-1 nicht mehr zum Einsatz kommen.
- SHA-2: Die Hashfunktionen der SHA-2-Familie wurden vom NIST in den Jahren 2002 (SHA-256, SHA-384 und SHA-512 erstmals in FIPS 140-2), 2008 (SHA-224 erstmals in FIPS 140-3) und 2015 (SHA-512/224 und SHA-512/256 erstmals in FIPS 140-4) veröffentlicht.

Der Standard FIPS 180-4 enthält die Spezifikation von SHA-1 und aller Hashfunktionen der SHA-2-Familie.

- SHA-3: So wie der AES-Algorithmus im Rahmen eines vom NIST ausgeschriebenen Wettbewerbs zur Spezifikation eines Blockverschlüsselungsalgorithmus auserkoren wurde, so wurde auch der Algorithmus zur Spezifikation einer weiteren Familie von Hashfunktionen im Rahmen eines vom NIST organisierten Wettbewerbs zunächst in einem mehrstufigen Verfahren als Sieger ausgezeichnet. Dieser von G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche entwickelte *Keccak*-Algorithmus bildet die Basis der im NIST-Standard FIPS 202 spezifizierten SHA3-Algorithmen.

MAC-Verfahren

5

5.1	HMAC	72
5.2	CMAC, CBC-MAC	74

Im Studienheft 01 finden sich in den Abschnitten 3.3 und 3.4 einige typische Anwendungen von MAC-Verfahren, die einen „digitalen Fingerabdruck“ von einer Nachricht m berechnen, der nicht nur von der Nachricht selbst, sondern zusätzlich von einem Schlüsselwert k abhängt.

MAC-Verfahren (Message Authentication Code)

Gegeben sei eine Nachrichtenmenge

$$\mathcal{M} = \mathcal{Z}^{\geq 0} \quad \text{oder} \quad \mathcal{M} = \mathcal{Z}^{\leq N} \quad \text{mit} \quad \mathcal{Z} = \mathbb{Z}_2^8$$

und ein Schlüsselraum:

$$\mathcal{K}$$

Eine Abbildung

$$M: \mathcal{K} \times \mathcal{M} \rightarrow \mathbb{Z}_2^l$$

mit einem festen $l \in \mathbb{N}$ wird *MAC-Verfahren* genannt, falls ohne Kenntnis eines Schlüssels $k \in \mathcal{K}$ die Berechnung eines MAC-Wertes

$$t = M_k(m) := M(k, m)$$

für keine Nachricht $m \in \mathcal{M}$ möglich ist. Dies soll effizient auch dann nicht möglich sein, falls bereits andere Paare

$$(m_i, t_i) \quad \text{mit} \quad t_i = M_k(m_i)$$

bekannt sind.

Um einen Schutz gegen *chosen message attacks* gewährleisten zu können, wird zusätzlich gefordert, dass M_k für jedes $k \in \mathcal{K}$ die Eigenschaften einer kryptographischen Hashfunktion besitzt.

5.1 HMAC

HMAC-Verfahren (Keyed-Hashing for Message Authentication)

Jede Hashfunktion h , die auf Grundlage einer Kompressionsfunktion definiert ist (s. Abschnitt 4.5), kann mit der in RFC 2104 (HMAC: Keyed-Hashing for Message Authentication) spezifizierten Konstruktion zu einem MAC-Verfahren erweitert werden.

Ausgehend von einer Hashfunktion H mit einer Kompressionsfunktion wie in (4.1)

$$f: \mathbb{Z}_2^{r+s} \rightarrow \mathbb{Z}_2^s$$

mit Eingabeblöcken der Bitlänge r (mit $r \bmod 8 = 0$) ist das MAC-Verfahren $M = \text{HMAC-}H$ definiert durch:

$$M_k(m) := H\left((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m)\right) \quad (5.1)$$

Hierbei sind opad und ipad fest definierte Bytearrays der Bytelänge $r/8$. Deren konkrete Werte und weitere Details können der Beispielimplementierung aus Abbildung 5.1 entnommen werden.

```

1 public class Hmac_Demo {

    private static final HashMap<String, Integer> blockSize
        = new HashMap();

5
    static {
        blockSize.put("MD5", 64);
        blockSize.put("SHA1", 64);
        blockSize.put("SHA-1", 64);
10        blockSize.put("SHA-224", 64);
        blockSize.put("SHA-256", 64);
        blockSize.put("SHA-512/224", 128);
        blockSize.put("SHA-512/256", 128);
        blockSize.put("SHA-384", 128);
15        blockSize.put("SHA-512", 128);
    }

    public static void main(String[] args) throws Exception {
        byte[] key = Dump.hexString2byteArray(
20            "0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b");
        byte[] m = "Hi There".getBytes();
        byte[] mac = getHashMac("SHA-1", key, m);
        System.out.println(Dump.dump(mac));
    }

25
    // -----
    public static byte[] getHashMac(
        String hashAlgo, byte[] key, byte[] m)
        throws NoSuchAlgorithmException {
30        MessageDigest md = MessageDigest.getInstance(hashAlgo);
        int b = 0;
        try {
            b = blockSize.get(hashAlgo.toUpperCase());
        } catch (Exception ex) {
35            throw new RuntimeException("Input Blocksize of "
                + hashAlgo + " not known!");
        }

        if (key.length > b) {
40            key = md.digest(key);
        }

        // h = H((key XOR ipad) || m)
        for (int i = 0; i < key.length; ++i) {
45            md.update((byte) (key[i] ^ 0x36));
        }
        for (int i = key.length; i < b; ++i) {
            md.update((byte) 0x36);
        }
50        byte[] h = md.digest(m);

        // H((key XOR opad) || h)
        for (int i = 0; i < key.length; ++i) {
55            md.update((byte) (key[i] ^ 0x5C));
        }
        for (int i = key.length; i < b; ++i) {
            md.update((byte) 0x5C);
        }
        return md.digest(h);
60    }
}

```

Abb. 5.1 Quellcode zur Berechnung von HMAC-Werten

5.2 CMAC, CBC-MAC

Dieser Abschnitt enthält einige MAC-Konstruktionen auf Basis von Blockverschlüsselungsverfahren, deren Blocklänge (in Bytes) mit n bezeichnet sei:

$$E : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M} \quad \text{mit} \quad \mathcal{M} := (\mathbb{Z}_2^8)^n$$

CMAC-Verfahren (Cipher-based MAC)

Das CMAC-Verfahren ist im NIST-Standard SP 800-38B (Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication) spezifiziert.

Gegeben sei ein Blockverschlüsselungsverfahren, deren Blocklänge (in Bytes) mit n bezeichnet sei:

$$E : \mathcal{K} \times (\mathbb{Z}_2^8)^n \rightarrow (\mathbb{Z}_2^8)^n$$

Für das MAC-Verfahren

$$M = \text{CMAC-}E : \mathcal{K} \times (\mathbb{Z}_2^8)^{\geq 0} \rightarrow (\mathbb{Z}_2^8)^n$$

ist die Berechnung des MAC-Werts

$$M_k(m) = M(k, m), \quad k \in \mathcal{K}, m \in (\mathbb{Z}_2^8)^{\geq 0}$$

für eine durch ein Bytearray m gegebene Nachricht folgendermaßen definiert.

■ *Padding von m :*

m wird mit einem Bytearray der Form

$$80\ 00 \dots 00$$

erweitert, sodass die Bytelänge des resultierenden Arrays ein Vielfaches der Blocklänge n ist. Bezeichnet $l(m)$ die Bytelänge von m , so gilt:

$$\tilde{m} := \begin{cases} m & \text{falls } l(m) \bmod n = 0 \text{ und } l(m) \neq 0 \\ m \parallel \underbrace{80\ 00 \dots 00}_{n - (l(m) \bmod n)} & \text{falls } l(m) \bmod n \neq 0 \text{ oder } l(m) = 0 \end{cases}$$

■ *Zerlegung von $\tilde{m}$ in Blöcke:*

$$\tilde{m} = m_1 \parallel \dots \parallel m_{r-1} \parallel \tilde{m}_r \quad \text{mit } l(m_1) = \dots = l(m_{r-1}) = l(\tilde{m}_r) = n$$

■ *XOR-Verschlüsselung von $\tilde{m}_r$:*

$$m_r := \begin{cases} \tilde{m}_r \oplus k_1 & \text{falls } l(m) \bmod n = 0 \text{ und } l(m) \neq 0 \\ \tilde{m}_r \oplus k_2 & \text{falls } l(m) \bmod n \neq 0 \text{ oder } l(m) = 0 \end{cases}$$

Hierbei sind k_1 und k_2 Modifikationen des Blocks k_0 , der sich durch Verschlüsselung des Nullblocks ergibt:

$$k_0 := E_k(\mathbf{0}) = E_k(00\ 00 \dots 00)$$

Die durchzuführenden Operationen zur Modifikation von k_0 zu k_1 beziehungsweise k_2 können der Beispielimplementierung aus Abbildung 5.2 entnommen werden.

■ *CBC-Verschlüsselung von $(m_1, \dots, m_{r-1}, m_r)$ mit $IV = \mathbf{0}$:*

$$(c_1, c_2, \dots, c_r) := E_{k, \mathbf{0}}(m_1, \dots, m_r)$$

■ *Ausgabe des MAC-Werts:*

$$M_k(m) = c_r$$


```

1  // CMAC for message m (see NIST SP 800-38B, 6.2)
   public static byte[] getCipherMac(
       String cAlgo, byte[] key, byte[] m)
       throws Exception {
5     Cipher cipher = Cipher.getInstance(cAlgo + "/CBC/NOPADDING");
     int n = cipher.getBlockSize();
     if( n != 8 && n != 16 ) {
         throw new RuntimeException(cAlgo + " not supported:"
10        + " CMAC is not defined for blocksize = " + n);
     }

     SecretKey secretKey = new SecretKeySpec(key, cAlgo);
     IvParameterSpec iv = new IvParameterSpec(new byte[n]);
     cipher.init(Cipher.ENCRYPT_MODE, secretKey, iv);

15     int padLength = n - (m.length % n);
     if( (padLength == n) && (m.length != 0) ) {
         padLength = 0;
     }
20     byte[] mPadded = new byte[m.length + padLength];
     System.arraycopy(m, 0, mPadded, 0, m.length);
     if( padLength != 0 ) {
         mPadded[m.length] = (byte) 0x80;
     }

25     // Calculation of k_1, k_2 (see NIST SP 800-38B, 6.1)
     byte rb = (n == 8) ? 0x1B : (byte) 0x87;
     byte[] k1 = cipher.doFinal(new byte[n]);
     if( leftshift(k1) != 0 ) {
30         k1[n - 1] ^= rb;
     }
     byte[] k2 = k1.clone();
     if( leftshift(k2) != 0 ) {
         k2[n - 1] ^= rb;
35     }

     for( int i = 0; i < n; ++i ) {
         mPadded[mPadded.length - n + i]
40         ^= padLength == 0 ? k1[i] : k2[i];
     }

     mPadded = cipher.doFinal(mPadded);
     byte[] t = new byte[n];
     System.arraycopy(mPadded, mPadded.length - n, t, 0, n);
45     return t;
 }

 // -----
50 private static byte leftshift(byte[] ba) {
     byte overflow = (byte) ((ba[0] >> 7) & 0x01);
     int l = ba.length - 1;
     for( int i = 0; i < l; ++i ) {
         ba[i] <<= 1;
         ba[i] |= ((ba[i + 1] >> 7) & 0x01);
55     }
     ba[l] <<= 1;
     return overflow;
 }

```

Abb. 5.2 Methode getCipherMac(...) zur Berechnung von CMAC-Werten

Kommentar Die Berechnung der von k abgeleiteten Schlüssel k_0 , k_1 und k_2 (Zeilen 27–36 in Abbildung 5.2) kann in einer Anwendung, die mehrere MAC-Berechnungen mit dem gleichen Schlüssel durchzuführen hat, *einmalig* im Voraus durchgeführt werden. ◀

CBC-MAC-Verfahren

Besteht die Nachrichtenmenge nur aus Nachrichten einer festen Bytelänge, die ein Vielfaches b der Blocklänge n ist, so kann anstelle des CMAC-Verfahrens auch das einfachere CBC-MAC-Verfahren benutzt werden. Hierbei wird kein Padding eingesetzt und die XOR-Verknüpfung des letzten Blocks der Nachricht mit einem von k abgeleiteten Schlüssel entfällt:

$$M = \text{CBC-MAC-}E : \mathcal{K} \times (\mathbb{Z}_2^8)^{n/b} \rightarrow (\mathbb{Z}_2^8)^n$$

$$M_k((m_1, \dots, m_b)) := c_b \quad \text{mit } (c_1, c_2, \dots, c_b) = E_{k, \mathbf{0}}(m_1, \dots, m_b)$$

Warnung Essenziell für eine sichere Anwendung des CBC-MAC-Verfahrens ist die genannte Voraussetzung, dass alle Nachrichten die gleiche Länge besitzen, die zudem ein Vielfaches der Blocklänge sein muss.

Sind $m_1, \dots, m_r$ und $\tilde{m}_1, \dots, \tilde{m}_s$ Blöcke, so gilt für $M = \text{CBC-MAC-}E$ beispielsweise:

$$M_k(m) = M_k((m, m \oplus M_k(m)))$$

$$M_k((m_1, \dots, m_r, \mathbf{0}, \tilde{m}_1, \dots, \tilde{m}_s)) = M_k((M_k(m_1, \dots, m_r), \tilde{m}_1, \dots, \tilde{m}_s))$$

